

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ**

**УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ  
(национальный исследовательский университет)»**

Кафедра 806

**УТВЕРЖДАЮ**

Заведующий кафедрой 806 \_\_\_\_\_ Крылов С. С.

(подпись) (инициалы, фамилия)

« \_\_\_\_ » \_\_\_\_\_ 2026 г.

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА МАГИСТРА**

по теме:

**Система автоматизированного обогащения товарных данных для  
интернет-магазина**

Студент: \_\_\_\_\_

(подпись) (фамилия, инициалы)

Группа: \_\_\_\_\_

Научный руководитель: \_\_\_\_\_

(подпись) (фамилия, инициалы, должность)

Москва 2026

## РЕФЕРАТ

**Объём работы.** ≈65 страниц основной части, 4 главы, ≈18 рисунков, ≈21 таблица, 46 источников (10 российских и 36 зарубежных), 6 приложений.

**Ключевые слова.** ОБОГАЩЕНИЕ ТОВАРНЫХ ДАННЫХ, КАСКАДНАЯ АРХИТЕКТУРА, СЛАБЫЙ НАДЗОР, OPEN FOOD FACTS, БАЙЕСОВСКАЯ СЕТЬ, БОЛЬШАЯ ЯЗЫКОВАЯ МОДЕЛЬ, КАЛИБРОВКА, ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ, ДОМИНИРОВАНИЕ ПО ПАРЕТО, АУДИТ-ЦИКЛ, ВЕКТОРНЫЕ ПРЕДСТАВЛЕНИЯ.

**Объект исследования.** Процесс автоматизированного обогащения товарных каталогов интернет-магазинов с использованием открытых источников данных.

**Предмет исследования.** Гибридная каскадная архитектура с ансамблевой оценкой уверенности классификаторов и адаптивным обращением к большой языковой модели как запасному слою.

**Цель работы.** Разработать гибридную каскадную систему автоматизированного обогащения товарных данных для интернет-магазина, обеспечивающую высокую точность извлечения категориальных атрибутов при сокращённой стоимости обращения к большой языковой модели и поддерживающую развёртывание на открытых моделях.

**Методы.** Многоязычные векторные представления текста на базе модели paraphrase-multilingual-MiniLM-L12-v2 (SBERT, поддержка 50+ языков), классификатор XGBoost с регуляризацией и поатрибутными порогами уверенности, байесовская сеть со структурой, найденной алгоритмом Hill Climb по критерию BIC (библиотека pgmpy), доверительные интервалы с кластеризацией по брендам через ресэмплированный бутстрэп, бакетирование числовых нутри-полей, согласие трёх независимых больших языковых моделей (Sonnet 4.5 + GPT-4o + Gemini 2.5 Flash) для построения эталонной разметки на текстово-семантических атрибутах, слепая верификация моделью Claude Opus 4, разбиение на обучающую и тестовую

части без пересечения брендов, предзарегистрированная проверка гипотезы с поправкой Бонферрони на множественные сравнения.

**Результаты.** Реализована и эмпирически проверена гибридная каскадная архитектура из пяти слоёв (категорайзер Layer 0 — регулярные выражения — классификатор XGBoost на векторных представлениях SBERT — байесовский валидатор плотностей — запасной слой большой языковой модели) на трёх продуктовых категориях открытого каталога Open Food Facts (паста, шоколад, сыры). На тестовой выборке без пересечения брендов размером 4350 ячеек по 22 парам «категория-атрибут» итоговая конфигурация достигает 92,8 % сквозной точности при обращении к большой языковой модели на 3,3 % ячеек в режиме «каскад + gemini-flash»; при развёртывании на собственных мощностях с открытой моделью gpt-oss-120b — 92,3 % точности, что на 23,0 п. п. выше прямого использования gpt-oss-120b (69,3 %). Каскад без запасного слоя самостоятельно покрывает 96,7 % ячеек со средней точностью 90,5 %. Достигнуто доминирование по Парето разработанной системы над прямым использованием большой языковой модели по двум осям одновременно — точности и стоимости (+9,0 п. п. точности и сокращение стоимости в 720 раз относительно прямого использования Claude Sonnet 4.5). Предзарегистрированная гипотеза о превосходстве обучаемого стоимостно-чувствительного маршрутизатора над статическим правилом отклонена после поправки Бонферрони на трёх бюджетах. Реализован замкнутый цикл «аудит — правки экстрактора — переобучение — измерение» с кросс-доменной репликацией 3 из 3. Разработана демонстрационная система из трёх компонентов (шлюз API на Go, ML-сервис на Python/FastAPI, веб-интерфейс), реализующая стабильный контракт REST API.

**Использование результатов.** Разработанная архитектура применима в системах управления товарной информацией (PIM-системах) интернет-магазинов. Внедрение системы позволяет: повысить полноту автоматического заполнения карточек товаров до 92,8 %, сократить

стоимость обработки в 720 раз по сравнению с прямым использованием Claude Sonnet 4.5, сократить ручную нагрузку оператора каталога в 10–20 раз за счёт перевода в режим выборочной верификации, обеспечить развёртывание на собственных мощностях без отправки партнёрских данных во внешние API больших языковых моделей.

## ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей выпускной квалификационной работе применяются следующие термины с соответствующими определениями.

**Серебряная разметка** (в литературе по слабому надзору также silver) — набор пар «товар — целевое значение атрибута», полученный автоматически из открытых источников данных (теги Open Food Facts) и используемый для обучения моделей. Серебряная разметка содержит шум и неполноту и противопоставляется эталонной разметке.

**Эталонная разметка** (в литературе также gold) — пары «товар — целевое значение», прошедшие верификацию (согласованным мнением нескольких языковых моделей и/или слепой разметкой более сильной языковой модели); используется как опорная истина при оценке каскада.

**Слабый надзор** — режим обучения с использованием неполной или зашумлённой разметки, формируемой автоматически из доступных источников вместо ручной разметки экспертом.

**Каскадная архитектура** — последовательность алгоритмических слоёв, каждый из которых обрабатывает только те объекты или атрибуты, которые не были закрыты предыдущими слоями с заданной уверенностью.

**Многоуровневая фильтрация по уверенности** — формализация поведения каскада как последовательного конъюнктивного условия на прохождение калиброванных порогов уверенности на каждом слое. Реализует требование утверждённой темы об «ансамблевой оценке уверенности моделей».

**Покрытие** — доля ячеек «товар, атрибут» в тестовой выборке, для которых каскад дал предсказание выше порога уверенности на слоях 1-3 (без обращения к запасному слою).

**Точность на покрытом срезе** — доля верных предсказаний среди тех ячеек, для которых каскад дал ответ без обращения к запасному слою.

**Сквозная точность** — точность системы на полной тестовой выборке, при которой отказы каскада засчитываются как неверные ответы (либо заменяются ответом запасного слоя при включённом Layer 4).

**Разбиение без пересечения брендов** — построение разбиения выборки на обучающую, валидационную и тестовую части, при котором один бренд попадает ровно в одну из трёх частей. Исключает утечку признака «бренд — атрибут» при оценке качества системы.

**Цикл «аудит — правки — переобучение — измерение»** — методологический приём улучшения системы, при котором обнаружение конкретных дефектов эталонной разметки через аудит приводит к правкам экстрактора, затем к переобучению моделей и измерению прироста точности на той же аудит-выборке.

**OFF** — Open Food Facts, открытый краудсорсинговый каталог продовольственных товаров под лицензией ODbL.

**SBERT** — Sentence-BERT, многоязычная модель векторного представления предложений (используется paraphrase-multilingual-MiniLM-L12-v2).

**XGBoost** — Extreme Gradient Boosting, библиотека градиентного бустинга деревьев решений.

**ECE** — Expected Calibration Error, ожидаемая ошибка калибровки вероятностей классификатора.

**CPD/DAG** — Conditional Probability Distribution / Directed Acyclic Graph, элементы байесовской сети.

**BIC** — Bayesian Information Criterion, критерий выбора структуры байесовской сети.

**PDO/AOP** — Protected Designation of Origin / Appellation d'Origine Protégée, защищённое наименование товара.

# ВВЕДЕНИЕ

## Актуальность темы исследования

Электронная коммерция сегодня устойчиво растёт: годовой объём российского сегмента в 2025 году составил 11,5 трлн руб. [1] на фоне продолжающегося роста мирового рынка электронной коммерции [2], а каталоги крупных интернет-магазинов исчисляются десятками миллионов товарных позиций (только индексируемая часть каталога Ozon в открытых аналитических базах превышает 38 млн позиций [3]), к которым ежедневно добавляются тысячи новых записей от партнёров-поставщиков. В этих условиях карточка товара становится центральным элементом цифровой витрины: от того, насколько она содержательно полна, зависят работа фасетного поиска, релевантность ранжирования и рекомендаций, корректность аналитики и в конечном счёте конверсия в покупки. При этом структура партнёрского ввода принципиально неоднородна — от одного наименования и бренда до расширенного набора полей с фотографиями и иноязычным составом. Образующиеся пробелы исторически закрывают контент-менеджеры вручную, проверяя и дополняя десятки целевых атрибутов на каждую карточку; при характерных масштабах каталога в миллионы позиций и тысячах новых записей в сутки такой подход не масштабируется ни по затратам, ни по доступности языковых компетенций [4].

Класс систем управления товарной информацией (Product Information Management) — Akeneo, Pimcore, Salsify и аналогичные — сложился из потребности хранить уже сформированную товарную информацию [5, 6, 7]. При этом задачу извлечения этой информации из неструктурированного партнёрского ввода такие системы систематически не решают: появляющиеся в них модули автоматического обогащения в большинстве случаев представляют собой обёртку над одной публичной большой языковой моделью, вызываемой напрямую на каждый товар (подробное

сопоставление приведено в 1.2.3). Такой вызов экономически приемлем лишь для каталогов небольшого и среднего размера и не сопровождается гарантиями фактической корректности — особенно на числовых полях, в которых большие языковые модели систематически допускают фактические ошибки (3.3.2.5). Более того, рост ставок коммерческих API, ужесточение требований к локализации данных и отраслевой тренд на открытые модели в локальном развёртывании [8] дополнительно усиливают запрос на архитектурные решения, которые не сводятся к прямому вызову одной модели.

Из изложенного следует противоречие, которое и определяет актуальность работы. С одной стороны, автоматизация наполнения карточек — инженерная необходимость. С другой стороны, ни одна изолированная технология не покрывает все типы целевых атрибутов с приемлемой точностью и стоимостью одновременно: правила надёжны на числовых атрибутах, но мало покрывают семантически нагруженные поля; классические классификаторы требуют значительной размеченной выборки; большие языковые модели универсальны, однако дороги и подвержены фактическим ошибкам, особенно на числовых полях. В качестве разрешения этого противоречия в работе рассматривается гибридная каскадная архитектура — композиция последовательных слоёв, в которой каждый следующий слой подключается только к остатку, не закрытому предыдущими с заданной уверенностью. Такая композиция объединяет семантические возможности генеративных моделей с предсказательной силой классического машинного обучения и автоматической фильтрацией недостоверных значений. Это определяет актуальность работы как с научной стороны (систематическая проверка эффективности гибридного каскада), так и с практической (создание производственно-применимого комплекса с контролируемой стоимостью и возможностью локального развёртывания).

## **Цель работы**

Целью настоящей выпускной квалификационной работы является разработка гибридной каскадной системы автоматизированного обогащения товарных данных интернет-магазина. Такая система должна обеспечивать высокую точность извлечения категориальных атрибутов при сокращённой стоимости обращения к большой языковой модели и поддерживать возможность локального развёртывания на открытых языковых моделях. Целевой результат — производственно-применимая система, которая доминирует по Парето над прямым вызовом большой языковой модели по точности и стоимости одновременно и при этом архитектурно независима от конкретного поставщика модели.

### **Задачи работы**

Для достижения поставленной цели сформулированы и решены следующие пять задач:

- 1. Разработать архитектуру гибридного каскадного конвейера** обработки товарных данных с многоуровневой фильтрацией по уверенности и явным разделением слоёв: предкаскадный категорайзер, экстрактор регулярных выражений, классификатор машинного обучения на векторных представлениях, байесовский валидатор плотностей, запасной слой большой языковой модели. Многоуровневая фильтрация формализует требование утверждённой темы об ансамблевой оценке уверенности моделей как последовательное конъюнктивное условие на калиброванные пороги слоёв.
- 2. Реализовать каждый слой каскада в виде самостоятельного программного компонента:** экстрактор регулярных выражений; классификатор машинного обучения, обучаемый на многоязычных векторных представлениях партнёр-доступных полей; байесовский валидатор, использующий обученную сеть атрибутов в роли фильтра статистически невероятных пар «бренд × значение»; запасной слой на

основе большой языковой модели с поддержкой нескольких поставщиков, включая открытые модели для локального развёртывания.

3. **Построить эталонную разметку и проверочную выборку без пересечения брендов** для оценки качества системы на трёх категориях товаров (паста, шоколад, сыры), сочетающую слабый надзор от тегов открытого каталога, поатрибутное согласие трёх независимых больших языковых моделей и слепую разметку референсной моделью, с разбиением на обучающую, валидационную и тестовую части без повторного появления одних и тех же брендов.
4. **Провести экспериментальную оценку разработанной системы** — измерить точность сквозного прохождения каскада, поатрибутное покрытие и стоимость обработки на эталонной тестовой выборке; сопоставить полученные показатели с показателями прямого использования большой языковой модели; оценить вклад каждого слоя каскада, устойчивость к смещению по языкам входных данных и поведение байесовского валидатора как селективного сигнала на отбраковку фактических ошибок.
5. **Разработать демонстрационный программный комплекс**, обеспечивающий взаимодействие конечного пользователя — оператора каталога — с каскадом через веб-интерфейс и стабильный программный интерфейс, и поддерживающий три производственных сценария развёртывания: с премиум-коммерческой большой языковой моделью, с бюджетной коммерческой языковой моделью и с открытой языковой моделью в локальном развёртывании.

Каждой задаче соответствует измеримый результат, который изложен в подразделе «Новизна и основные результаты».

### **Объект и предмет исследования**

**Объектом исследования** в настоящей работе является процесс автоматизированного обогащения товарных каталогов интернет-магазинов на

основе открытых источников данных, при котором на входе имеется минимальный набор партнёр-доступных полей, а на выходе требуется получить структурированный набор целевых атрибутов.

**Предметом исследования** является гибридная каскадная архитектура обработки товарных данных с ансамблевой оценкой уверенности классификаторов и адаптивным селективным обращением к большой языковой модели в роли запасного слоя. Рассматриваются её точность, стоимостные характеристики, устойчивость к смещению входного распределения, а также вычислительные и эксплуатационные свойства, существенные для производственного применения.

### **Теоретические и методические основы работы**

Теоретическую основу работы составили классические результаты теории извлечения признаков из неструктурированного текста, статистического обучения и вероятностных графических моделей. Обучающая выборка строится методом слабого надзора (weak supervision) на тегах открытого каталога Open Food Facts; эталонная разметка для оценки качества формируется сочетанием детерминированных правил для нутриентно-производных атрибутов, регуляторных тегов и поатрибутного согласия трёх независимых больших языковых моделей со слепой проверкой референсной моделью (подробное описание процедуры — 3.2.3). Слой машинного обучения — это градиентный бустинг XGBoost поверх векторных представлений многоязычного дистиллированного энкодера SBERT (paraphrase-multilingual-MiniLM-L12-v2). Слой байесовского валидатора реализован библиотекой pgmpy, причём структура сети обучается алгоритмом Hill Climb по критерию BIC. Применимость изотонической посткалибровки выходов классификатора исследована отдельно в 3.3.3.3 (фиксированная серебряная калибровка деградирует на эталонном срезе из-за дрейфа распределений; перекрёстная калибровка на распределении эталонной выборки снижает среднюю ECE с 0,070 до 0,043 — направление

развития 5.5). Доверительные интервалы метрик качества получены процедурой кластерного бутстрэпа с ресэмплированием брендов; экспериментальный фрагмент организован по принципу предзарегистрированной проверки гипотез с поправкой Бонферрони на множественные сравнения.

Архитектурный подход к стоимостно-чувствительной композиции больших языковых моделей опирается на работы L. Chen и соавторов (FrugalGPT [9]), P. Aggarwal и соавторов (AutoMix [10]), D. Ding и соавторов (Hybrid LLM [11]), а также на бенчмарк Q. J. Hu и соавторов (RouterBench [12]). Методологическое оформление работы выполнено в соответствии с ГОСТ 7.32-2018, а библиографическое описание источников — с ГОСТ 7.1-2003.

### **Новизна и основные результаты**

Каждой из сформулированных выше задач в работе соответствует не менее одного измеримого результата.

**По задаче 1 (проектирование архитектуры каскада)** — разделы 2.1, 3.1.1 — спроектирован гибридный каскад из пяти последовательных слоёв с конъюнктивным условием на калиброванные пороги уверенности каждого слоя. Разработанная формальная постановка задачи допускает явный отказ системы от ответа и явное определение слоя-источника каждого предсказания.

**По задаче 2 (реализация слоёв)** — раздел 3.1.2 — каскад реализован в виде набора независимо обновляемых программных компонентов; разработана демонстрационная система в трёхкомпонентной архитектуре «шлюз API на языке Go — ML-сервис на Python/FastAPI — статический веб-интерфейс»; система запускается одной командой и проходит проверку готовности всех слоёв через диагностический интерфейс.

**По задаче 3 (построение эталонной разметки)** — раздел 3.2 — построена многоисточниковая эталонная разметка для трёх категорий

товаров на основе тегов открытого каталога, нутриентно-производных правил, поатрибутного согласия трёх независимых больших языковых моделей и слепой разметки референсной моделью; разбиение на обучающую, валидационную и тестовую части выполнено без пересечения брендов.

**По задаче 4 (экспериментальная оценка)** — разделы 3.3.2, 3.3.3, 3.3.4, 3.3.7 — получена сквозная точность 92,8 % на тестовой выборке без пересечения брендов из 4350 ячеек при 720-кратном сокращении стоимости относительно прямого вызова коммерческой референсной модели; доминирование каскада по Парето над прямым использованием большой языковой модели подтверждено на пяти моделях запасного слоя; вклад каждого слоя количественно охарактеризован, в том числе селективный вклад байесовского валидатора. Каскад без запасного слоя самостоятельно покрывает 96,7 % ячеек со средней точностью 90,5 %.

**По задаче 5 (разработка демонстрационного программного комплекса)** — разделы 4.1, 4.2 — реализована поддержка трёх производственных сценариев развёртывания (премиум-коммерческий, бюджетный коммерческий, локальный с открытой моделью) и трёх ролей пользователей: контент-менеджер, системный аналитик, инженер машинного обучения.

На основании совокупности полученных результатов сформулированы три пункта научной новизны.

**Первый пункт научной новизны.** Доказано доминирование разработанной гибридной каскадной системы по Парето над прямым использованием большой языковой модели на задаче обогащения товарных каталогов: на тестовой выборке без пересечения брендов размером 4350 ячеек одновременно достигаются превышение точности на 9,0 процентного пункта (92,8 % против 83,8 %) и сокращение стоимости обработки в 720 раз — без компромисса между точностью и стоимостью.

**Второй пункт научной новизны.** Эмпирически установлено, что в сценарии локального развёртывания каскадной системы с открытой большой

языковой моделью (gpt-oss-120b) в роли запасного слоя каскад компенсирует ограниченные возможности малой открытой модели на сложных атрибутах за счёт специализированного слоя машинного обучения и обеспечивает прирост сквозной точности на 23,0 процентного пункта над прямым использованием той же открытой модели при тридцатикратном сокращении стоимости. Это позволяет говорить об архитектурной независимости решения от конкретного поставщика большой языковой модели.

**Третий пункт научной новизны.** Впервые на задаче обогащения товарных каталогов по предзарегистрированной процедуре с поправкой Бонферрони на множественные сравнения количественно установлено, что статическое per-(категория, атрибут) правило маршрутизации не уступает обучаемому стоимостно-чувствительному маршрутизатору по сквозной точности при равных бюджетах вызовов запасного слоя. Полученный результат уточняет применимость подхода обучаемой маршрутизации к рассматриваемому классу задач и согласуется с выводами бенчмарка RouterBench [12]. Кроме того, в работе разработан и кросс-доменно реплицирован замкнутый цикл «аудит эталонной разметки — корректировка экстрактора признаков — переобучение моделей — повторное измерение»: цикл воспроизведён на трёх категориях товаров из трёх с приростом сквозной точности на подвыборке аудированных проблемных ячеек +18,2, +16,3 и +14,4 процентного пункта для пасты, шоколада и сыров соответственно.

### **Апробация и внедрение результатов**

Апробация разработанной системы выполнена в виде демонстрационного программного комплекса, который запускается одной командой и проверяется на работоспособность через диагностический программный интерфейс. Демонстрация подтверждает, что все слои каскада функционируют на трёх предметных категориях (паста, шоколад, сыры) и в трёх сценариях обращения к запасному слою (коммерческая, бюджетная,

открытая большая языковая модель). Воспроизводимость численных показателей обеспечена сценарием `reproduce.sh`, который регенерирует основные результаты из артефактов в каталоге `datasets/processed/` репозитория. Разработанный комплекс представлен как самостоятельный демонстрационный артефакт, готовый к интеграции в производственный цикл интернет-магазина без перестройки соседних звеньев процесса обогащения каталога (4.2.1).

### **Структура и объём работы**

Выпускная квалификационная работа состоит из введения, четырёх глав, заключения, списка использованных источников и приложений. Список использованных источников включает 46 наименований (10 российских источников, индексируемых в РИНЦ, и 36 зарубежных публикаций, отраслевых отчётов и стандартов). Первая глава содержит обоснование актуальности темы, литературный обзор по трём направлениям и техническое задание на разрабатываемую систему. Вторая глава посвящена теоретическому обоснованию решения: в ней приводятся формальная постановка задачи, обоснование архитектуры и стека технологий, а также сценарии прохождения запроса через каскад. Третья глава содержит описание программной реализации, использованных данных и доказательство работоспособности каскада на эталонной тестовой выборке без пересечения брендов. Четвёртая глава представляет порядок применения системы, сценарии её развёртывания и характеристику достигнутых результатов в формате «повышено / ускорено / сокращено». Наконец, в заключении собраны выводы по главам, научная новизна и перспективы развития темы.

# **ГЛАВА 1. РАЗВЁРНУТАЯ АКТУАЛЬНОСТЬ ТЕМЫ**

Глава посвящена тому, чтобы обосновать тему работы, систематизировать уже существующие подходы к автоматическому обогащению товарных данных и сформулировать техническое задание на разрабатываемую систему.

## **1.1 Проблема обогащения товарных каталогов интернет-магазина**

### **1.1.1 Электронная коммерция и роль товарных данных**

Российский и мировой рынки электронной коммерции устойчиво растут: годовой оборот российского сегмента в 2025 году достиг 11,5 трлн руб. [1] на фоне продолжающегося роста мирового рынка [2]. Каталоги крупных площадок исчисляются десятками миллионов позиций (только индексируемая часть каталога Ozon в открытых аналитических базах превышает 38 млн позиций [3]), и ежедневно к ним добавляются тысячи новых артикулов товаров от партнёров-поставщиков. В таких условиях карточка товара становится центральным элементом цифровой витрины: от её структурированного набора атрибутов (тип, состав, габариты, маркировки качества, страна происхождения, предупреждения) зависит работа фасетного поиска, релевантность ранжирования и рекомендаций, точность аналитики и конверсия. Стоит отметить, что при неполном заполнении атрибутов цепочка «партнёрский фид — наполнение каталога — редакторский контроль — витрина» прерывается во втором звене: товар либо попадает на витрину в неполной форме — и тогда страдают поиск и конверсия, — либо задерживается до ручной доработки, отчего растут операционные затраты.

### **1.1.2 Неоднородность партнёрского ввода**

Партнёрские данные принципиально неоднородны: для одного товара поставщик передаёт лишь наименование и бренд, для второго — расширенный набор полей с фотографиями, для третьего — состав произвольным текстом на иностранном языке, а для четвёртого присутствует

только штрихкод. Заполненность нутри-полей упакованной пищевой продукции, как показывает анализ открытого каталога Open Food Facts [13], систематически ниже заполненности базовых полей наименования и бренда. Образующиеся пробелы исторически закрываются контент-менеджерами вручную, проверяя и дополняя десятки целевых атрибутов на каждую карточку, и при масштабах каталога в миллионы позиций и тысячах новых записей в сутки такой путь экономически неприемлем [4]. Более того, ручной труд плохо масштабируется по языкам: контент-команда, работающая с русскоязычным и англоязычным контентом, теряет качество, когда поступают описания на французском, немецком, итальянском или испанском. Особенно заметно это в категориях пищевой продукции, импортируемой из европейских стран.

### **1.1.3 Эффекты неполной карточки на пользовательский опыт**

Неполная карточка оказывает три независимых воздействия на пользовательский опыт и экономику магазина. **Первое** — снижается качество фасетного поиска: если хотя бы один атрибут не заполнен, товар выпадает из релевантных подборок, когда покупатель применяет соответствующие фильтры (например, упаковка пасты без указания формы или типа зерна). **Второе** — растут возвраты и негативная обратная связь; это особенно критично для атрибутов с правовым или диетическим значением, таких как органическое происхождение, отсутствие глютена, аллергены, поскольку покупатель фиксирует расхождение между обещанием карточки и фактическим товаром. **Третье** — возникает нагрузка на инфраструктуру модерации: «брошенная» карточка либо удлиняет цикл выхода товара на витрину, либо становится источником повторных правок.

### **1.1.4 Существующие системы управления товарной информацией не решают задачу**

Класс систем управления товарной информацией (Product Information Management, PIM-системы) — например, такие как Akeneo, Pimcore, Salsify,

Riversand, inRiver — сложился из корпоративной потребности хранить, согласовывать и публиковать товарную информацию в едином каталоге [5, 6, 7]. Однако функциональность PIM-систем сосредоточена на управлении уже сформированной информацией, а не на её извлечении из неструктурированного партнёрского ввода. Появляющиеся модули автоматического обогащения в большинстве случаев представляют собой обёртку над одной публичной большой языковой моделью с прямым вызовом на каждый товар; это экономически приемлемо лишь для каталогов небольшого и среднего размера и не сопровождается гарантиями фактической корректности. Альтернативой могла бы выступать интеграция с агрегаторами готовых описаний товаров от вендоров — однако такие агрегаторы не решают задачу извлечения атрибутов из произвольного партнёрского описания, а их покрытие ограничено крупными брендами.

### **1.1.5 Противоречие и постановка задачи**

Изложенное определяет противоречие, которое и лежит в основе актуальности работы. С одной стороны, автоматизация наполнения карточек — инженерная необходимость. С другой стороны, ни одна изолированная технология не покрывает все типы целевых атрибутов с приемлемой точностью и стоимостью одновременно: правила надёжны на числовых атрибутах, но дают низкое покрытие на семантически нагруженных полях; классические классификаторы требуют значительной размеченной выборки и плохо переносятся между языками входных данных; большие языковые модели универсальны, но дороги [9] и систематически допускают фактические ошибки на числовых полях, требующих агрегирования из текста (эмпирическая характеристика — 3.3.2.5). Из противоречия вытекает рабочая архитектурная гипотеза: каскадная композиция слоёв обработки, в которой каждый следующий слой подключается только к остатку, не закрытому предыдущими с заданной уверенностью, способна совместить преимущества отдельных технологий. Проверка этой гипотезы и составляет содержательное ядро работы.



## 1.2 Литературный обзор

В настоящем разделе систематизируется научный и индустриальный задел по трём направлениям, которые образуют теоретическую основу разрабатываемой системы: извлечение признаков из текстовых описаний товаров, стоимостно-чувствительные каскады больших языковых моделей и существующие промышленные системы управления товарной информацией.

### 1.2.1 Извлечение признаков из текстовых описаний товаров

В академической литературе задача восполнения пропущенных атрибутов в товарных каталогах рассматривается как частный случай задачи извлечения признаков из неструктурированного текста (information extraction, IE). Сложившиеся подходы делятся на три группы: формальные правила, классические статистические модели и нейросетевые модели на основе трансформеров. Стоит отметить, что эти группы не являются взаимоисключающими; именно их сочетание и образует основу для каскадных архитектур, которые рассматриваются ниже.

**Правила и регулярные выражения.** Формальные шаблоны надёжно извлекают атрибуты с устойчивой лексической разметкой, например, такие как процент жира, масса нетто, время приготовления, минимальный возраст; стоимость инференса при этом близка к нулю, а интерпретируемость обеспечивается по построению. Главное ограничение — низкое покрытие на атрибутах, которые требуют семантического понимания, таких как тип крупы, форма пасты, маркировка органического происхождения.

**Статистические модели над поверхностными признаками.** Классификаторы поверх TF-IDF-векторизации и градиентный бустинг деревьев решений (XGBoost [14], LightGBM [15]) хорошо работают при достаточной обучающей выборке и одноязычном входе; результаты применения методов машинного обучения к прогнозированию бизнес-решений и классификации в системах электронной коммерции приведены в работе Г. И. Афанасьева и соавторов [16]. Однако такие модели слабо

переносятся на многоязычные тексты и не способны учитывать семантическое сходство.

**Векторные представления на основе трансформеров.** Архитектура трансформера [17] и модели семейства BERT [18] обеспечили новое качество в задачах извлечения текста. Sentence-BERT [19] и его многоязычные дистиллированные варианты [20] позволяют получать векторные представления предложений фиксированной размерности, поверх которых применяется произвольный классификатор; задачи автоматической категоризации и извлечения признаков из товарных каталогов рассмотрены в работе А. Г. Колонина [21], а систематическое изложение методов автоматической обработки естественного языка и извлечения информации дано в учебнике Е. И. Большаковой и соавторов [22].

**Специализированные системы извлечения атрибутов товаров.** В индустриальной литературе крупных торговых площадок выделяются три эталонные системы:

- TXtract [23] (Karamanolakis et al., ACL 2020, Amazon) — многозадачный нейронный кодировщик с условием на таксономию категорий, обеспечивающий извлечение атрибутов в крупных каталогах при значительном дисбалансе классов;
- MAVE [24] (Yang et al., WSDM 2022, Google) — многоисточниковый кодировщик на основе BERT с кросс-вниманием, использующий не только наименование, но и описание, спецификации и другие текстовые поля карточки;
- OpenTag [25] (Zheng et al., KDD 2018) — двунаправленная рекуррентная архитектура BiLSTM-CRF с механизмом внимания, формулирующая задачу извлечения атрибутов как задачу разметки последовательности.

Перечисленные системы демонстрируют качество извлечения отдельных атрибутов в диапазоне 85–92 % F1 на специализированных корпусах. При этом сопоставление этих подходов с каскадными архитектурами, привлекающими большую языковую модель в роли

запасного слоя, в открытой литературе систематически не представлено: специализированные системы и каскады LLM рассматриваются как самостоятельные направления. Настоящая работа частично закрывает этот пробел.

### **1.2.2 Стоимостно-чувствительные каскады больших языковых моделей**

С появлением спектра больших языковых моделей разной мощности и стоимости в литературе оформилась подобласть стоимостно-чувствительной маршрутизации (cost-aware LLM routing). Её задача — направлять каждый запрос к минимально достаточной модели так, чтобы не потерять качество относительно самой мощной модели.

**FrugalGPT.** Главный референс направления — работа L. Chen, M. Zaharia и J. Zou [9], в которой предложена каскадная композиция языковых моделей по возрастанию стоимости с прерыванием на первом достаточно уверенном ответе; каскад из трёх-четырёх моделей с откалиброванной уверенностью обеспечивает то же качество, что и прямой вызов самой дорогой модели, при стоимости на порядок ниже. Настоящая работа опирается на идеи FrugalGPT в части последовательного прерывания и калибровки уверенности.

**AutoMix.** Работа P. Aggarwal, A. Madaan и соавторов [10] основана на самопроверке ответа моделью меньшего размера: при сомнении модель сама делегирует запрос вышестоящей. Подход реализуется внутри пары «малая LLM — большая LLM», тогда как разрабатываемая система, наоборот, выносит роли валидатора и предсказателя в отдельные слои каскада.

**Hybrid LLM.** Работа D. Ding, A. Mallick и соавторов [11] рассматривает маршрутизацию между моделью на пограничном устройстве и облачной моделью; сценарий настоящей работы «on-premise gpt-oss-120b плюс коммерческое API как запасной слой» прямо соответствует этой постановке.

**RouterBench.** Бенчмарк Q. J. Hu, J. Bieker и соавторов [12] систематически измерил эффективность обучаемых маршрутизаторов между большими языковыми моделями и показал, что обучаемые маршрутизаторы не доминируют над простыми статическими политиками на значительной части задач при локализованной структуре ошибок. Этот вывод согласуется с результатом главы 3, где на задаче обогащения товарных каталогов впервые количественно установлено и проверено по предзарегистрированной процедуре, что статическое per-(категория, атрибут) правило не уступает обучаемому маршрутизатору по сквозной точности при равном бюджете вызовов запасного слоя.

### 1.2.3 Существующие промышленные PIM-системы и анализ аналогов

Чтобы позиционировать разработку относительно индустриальных аналогов, отобраны три коммерческие PIM-системы — Akeneo, Pimcore и Salsify — и построена сравнительная таблица по семи функциональным критериям, существенным для задачи автоматического обогащения каталога (таблица 1.1; источники — документация вендоров [5, 6, 7]).

**Таблица 1.1 — Сравнение существующих PIM-систем и разрабатываемой системы**

| № | Критерий                            | Akeneo                       | Pimcore                       | Salsify                                           | Разрабатываемая система    |
|---|-------------------------------------|------------------------------|-------------------------------|---------------------------------------------------|----------------------------|
| 1 | Централизованное хранение каталога  | да                           | да                            | да                                                | не входит в задачу         |
| 2 | Распределение по каналам сбыта      | да                           | да                            | да                                                | не входит в задачу         |
| 3 | Автоматическое обогащение атрибутов | нет (только ручная редакция) | частично (правила обогащения) | нет (есть AI Suite как обёртка над сторонней LLM) | да (каскадная архитектура) |

|   |                                                              |                             |                          |                             |                                                                                                         |
|---|--------------------------------------------------------------|-----------------------------|--------------------------|-----------------------------|---------------------------------------------------------------------------------------------------------|
| 4 | Извлечение признаков<br>из<br>неструктурированного<br>текста | нет                         | нет                      | нет                         | да (слои ML и LLM)                                                                                      |
| 5 | Поддержка<br>многоязычного входа                             | через<br>ручные<br>переводы | через ручные<br>переводы | через<br>ручные<br>переводы | <b>встроенная<br/>(многоязычные<br/>эмбединги, 50+<br/>языков)</b>                                      |
| 6 | Открытый исходный<br>код                                     | community-<br>редакция      | да                       | нет                         | <b>да (включая<br/>возможность on-<br/>premise<br/>развёртывания<br/>LLM)</b>                           |
| 7 | Стоимостно-<br>чувствительная<br>обработка                   | нет                         | нет                      | нет                         | <b>да (каскадная<br/>композиция с<br/>доминированием<br/>по Парето по<br/>стоимости и<br/>качеству)</b> |

Из таблицы 1.1 следует, что коммерческие PIM-системы хорошо покрывают хранение, версионирование и распределение уже сформированной информации, однако систематически не решают задачи извлечения атрибутов из произвольного партнёрского текста, а появляющаяся функция автоматического обогащения реализуется как обёртка над сторонней языковой моделью. Таким образом, ни одно из проанализированных решений не закрывает совокупность критериев интернет-магазина одновременно — автоматическое обогащение, многоязычную обработку, контролируемую стоимость и возможность on-premise развёртывания, — и это обосновывает целесообразность разработки самостоятельной системы.

#### 1.2.4 Слабый надзор, калибровка и сопутствующая методология

Принципиальная сложность построения систем автоматического обогащения состоит в том, что отсутствует достаточно большая ручная разметка целевых атрибутов; соответствующая постановка обозначается термином «слабый надзор» (weak supervision), методический инструментарий которого систематизирован в работе А. Ratner и соавторов «Snorkel» [26]. Пригодным источником слабого эталона для товарных каталогов выступает Open Food Facts [13] — открытая база более 4,5 млн записей о пищевых продуктах с пользовательскими тегами.

Чтобы каскад работал корректно, выходы классификаторов должны быть откалиброваны. Базовыми процедурами калибровки выступают масштабирование Платта [27] и изотоническая регрессия [28]; современный анализ калибровки глубоких моделей и поведения метрики Expected Calibration Error (ECE) приведён в работе С. Guo и соавторов [29]. Точность при ограниченной выборке оценивается кросс-валидацией и бутстрэпом [30], а интервальная оценка пропорций строится по формуле Вильсона [31]. В части аппарата байесовских сетей работа опирается на монографии J. Pearl [32], D. Heckerman et al. [33], А. Л. Тулупьева и соавторов [34] и библиотеку pgmpy [35]; для поиска структуры применяется алгоритм Hill Climb с критерием BIC [36]. Прикладные вопросы построения рыночно-ориентированных моделей машинного обучения и устойчивости к дрейфу данных в розничных каталогах рассмотрены А. А. Артемовым [37], а отраслевой контекст ограниченной масштабируемости стандартных подходов к управлению ассортиментом на маркетплейсах — С. А. Сергеевым [38]. Активное обучение и краудсорсинг как методы оптимизации разметки в условиях ограниченного бюджета систематизированы Р. А. Гилязовым и Д. Ю. Турдаковым [39], а ранние работы по семантическим пространствам знаний — В. Ф. Хорошевским [40].

### 1.3 Техническое задание на разрабатываемую систему

В настоящем разделе формулируется техническое задание на разрабатываемую систему. Оно подразделяется на функциональные требования (что система должна делать), нефункциональные (с какими характеристиками) и архитектурные (каким способом она организована).

#### 1.3.1 Функциональные требования

**Ф-1. Принимаемый формат входных данных.** Система должна принимать на входе товар, описанный полями, доступными от партнёра-поставщика: наименование (`product_name`), бренд (`brands`), список ингредиентов в произвольной форме (`ingredients_text`), масса / объём / упаковка (`quantity`), идентификатор категории (`category`, опционально). Все поля, кроме наименования, считаются необязательными.

**Ф-2. Возврат заполненных значений целевых атрибутов.** Система должна возвращать для каждого целевого атрибута предсказанное значение, оценку уверенности и идентификатор слоя каскада, на котором было принято финальное решение. Тем самым обеспечивается отслеживаемость источника каждого предсказания и возможность ручной верификации контент-менеджером.

**Ф-3. Покрытие предметных категорий.** В пределах настоящей работы система поддерживает заполнение целевых атрибутов для трёх категорий пищевой продукции: паста, шоколад и шоколадные изделия, сыры. Выбор именно этих трёх категорий обоснован тем, что в Open Food Facts по ним имеется достаточный объём обучающих данных, доступен аудит-эталон с согласием трёх больших языковых моделей и слепой разметкой Orpus, а сами категории репрезентативны с точки зрения распределения типов целевых атрибутов — числовых, бинарных, мультиклассовых семантических.

**Ф-4. Автоматическое определение категории.** Если категория явно не указана, система должна определять её автоматически — на основании

поступившего текстового описания товара. Эта функция реализуется отдельным предкаскадным слоем (Layer 0).

**Ф-5. Запасной слой на основе большой языковой модели.** Если получить однозначный ответ на конкретный атрибут средствами слоёв 1–3 каскада не удаётся, система должна обращаться к большой языковой модели как к запасному слою. При этом запасной слой подключается селективно — только для тех атрибутов конкретного товара, по которым предыдущие слои не выдали уверенного ответа.

**Ф-6. Диагностический вывод.** Система должна возвращать диагностическую информацию: результат работы валидатора (понижение или подтверждение предсказания), агрегированные оценки уверенности по слоям и маршрут запроса по каскаду. Эта информация необходима для отладки, аудита и накопления данных о поведении системы в эксплуатации.

### 1.3.2 Нефункциональные требования

**НФ-1. Точность.** Целевая сквозная точность каскада на эталонной тестовой выборке без пересечения брендов между обучающей и тестовой частями — не менее 85 %; в конфигурации с коммерческой большой языковой моделью в запасном слое — не менее 90 %.

**НФ-2. Стоимость.** Стоимость обработки одного товара каскадом должна быть существенно ниже стоимости прямого вызова большой языковой модели на тот же товар. Целевой коэффициент снижения — не менее 2,5×; ограничение сверху не задаётся, фактически достижимое значение фиксируется по результатам эмпирической оценки в 3.3.2.

**НФ-3. Производительность.** Время отклика на один товар при работе без обращения к запасному слою — не более 500 мс на потребительском ноутбуке без специализированного ускорителя; при необходимости обращения к запасному слою — не более 5 секунд (определяется временем ответа внешнего API большой языковой модели).

**НФ-4. Многоязычность.** Система должна поддерживать обработку описаний товаров на пяти рабочих языках европейского рынка пищевой

продукции: французском, английском, немецком, итальянском и испанском. Это соответствует наиболее представленным языкам в Open Food Facts и моделирует реалистичный многоязычный партнёрский поток.

**НФ-5. Возможность on-premise развёртывания.** Система должна поддерживать сценарий полностью локального развёртывания, при котором обращения к внешним коммерческим API исключаются. Этот сценарий реализуется через подключение в запасной слой открытой языковой модели (gpt-oss-120b) и предъявляет к каскаду требование вендорной независимости.

### 1.3.3 Архитектурные требования

**А-1. Каскадная архитектура с отдельными слоями.** Система реализует последовательность слоёв обработки по принципу «передачи остатка»: каждый последующий слой обрабатывает только те атрибуты, по которым предыдущие слои не выдали ответа с уверенностью выше калиброванного порога. При этом архитектурно слои разделены и могут обновляться независимо.

**А-2. Независимое обновление слоёв.** Изменение или переобучение любого отдельного слоя — обновление правил, дообучение классификаторов слоя 2, замена языковой модели в запасном слое — не должно требовать перезапуска или повторного развёртывания других слоёв, а также API-шлюза системы.

**А-3. Стабильный контракт API.** Внешний интерфейс системы — формат запроса, формат ответа, перечень полей — фиксируется на этапе разработки и не изменяется в течение производственного цикла. Тем самым система интегрируется с внешними каталогами и РИМ-системами без многократного переписывания клиентского кода.

**А-4. Демонстрационный программный комплекс с веб-интерфейсом.** Система сопровождается демонстрационным комплексом, который состоит из трёх компонентов: API-шлюз на языке Go, ML-сервис на языке Python (FastAPI) и одностраничный веб-интерфейс. Демонстрация

обеспечивает верификацию работоспособности и наглядно представляет результаты работы каскада для контент-менеджера.

#### 1.3.4 Соотношение технического задания и решаемых задач работы

Сформулированные требования полностью покрываются теми задачами, которые поставлены во введении (таблица 1.2).

**Таблица 1.2 — Соответствие требований технического задания и задач работы**

| № задачи | Краткая формулировка задачи                                                | Покрываемые требования ТЗ                                      |
|----------|----------------------------------------------------------------------------|----------------------------------------------------------------|
| 1        | Разработать архитектуру гибридного каскадного конвейера                    | Ф-1, Ф-2, Ф-4, Ф-5, А-1                                        |
| 2        | Реализовать каждый слой каскада как самостоятельный программный компонент  | Ф-6, А-2, А-3                                                  |
| 3        | Построить эталонную разметку и проверочную выборку без пересечения брендов | Ф-3 (в части обеспечения данных для оценки на трёх категориях) |
| 4        | Провести экспериментальную оценку точности, покрытия и стоимости           | НФ-1, НФ-2, НФ-3, НФ-4, НФ-5                                   |
| 5        | Разработать демонстрационный программный комплекс                          | А-4                                                            |

Таким образом, установленное соответствие подтверждает, что выполнение пяти задач работы влечёт за собой выполнение технического задания в полном объёме.

## 1.4 Выводы по главе 1

В главе проведён анализ предметной области, систематизировано состояние литературы и обосновано техническое задание. По результатам анализа можно сформулировать следующие выводы.

**6. Проблема автоматического обогащения остаётся открытой.**

Промышленные PIM-системы, такие как Akeneo [5], Pimcore [6], Salsify [7], ориентированы на хранение и распределение уже сформированной информации, а появляющиеся модули автоматического обогащения реализуются как обёртки над одной коммерческой большой языковой моделью и не отвечают требованиям стоимостно-чувствительной обработки больших каталогов.

**7. Научные подходы делятся на две группы, и ни одна из них не**

**покрывает задачу полностью.** Специализированные системы извлечения атрибутов, такие как TXtract [23], MAVe [24], OpenTag [25], решают задачу с высоким качеством, однако требуют значительной размеченной выборки и не учитывают экономики обращения к моделям. Стоимостно-чувствительные каскады больших языковых моделей — FrugalGPT, AutoMix, Hybrid LLM — дают инструменты управления стоимостью, систематически сопоставленные в бенчмарке RouterBench; однако эти каскады рассматривают композицию из однородных моделей и не интегрируют классическое машинное обучение.

**8. Разрабатываемая система объединяет оба подхода в гибридный**

**каскад, который сочетает специализированные слои извлечения признаков — правила, машинное обучение на многоязычных эмбедингах, байесовский валидатор — и стоимостно-чувствительный запасной слой LLM. Совокупность шести функциональных, пяти нефункциональных и четырёх архитектурных требований ТЗ формирует обязательство, выполнение которого верифицируется в экспериментальной части работы.**

## ГЛАВА 2. ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РЕШЕНИЯ

В этой главе формально описывается разрабатываемая система и обосновываются принятые архитектурные решения. Сначала приводится формальная постановка задачи и логика работы программного обеспечения (2.1), затем рассматриваются архитектура и стек технологий с обоснованием выбора каждой компоненты (2.2), и наконец разбирается логика прохождения типового запроса через систему (2.3).

### 2.1 Логика работы разрабатываемого программного обеспечения

#### 2.1.1 Формальная постановка задачи обогащения

Пусть товарная позиция, поступающая в систему от партнёра, представлена кортежем партнёр-доступных полей

$$x = (x^{\text{name}}, x^{\text{brand}}, x^{\text{ingr}}, x^{\text{qty}}),$$

где  $x^{\text{name}}$  — название товара,  $x^{\text{brand}}$  — указанный бренд,  $x^{\text{ingr}}$  — текст состава,  $x^{\text{qty}}$  — поле количества. Любое из этих полей, кроме названия, может быть пустым. Именно в этом и состоит задача обогащения: требуется восстановить недостающие структурированные характеристики товара по тому минимуму, который партнёр предоставил.

Каждому товару ставится в соответствие категория  $c \in \mathcal{C} = \{c_1, \dots, c_M\} \cup \{\text{OOD}\}$ , где  $\mathcal{C}$  — конечное множество поддерживаемых системой доменов (в настоящей работе  $M = 3$ : паста, шоколад, сыры), а метка  $\text{OOD}$  (out-of-distribution — выход за пределы распределения обучающей выборки) соответствует ситуации, когда товар не относится ни к одной из поддерживаемых категорий. Категорию определяет отдельная модель

$$c = f_0(x),$$

описанная в 2.2.1 как Layer 0 (категорайзер). Для каждой категории  $\$c\$$  задана конечная схема целевых атрибутов  $\mathcal{A}_c = \{a_1, \dots, a_{K_c}\}$  с областями значений  $\mathcal{Y}_k$ . Таким образом, для известного  $x$  и определённого  $c$  система должна построить вектор предсказаний

$$\hat{y} = (\hat{y}_1, \dots, \hat{y}_{K_c}) \in (\mathcal{Y}_1 \cup \{\text{null}\}) \times \dots \times (\mathcal{Y}_{K_c} \cup \{\text{null}\}),$$

где значение  $\hat{y}_k = \text{null}$  означает явный отказ системы от ответа на  $k$ -м атрибуте. Возможность отказа здесь принципиальна и закладывается в архитектуру изначально. Применительно к задаче обогащения каталога честное молчание системы предпочтительнее неверного заполнения: повторное обращение к оператору обходится дешевле, чем ошибка, которую потом обнаружит конечный пользователь.

Опорное (эталонное) значение  $k$ -го атрибута для товара  $i$  обозначается  $y_k^{(i)}$ . Товары с известным эталоном составляют проверочную выборку и используются для расчёта метрик качества (2.1.4).

### 2.1.2 Каскадная схема принятия решения

Логика обогащения реализована как каскад из четырёх содержательных слоёв, которому предшествует определение категории. Для каждого атрибута  $k$  итоговое предсказание формируется по правилу

$$\begin{aligned} \hat{y}_k = & \begin{cases} f_k^{\text{regex}}(x), & \text{если } f_k^{\text{regex}}(x) \neq \text{null}; \\ f_k^{\text{ML}}(x), & \text{иначе если } P_{\text{ML}}(\hat{y}_k \mid x) \geq \tau_k \text{ и } P_B(\hat{y}_k \mid \mathbf{e}) \geq \theta_k; \\ f_k^{\text{LLM}}(x), & \text{иначе.} \end{cases} \end{aligned}$$

Здесь использованы следующие обозначения:

- $f_k^{\{\text{regex}\}}$  — экстрактор регулярных выражений (Layer 1), детерминированная функция от текстовых полей; считается, что слой «не сработал», если ни один шаблон не дал совпадения;
- $f_k^{\{\text{ML}\}}$  — классификатор машинного обучения (Layer 2) над многоязычным векторным представлением партнёр-доступных полей;
- $P_{\{\text{ML}\}}(\hat{y}_k \mid x)$  — апостериорная вероятность класса по оценке классификатора после калибровки;
- $\tau_k$  — поатрибутный порог уверенности классификатора, подобранный на отложенной части обучающей выборки;
- $P_B(\hat{y}_k \mid \mathbf{e})$  — *оценка вероятности предсказанного значения по обученной байесовской сети при свидетельствах  $\mathbf{e} = (x^{\{\text{brand}\}}, \hat{y}_j \mid j \neq k)$* , то есть при известном бренде и уже сформированных предсказаниях других атрибутов товара;
- $\theta_k$  — поатрибутный порог байесовского валидатора; в реализации он задаётся как пятый перцентиль распределения  $P_B$  по обучающим примерам с верной разметкой;
- $f_k^{\{\text{LLM}\}}$  — обращение к большой языковой модели в роли запасного слоя (Layer 4), возвращающее либо значение из  $\mathcal{Y}_k$ , либо  $\text{null}$ .

Из приведённой формулы видно, что предсказание Layer 2 принимается тогда и только тогда, когда **одновременно** выполнены два неравенства:

$$\begin{aligned}
 & P_{\{\text{ML}\}}(\hat{y}_k \mid x) \geq \tau_k \quad \wedge \\
 & P_B(\hat{y}_k \mid \mathbf{e}) \geq \theta_k.
 \end{aligned}$$

Это конъюнктивное условие реализует **многоуровневую фильтрацию по уверенности** и формализует требование утверждённой темы работы об «ансамблевой оценке уверенности моделей». При этом важно понимать, что речь идёт не о параллельном голосовании нескольких моделей и не о

взвешенном усреднении их выходов, а именно о **последовательной фильтрации**: каждый слой имеет собственный калиброванный порог и вправе отказаться от ответа, передав решение следующему слою.

Стоит отметить, что условие  $P_B \geq \theta_k$  задаёт **общую (проектную) форму** каскада. По результатам эмпирической проверки байесовского валидатора (3.3.4) в финальную конфигурацию он включён **селективно** — только на тех парах (категория, атрибут), где даёт неотрицательный прирост точности; для остальных атрибутов порог  $\theta_k$  устанавливается так, что условие валидатора не отсекает предсказание Layer 2. Таким образом, формула описывает архитектуру в общем виде, а 3.3.4 уточняет, на каком подмножестве атрибутов валидатор фактически активен.

Описанная схема обладает рядом полезных свойств, на которые опирается обоснование выбора архитектуры в 2.2:

9. **Стратифицированное решение.** Простые случаи, когда ответ явно присутствует в тексте, закрываются дешёвым детерминированным слоем (Layer 1); случаи средней сложности — обученным классификатором (Layer 2); трудный остаток отправляется на дорогую большую языковую модель (Layer 4). Слои упорядочены по росту стоимости — от нулевой для регулярных выражений до десятков миллицентов за вызов LLM.
10. **Композициональная отбраковка.** Байесовский валидатор работает не вместо классификатора, а после него и отбраковывает статистически невероятные пары «бренд × значение». Это покрывает букву утверждённой темы об «автоматической отбраковке фактических ошибок до их попадания в каталог» и сохраняет при этом эмпирически подтверждённую конфигурацию (см. 3.3.4).
11. **Архитектурная объяснимость.** Для каждого предсказания известно, какой слой принял решение. Это обеспечивает локальную интерпретируемость на уровне отдельного товара даже тогда, когда в середине каскада стоит чернаящичный классификатор.

### 2.1.3 Поведение системы при выходе категории за пределы поддерживаемых

Layer 0 определяет категорию  $c = f_0(x)$  ещё до запуска основного каскада. Если оценка достоверности категоризации оказывается ниже порога OOD-детектора, классификатор присваивает товару метку  $\text{OOD}$ , а система отказывается от обработки и возвращает партнёру ответ вида «категория не поддерживается». Формально это записывается так:

$$\begin{aligned}
 & f_0(x) \in \mathcal{C} \setminus \{\text{OOD}\} \rightarrow \text{запустить} \\
 & \text{per-category каскад с схемой } \mathcal{A}_{f_0(x)}, \\
 & f_0(x) = \text{OOD} \rightarrow \hat{y} = \text{varnothing}.
 \end{aligned}$$

Такой явный режим отказа выбран по соображениям эксплуатационной безопасности. Если товар ошибочно направить в чужую категорию, все его атрибуты получают неверную схему, а это заметно хуже честного отказа от обогащения.

### 2.1.4 Метрики качества и стоимости

Поскольку каскад поддерживает явный отказ от ответа, для оценки качества используются две согласованные метрики и одна сводная.

**Точность на покрытой части (accuracy on covered)** — это доля верных предсказаний среди тех ячеек, на которых система выдала непустой ответ:

$$\begin{aligned}
 & \text{acc}_k^{\text{cov}} = \frac{|\{i : \hat{y}_k^{(i)} = y_k^{(i)} \wedge \hat{y}_k^{(i)} \neq \text{null}\}|}{|\{i : \hat{y}_k^{(i)} \neq \text{null}\}|}. \\
 &
 \end{aligned}$$

**Покрытие (coverage)** — это доля ячеек, на которых система решилась дать ответ:

$$\text{cov}_k = \frac{|\{i : \hat{y}_k^{(i)} \neq \text{null}\}|}{N},$$

где  $N$  — мощность проверочной выборки.

**Сквозная точность (end-to-end accuracy).** Чтобы предотвратить «искусственное завышение качества за счёт молчания», ячейки с  $\hat{y}_k^{(i)} = \text{null}$  засчитываются как неверные:

$$\text{acc}_k^{\text{e2e}} = \text{acc}_k^{\text{cov}} \cdot \text{cov}_k.$$

Сквозная точность является основной метрикой работы и используется в качестве заголовочного числа (3.3.2). Раздельный учёт  $\text{acc}^{\text{cov}}$  и  $\text{cov}$  необходим для того, чтобы добросовестно сравнивать каскад с конфигурациями, в которых отказ от ответа невозможен (например, со сценарием end-to-end LLM).

**Стоимость.** В качестве архитектурной метрики стоимости используется ожидаемая доля вызовов запасного слоя (нормирована на 1000 обработанных ячеек):

$$\text{cost}_{\text{LLM}} = \frac{1000}{N} \cdot K_c \cdot \sum_{i=1}^N \sum_{k=1}^{K_c} \mathbb{1}_{|\hat{y}_k^{(i)}| \neq \text{получен через } f_k^{\text{LLM}}}.$$

Архитектурная метрика инвариантна к выбору конкретной модели запасного слоя и характеризует именно структуру каскада. Для итогового экономического сопоставления в 3.3.2 эта величина дополнительно умножается на удельную стоимость обращения к выбранной модели (по тарифам провайдера) и нормируется относительно стратегии «все ячейки

решаются Sonnet 4.5» — это и даёт денежную ось матрицы «точность ↔ стоимость». При фиксированной точности минимизация  $\text{cost}\{\text{LLM}\}$  эквивалентна максимизации доли решений, принятых на дешёвых слоях; совместный анализ  $\text{acc}^{\text{e2e}}\{\text{cost}\{\text{LLM}\}\}$  образует матрицу «точность ↔ стоимость», на которой строится заголовочный результат работы и обоснование доминирования каскада по Парето над безусловными LLM-конфигурациями (3.3.2).

## 2.2 Архитектура и стек технологий

В этом разделе описывается архитектурное решение, которое реализует формальную постановку 2.1, и обосновывается выбор каждой компоненты стека.

### 2.2.1 Архитектура системы

Система реализована как пятислойный конвейер (Layer 0 плюс четыре содержательных слоя). Ниже кратко описаны состав слоёв и роль каждого из них; подробное описание реализации вынесено в 3.1.

- **Layer 0 — категорайзер.** Определяет категорию товара  $c \in \mathcal{C}$  либо относит его к классу OOD. Реализован как LightGBM-классификатор поверх TF-IDF-представления названия и бренда и дополнен порогом детектирования выхода за пределы распределения.
- **Layer 1 — экстрактор регулярных выражений.** Извлекает значения, которые явно присутствуют в тексте, например такие как форма пасты, процент какао, наличие пометки «цельнозерновой» или международные шаблоны количества. Применяется только к шаблонам, заведомо дающим высокую точность на совпадении.
- **Layer 2 — классификатор машинного обучения.** Это основной рабочий слой. Партнёр-доступные поля конкатенируются и проходят через многоязычный кодировщик SBERT; полученное 384-мерное векторное представление подаётся на вход поатрибутному классификатору XGBoost.

Каждый классификатор снабжён собственным поатрибутным порогом уверенности  $\tau_k$ , подобранным на отложенной части обучающей выборки. Применимость изотонической пост-калибровки выходов XGBoost исследована в двух режимах (3.3.3.3): калибровка на серебряной выборке деградирует на эталонном срезе из-за дрейфа распределений, а перекрёстная калибровка на распределении эталонной выборки даёт устойчивое улучшение средней ECE с 0,070 до 0,043; в текущую производственную конфигурацию пост-калибровка ещё не включена.

- **Layer 3 — байесовский валидатор.** На обученной байесовской сети, построенной по атрибутам товара (структура — Hill Climb + BIC), вычисляется  $P_B(\hat{y}_k \mid \mathbf{e})$ . Если вероятность оказывается ниже порога  $\theta_k$ , предсказание Layer 2 помечается флагом и передаётся на следующий слой. В 3.1 поясняется, что роль этого слоя была пересмотрена: вместо классификатора, как задумывалось изначально, сеть используется как валидатор плотностей.
- **Layer 4 — запасной слой на большой языковой модели.** Это обращение к одной из взаимозаменяемых больших языковых моделей семейств GPT [41] и Claude [42] (Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, открытая gpt-oss-120b, открытая малая llama-3.2-3b — полный список и сопоставление в 3.3.2) со структурированным запросом, который содержит схему атрибутов категории и партнёр-доступные поля. Возвращает значение из  $\mathcal{Y}_k$  либо честный отказ.

Слои упорядочены по росту удельной стоимости решения. Regex детерминирован и по существу бесплатен; ML работает на уже вычисленном векторном представлении и потому тоже почти бесплатен; байесовскому валидатору требуется один inference-вызов на сеть; запасной слой — самый дорогой.

## 2.2.2 Почему каскад, а не сквозное решение

Альтернативой каскаду служат две крайние конфигурации: «всё через большую языковую модель» (all-LLM) и «всё через один классификатор» (all-ML). Обе рассмотрены в работе как точки отсчёта, и обе отвергнуты.

**Сквозное решение на LLM** даёт точность 83–94 % на различных моделях (3.3.2). Однако его стоимость линейно растёт с числом ячеек, ведь каждая ячейка — это отдельный платный вызов. Каскад же экономит порядка двух порядков стоимости при равной или большей точности, и достигается это за счёт стратифицированного решения: дешёвые детерминированные слои закрывают типовые случаи, а дорогая модель работает только на трудном остатке. **Сквозное решение на классическом ML** не подходит по принципиальной причине. Ряд целевых атрибутов является производным и требует знаний за пределами товарного описания — например, классификация по диетическим свойствам опирается на состав в сочетании с нормативными правилами. Для таких атрибутов, а также для холодного старта новых категорий необходим запасной слой LLM. Таким образом, каскад занимает нишу между двумя крайностями.

### 2.2.3 Почему SBERT, а не BERT или RoBERTa

В качестве кодировщика текстовых полей выбрана модель `paraphrase-multilingual-MiniLM-L12-v2` (SBERT [20]), и сделано это по трём соображениям. Во-первых, **многоязычность достаётся «бесплатно»**: SBERT поддерживает свыше 50 языков в едином векторном пространстве без дообучения, благодаря чему каскад легко переносится на новые рынки. Во-вторых, **представление компактно**: вектор размерности 384 в два раза меньше представлений полноразмерных BERT (768) и в 2,7 раза меньше представлений RoBERTa-large (1024); при этом, по результатам публикации многоязычных дистиллированных вариантов [20], качество семантической классификации сохраняется, а расходы памяти и времени вывода падают пропорционально. В-третьих, **модель готова к работе без доменной адаптации**: она обучалась на задаче парафразирования с контрастивным сигналом, и поэтому её представление хорошо подходит для классификации

по семантической близости — дорогое доменное предобучение здесь не требуется.

#### 2.2.4 Почему XGBoost, а не нейронная сеть

В качестве классификатора Layer 2 для каждого атрибута используется градиентный бустинг XGBoost [14] с регуляризацией (`subsample=0.8`, `colsample=0.8`, ранняя остановка, `gamma=0.1`). На первый взгляд более очевидным выглядел бы выбор нейросетевой головы, однако предпочтение отдано XGBoost, и тому есть несколько причин. Градиентный бустинг устойчиво работает на плотных векторных признаках при ограниченном объёме эталонной разметки ( $n \approx 1\,250$  на категорию, см. 3.2.1), допускает пост-калибровку изотонической регрессией [28] и выполняет inference на CPU за единицы миллисекунд без графического ускорителя. Более того, поатрибутная архитектура «по одному классификатору на атрибут» упрощает цикл «аудит — правка — переобучение — измерение» (3.3.6).

#### 2.2.5 Почему байесовская сеть и почему именно pgmpy

Чтобы реализовать валидатор плотностей и закрыть пункт темы о «скрытых статистических закономерностях каталога», выбрана дискретная байесовская сеть в реализации библиотеки `pgmpy` [35] версии 1.1.2. По сравнению с альтернативами — марковскими случайными полями и нейросетевыми автоэнкодерами — байесовская сеть допускает структурное обучение из данных (Hill Climb + BIC [36]) с интерпретируемым ориентированным графом атрибутов, обеспечивает дешёвый вывод по запросу через алгоритм Variable Elimination и явно выражает условные зависимости между атрибутами товара в форме набора локальных распределений. Выбор именно `pgmpy` обусловлен совместимостью с экосистемой Python и тем, что библиотека поддерживает как обучение структуры, так и параметрическую оценку распределений через `BayesianEstimator`.

#### 2.2.6 Почему шлюз API на языке Go, а не монопольный Python

Демонстрационная система реализована как два сервиса — шлюз на Go (:8080) и ML-сервис на Python/FastAPI (:8001), — соединённых внутренним HTTP-вызовом. Шлюз отвечает за сетевой ввод-вывод, валидацию JSON, CORS, логирование и отдачу статического фронтенда; для этой роли подходит Go, поскольку он даёт низкое потребление памяти, быстрый запуск и нативную поддержку конкурентного ввода-вывода. ML-сервис, в свою очередь, отвечает за модели и inference, а здесь Python-экосистема (PyTorch, sentence-transformers, XGBoost, pgmpy) не имеет полноценных аналогов. Такое разделение зон ответственности соответствует типовой production-архитектуре и позволяет масштабировать каждый компонент независимо.

### **2.2.7 Почему открытая большая языковая модель в качестве запасного слоя**

В роли Layer 4 поддерживается набор взаимозаменяемых моделей, доступных через единый API OpenRouter: коммерческие Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, открытая gpt-oss-120b и открытая малая Llama-3.2-3b. Все они функционально эквивалентны и различаются точностью, латентностью и стоимостью; в 3.3.2 они сопоставлены количественно. Многомодельная конфигурация с возможностью использовать открытую модель обоснована тремя соображениями. Во-первых, открытую gpt-oss-120b можно запустить в инфраструктуре заказчика, и тогда партнёрские данные вообще не передаются во внешние сервисы. Во-вторых, коммерческие LLM в роли Layer 4 взаимозаменяемы, что снижает стратегическую зависимость от конкретного провайдера. В-третьих, каскад компенсирует слабость малой открытой модели: при самостоятельном применении gpt-oss-120b проигрывает Claude Sonnet 14–15 п. п., однако в составе каскада, где запасной слой обрабатывает лишь  $\approx 3\%$  ячеек, итоговая точность каскада с gpt-oss-120b превышает безусловное обращение к этой модели на 23,0 п. п. (3.3.2).

### **2.2.8 Стек технологий — сводная таблица**

Сводное представление принятых решений приведено в таблице 2.1.

| Компонент               | Технология                                                                                                                                                                                    | Ключевое обоснование                                                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Шлюз API                | Go, стандартная библиотека <code>net/http</code>                                                                                                                                              | Малое потребление ресурсов, быстрый запуск, отсутствие внешних зависимостей                            |
| ML-сервис               | Python 3.14, FastAPI, Uvicorn                                                                                                                                                                 | Доступ к ML-экосистеме, автоматическая валидация через Pydantic, поддержка OpenAPI                     |
| Векторное представление | SBERT <code>paraphrase-multilingual-MiniLM-L12-v2</code>                                                                                                                                      | Многоязычность (50+ языков), компактный 384-мерный вектор, отсутствие необходимости в fine-tuning      |
| Классификатор Layer 2   | XGBoost с регуляризацией; перекрёстная изотоническая калибровка на эталонной выборке снижает среднюю ECE с 0,070 до 0,043 (3.3.3.3), внедрение в производственную конфигурацию вынесено в 5.5 | Устойчивость на плотных признаках, калибруемые вероятности, дешёвый вывод                              |
| Байесовская сеть        | <code>pgmpy</code> 1.1.2, <code>DiscreteBayesianNetwork</code>                                                                                                                                | Структурное обучение Hill Climb + BIC, дешёвый inference (Variable Elimination), интерпретируемый граф |
| Категорайзер Layer 0    | LightGBM (поверх TF-IDF-представления) + порог OOD                                                                                                                                            | Multi-class классификация с вероятностями + явный отказ при низкой уверенности                         |
| Запасной слой           | OpenRouter API: Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, gpt-oss-120b, llama-3.2-3b                                                                                                              | Единый интерфейс, возможность on-premise, независимость от                                             |

|          |                                           |                                                               |
|----------|-------------------------------------------|---------------------------------------------------------------|
|          |                                           | поставщика                                                    |
| Фронтенд | HTML/CSS/JavaScript без сборочной системы | Минимальные требования к окружению, прозрачность демонстрации |

Таблица 2.1. Сводная таблица стека технологий с обоснованием выбора.

## 2.3 Сценарий работы программного обеспечения

### 2.3.1 Сквозной пример прохождения запроса

Прохождение товара через систему рассмотрим на примере типового продукта — пасты *Spaghetti #5 Barilla*; этот же пример повторяется в 3.1.2. Партнёрская система формирует HTTP-запрос к шлюзу:

```
POST /api/enrich
Content-Type: application/json

{
  "category": null,
  "product_name": "Spaghetti #5 Barilla",
  "brands": "Barilla",
  "ingredients_text": "Durum wheat semolina, water",
  "quantity": "500g"
}
```

Поле `category` оставлено пустым, то есть определение категории делегируется системе.

**Шаг 1. Определение категории (Layer 0).** Категорайзер возвращает  $c = \text{pasta}$  с уверенностью  $0.97$ , метка OOD не активируется, и система выбирает схему атрибутов  $\mathcal{A}_{\text{pasta}}$  (8 целевых атрибутов).

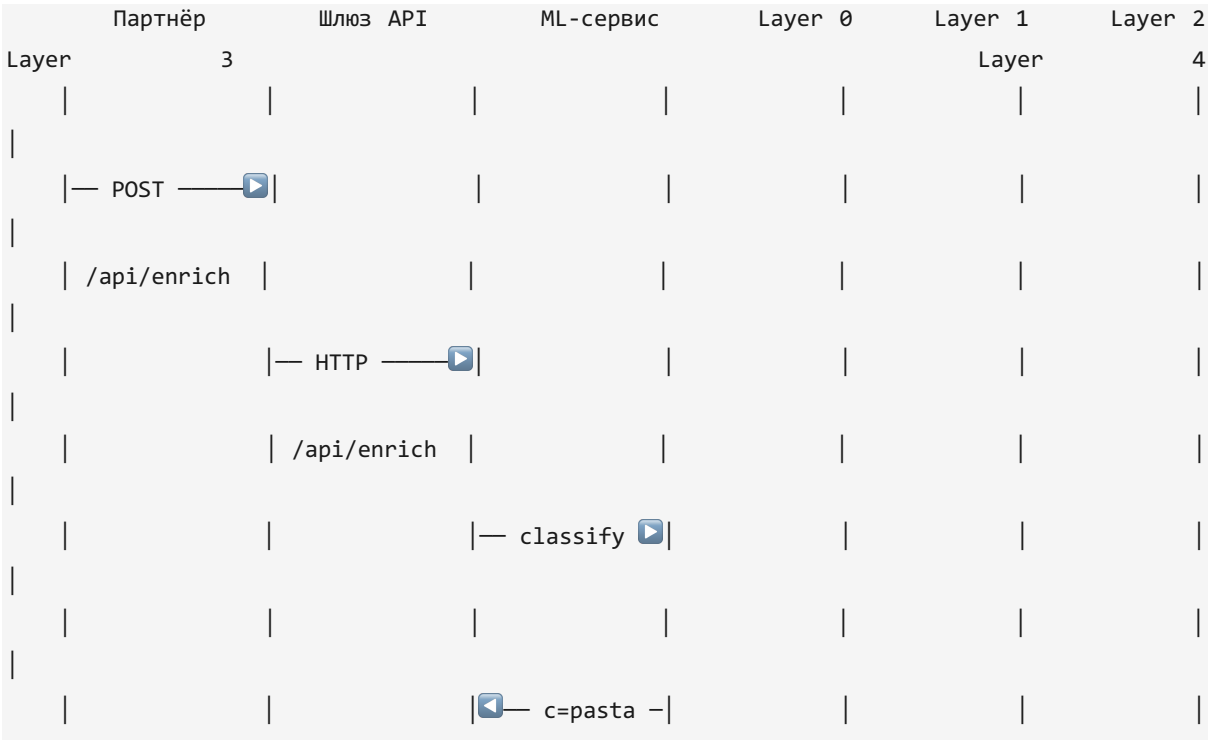
**Шаг 2. Поочерёдная обработка атрибутов.** Каскад обходит атрибуты в порядке схемы; рассмотрим четыре характерных случая. Для `pasta_shape` срабатывает шаблон Layer 1 на лексеме «spaghetti» — возвращается значение с уверенностью  $1.0$ , и дальнейшие слои уже не вызываются. Для `grain_type` Layer 1 совпадения не даёт; тогда Layer 2 кодирует поля SBERT и выдаёт

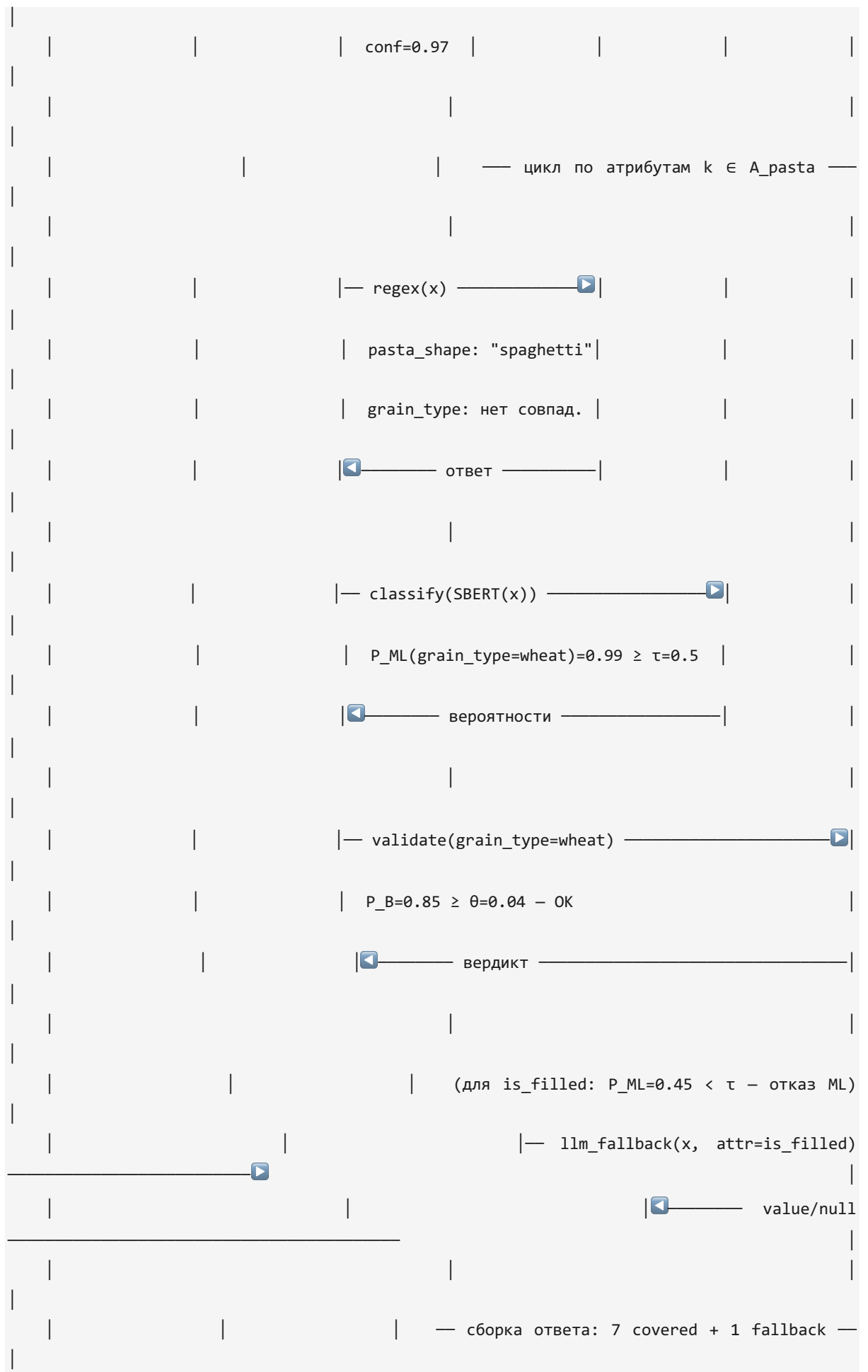
класс "wheat" с  $P_{\{\text{ML}\}} = 0{,}99 \geq \tau = 0{,}5$ , а байесовский валидатор это подтверждает ( $P_B = 0{,}85 \geq \theta = 0{,}04$ ) — ответ принимается с отметкой ml. Для nutri\_score\_grade классификатор возвращает класс "A" с  $P_{\{\text{ML}\}} = 0{,}83$ , и валидатор снова подтверждает. А вот для is\_filled максимальная вероятность классификатора  $0{,}45$  ниже порога  $\tau = 0{,}5$  — условие конъюнкции 2.1.2 не выполнено, и ячейка маршрутизируется на запасной слой.

**Шаг 3. Формирование ответа.** ML-сервис собирает поатрибутный ответ, добавляет агрегированные счётчики (число заполненных атрибутов и число ушедших в LLM-fallback) и возвращает JSON шлюзу, а тот пересылает его партнёру (формат ответа описан в 3.1.2). В рассмотренном сценарии 7 из 8 атрибутов закрыты дешёвыми слоями (1 — экстрактором, 6 — классификатором), и лишь один трудный атрибут передан на запасной слой. Именно такое распределение нагрузки между слоями обеспечивает доминирование каскада в матрице «точность ↔ стоимость» (3.3.2).

### 2.3.2 Диаграмма взаимодействия компонентов

Ниже логика взаимодействия компонентов системы при обработке одного запроса представлена в нотации последовательной диаграммы.





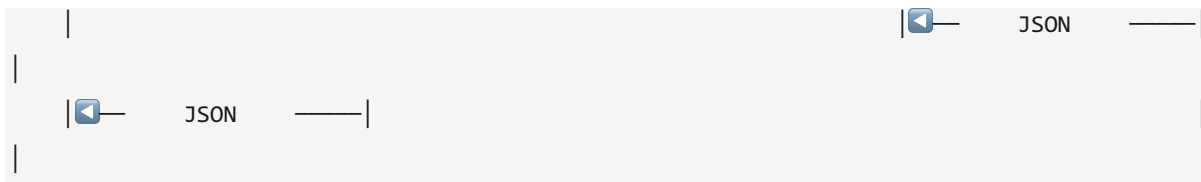


Рис. 2.1. Диаграмма последовательности обработки запроса в гибридной каскадной системе обогащения.

В диаграмме стоит обратить внимание на несколько существенных моментов. Во-первых, Layer 0 вызывается ровно один раз — он определяет схему ещё до начала цикла по атрибутам. Во-вторых, сам цикл устроен так, что переход к более дорогому слою происходит **только** при честном отказе всех предыдущих, а не параллельно с ними. В-третьих, единственное обращение к запасному слою (Layer 4) приходится на тот атрибут, в котором Layer 2 не уверен, и это наглядная иллюстрация центрального экономического свойства каскада, описанного в 2.1.4.

## 2.4 Выводы по главе

12. Задача обогащения формализована как поатрибутное предсказание  $\hat{y}_k$  при заданном кортеже партнёр-доступных полей и определённой Layer 0 категории. Логика работы программного обеспечения формализована как последовательная фильтрация по уверенности: предсказание классификатора принимается тогда и только тогда, когда одновременно выполнены  $P_{\text{ML}} \geq \tau_k$  и  $P_B \geq \theta_k$ , а иначе ячейка передаётся на запасной слой. Таким образом, эта конструкция реализует требование темы об ансамблевой оценке уверенности именно как последовательное AND-условие на пороги слоёв, а не как параллельное голосование.
13. Архитектурный выбор каждой компоненты — каскад вместо сквозного решения, SBERT, XGBoost, байесовская сеть `pgmpy`, разделение Go-шлюз / Python-ML, открытая модель gpt-oss-120b как вариант запасного слоя — обоснован эксплуатационными, методологическими и стратегическими

соображениями. Более того, каскад превращает «слабую и бесплатную» открытую модель в работоспособную часть production-стека.

14. Сценарий прохождения типового запроса показывает, что большинство атрибутов закрывается дешёвыми слоями каскада, а обращение к большой языковой модели остаётся точечным. Именно такое распределение нагрузки и объясняет доминирование системы по Парето, которое эмпирически подтверждается в 3.3.

### **3 ОПИСАНИЕ ПРОГРАММНО-ТЕХНОЛОГИЧЕСКОЙ РЕАЛИЗАЦИИ**

В настоящей главе описывается программно-технологическая реализация системы автоматизированного обогащения товарных данных. В разделе 3.1 представлено разработанное программное обеспечение: архитектура каскадного конвейера обработки и демонстрационная система, через которую конечный пользователь взаимодействует с каскадом. В разделе 3.2 рассмотрены используемые данные — их источники, структура и предобработка. Наконец, раздел 3.3 содержит доказательство работоспособности и применимости системы: численные показатели, поатрибутные таблицы и сценарии тестирования.

## 3.1 ОПИСАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

### 3.1.1 Архитектура каскада

Гибридный каскад спроектирован как последовательность из пяти слоёв; каждый из них обрабатывает отдельный класс случаев и передаёт «трудный остаток» следующему слою. Архитектура соответствует формальной постановке задачи (2.1) и реализует многоуровневую фильтрацию по уверенности: каждый слой располагает собственным калиброванным порогом уверенности, а итоговый вердикт системы формируется как последовательное AND-условие прохождения этих порогов.

#### Состав слоёв

**Layer 0 — категорайзер.** Перед запуском основного каскада требуется определить категорию товара (паста, шоколад, сыры или метка «не поддерживается»). Для этого служит отдельный классификатор LightGBM поверх TF-IDF-представления текста (n-граммы 1–2) с порогом для детектирования выхода за пределы распределения (OOD). Артефакты: `models/category_router_v3_lgbm.pkl`, `models/category_router_v3_vec.pkl`, `models/category_router_v3_threshold.json`. Стоит отметить, что архитектурно Layer 0 не входит в общую логику каскада — он лишь решает, **какой** per-category каскад вообще запускать. На итоговую точность системы этот слой влияет через штраф маршрутизации: при ошибке в категории все атрибуты товара получают неверную схему и засчитываются как неверные предсказания.

**Layer 1 — экстрактор регулярных выражений.** Этот слой разбирает партнёр-доступные поля (`product_name`, `quantity`) и извлекает значения, явно присутствующие в тексте: форму пасты («spaghetti», «penne»), процент какао («70 % соеа» — класс «70–85»), индикатор цельнозернового состава, международные шаблоны вроде «300 g». Он работает на детерминированных шаблонах, где наличие совпадения само по себе означает высокую уверенность. Реализация — `src/pipeline/regex/extractor.py`. Покрытие первого слоя ограничено: на pasta около 18 %, на chocolate 23 %, на cheeses 0 %

(правила для этого домена не разработаны, что явно фиксируется как ограничение системы — см. 5.4); точность слоя на ячейках его закрытия — 87,0–90,5 % (поатрибутно отдельные шаблоны достигают 95–100 %, см. 3.3.3.2).

**Layer 2 — классификатор машинного обучения.** Это основной рабочий слой, на который приходится 72–96 % решений по категориям. На вход подаётся конкатенация партнёр-доступных полей `product_name + brands + ingredients_text + quantity`, которую многоязычный SBERT-кодировщик `paraphrase-multilingual-MiniLM-L12-v2` преобразует в 384-мерное векторное представление (он поддерживает 50+ языков «бесплатно», включая 5 рабочих языков выборки FR/EN/DE/IT/ES). Далее для каждой пары (категория, атрибут) обучен отдельный XGBoost-классификатор (`{cat}_{attr}_xgb_hybrid.pkl`) с регуляризацией (`subsample=0.8`, `colsample=0.8`, ранняя остановка) и поатрибутным порогом уверенности (`{cat}_thresholds.pkl`). Если максимальная вероятность класса ниже порога, слой выдаёт отказ от ответа, и решение переходит к следующему слою. Поатрибутная точность Layer 2 на покрытом срезе составляет 94,9–98,6 % (см. 3.3.3.2).

**Layer 3 — байесовский валидатор.** Обученная байесовская сеть атрибутов (`models/{cat}_stratified_bayesian.pkl`, структура Hill Climb + BIC) применяется в **иной роли**, нежели предполагала исходная спецификация. Изначально сеть планировалась как классификатор — восстановление пропусков через  $P(\text{attr}_X \mid \text{brand}, \text{attr}_Y)$ , — однако эмпирическое исследование (3.3.4) показало, что её классификаторная точность на трудном остатке после Layer 2 составила около 52 %, то есть ниже мажоритарной точки отсчёта. Поэтому в финальной конфигурации сеть используется как **валидатор плотностей**: для предсказания Layer 2 вычисляется  $P(\text{value} \mid \text{evidence})$ , и при попадании в нижний 5-перцентиль порога предсказание помечается флагом и понижается на следующий слой. Тем самым покрывается буква утверждённой темы об «автоматической отбраковке фактических ошибок» и «скрытых статистических закономерностях каталога».

**Layer 4 — запасной слой на большой языковой модели.** Если предыдущие слои не дали ответа либо Bayes-валидатор пометил ответ Layer 2 как сомнительный, ячейка отправляется в LLM. Оценке в 3.3.2 подлежат пять моделей: Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, открытая gpt-oss-120b через OpenRouter и открытая малая llama-3.2-3b — функционально они эквивалентны и различаются лишь стоимостью и скоростью. Промпт содержит структурированную схему атрибутов категории, требуемый JSON-формат и партнёр-доступные поля без подсказок об ожидаемом значении (src/pipeline/llm\_fallback/enrich.py).

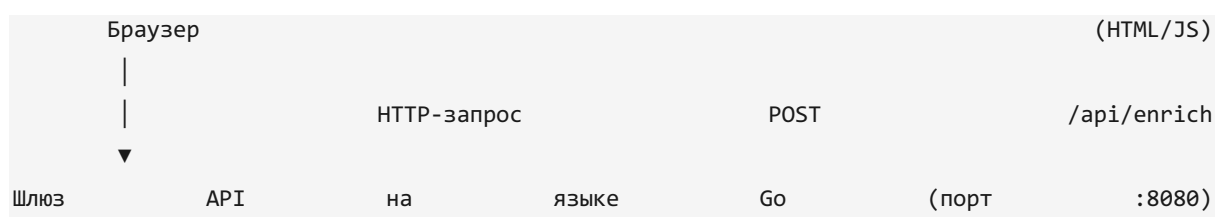
### Поток управления и вход/выход

Поток управления описан в формальной постановке (2.1, формула  $\hat{y}_k$ ). На каждом слое принимается одно из двух решений — «выдать значение и закрыть атрибут» либо «передать решение следующему слою»; слои упорядочены по росту стоимости, так что дорогой LLM-вызов делается только на тех ячейках, где более дешёвые слои честно отказались. На вход системы поступают партнёр-доступные поля, на выходе — заполненные значения атрибутов категории. Архитектурно важно, что Layer 2 обучается только на партнёр-доступных полях: это исключает утечку через теги OFF, поскольку последние являются результатом обогащения, а не входом для обучения.

### 3.1.2 Демонстрационная система

Чтобы показать работоспособность каскада на практике, разработана трёхкомпонентная демо-система, которая имитирует встраивание в реальный бизнес-процесс интернет-магазина. Её архитектура показана на рис. 3.1.

#### Архитектура компонентов





**Шлюз API на Go** (`demo/gateway/`, около 200 строк на стандартной библиотеке `net/http` без внешних зависимостей) отвечает за валидацию входящих JSON-запросов, проксирование к ML-сервису, обработку CORS, логирование и отдачу статического фронтенда. Выбор Go обеспечивает низкое потребление памяти, быстрый запуск и технологическое разделение зон ответственности: сеть — на Go, машинное обучение — на Python.

**ML-сервис на Python/FastAPI** (`demo/ml_service/`) — основная часть демо. При старте он загружает многоязычный SBERT-кодировщик, 24 XGBoost-классификатора (по одному на пару «категория, атрибут»: 8 в `pasta`, 7 в `chocolate`, 9 в `cheeses`; 18 уникальных имён атрибутов с учётом повторяющихся между категориями). Из них экспериментальной оценке в 3.3 подлежат 22 пары — два классификатора для `cheeses` (`nutri_score_grade`, `protein_class`) обучены и развернуты, но не оцениваются на эталонном срезе ввиду отсутствия в OFF разметки этих производных атрибутов для категории сыров. Также загружаются байесовские сети, словари порогов и модель категорайзера Layer 0. После прогрева ( $\approx 15\text{--}20$  с на загрузку SBERT) запросы к слоям 0–3 обрабатываются за десятки миллисекунд. Сервис предоставляет ЭНДПОИНТЫ: `POST /api/enrich` (обогащение товара), `GET /api/categories` (поддерживаемые категории и атрибуты), `GET /health` (готовность слоёв), `POST /api/explain` (поатрибутное объяснение через Shapley-значения).

**Веб-интерфейс** (`demo/frontend/`) — статическая страница HTML/CSS/JavaScript без сборочной системы. Она содержит форму ввода партнёр-доступных полей, переключатель ручной и автоматической категоризации, пресеты примеров для трёх категорий, а также блок результата с отметкой слоя-источника для каждого значения.

## Контракт API

Тело запроса `POST /api/enrich`:

```
{
  "category": "pasta",
  "product_name": "Spaghetti",
  "brands": "Barilla",
  "ingredients_text": "Durum wheat semolina, water",
  "quantity": "500g"
}
```

Поле `category` опционально: если оно не указано, срабатывает Layer 0 и определяет категорию автоматически.

Тело ответа содержит поля `category`, агрегированные счётчики (`n_attrs_total`, `n_covered`, `n_llm_fallback`) и объект `predictions`, в котором для каждого атрибута указаны значение, слой-источник (`regex` / `ml` / `llm_fallback`), уверенность и результат валидации:

```
{
  "category": "pasta",
  "n_attrs_total": 8,
  "n_covered": 7,
  "n_llm_fallback": 1,
  "predictions": {
    "pasta_shape": { "value": "spaghetti", "layer": "regex", "confidence": 1.00 },
    "grain_type": { "value": "wheat", "layer": "ml", "confidence": 0.99,
      "validation": { "flagged": false, "p": 0.85, "threshold": 0.04 }
    },
    "is_filled": { "value": null, "layer": "llm_fallback", "confidence": 0.0 }
  }
}
```

Стоит отметить, что явная отметка слоя-источника компенсирует ограниченную интерпретируемость классификаторов Layer 2 и Layer 3 на уровне отдельного товара: пользователь всегда понимает, какому компоненту он доверяет.

## Запуск и эксплуатация

Запуск выполняется тремя командами (ML-сервис на `:8001`, Go-шлюз на `:8080`, браузер) либо одной командой `bash reproduce.sh demo`. Готовность системы проверяется через `GET /health` — корректным считается ответ `{"status":"ok","pipeline_ready":true,"validator_ready":true,"router_ready":true}`. Стек технологий и обоснование выбора каждого компонента приведены в 2.2.8

(таблица 2.1); приём «3-компонентное разделение» (Go-шлюз / Python-ML / JS-фронт) соответствует типовой производственной архитектуре отрасли.

### **Выводы по 3.1**

15. Каскад реализован как последовательность из пяти слоёв с возрастающей стоимостью и убывающей детерминированностью — от регулярных выражений до большой языковой модели. Каждый слой располагает собственным калиброванным порогом уверенности и явным режимом отказа от ответа.
16. Архитектурный выбор «Bayes как валидатор плотностей, а не как классификатор» обоснован эмпирически (см. 3.3.4) и закрывает букву утверждённой темы ВКР («автоматическая отбраковка фактических ошибок», «скрытые статистические закономерности каталога»).
17. Демонстрационная система реализует производственно-подобный сценарий: на входе — партнёр-доступные поля, на выходе — обогащённые атрибуты с отметкой слоя-источника. Технологическое разделение Go-шлюз / Python-ML / JS-фронт соответствует типовой архитектуре отрасли.

## **3.2 Данные**

В настоящем подразделе рассматриваются источник исходных данных, состав партнёр-доступных и целевых полей, формирование стратифицированной выборки, иерархическая схема эталонной разметки и разбиение без пересечения брендов.

### **3.2.1 Источник данных: Open Food Facts**

В качестве основы для построения и оценки системы выбран открытый каталог Open Food Facts (далее — OFF) — краудсорсинговая база продовольственных товаров, которая на момент загрузки содержала порядка 4,5 млн уникальных позиций под лицензией Open Database License (ODbL), допускающей коммерческое использование. Полный дамп каталога получен

стандартными средствами проекта (`python -m src.data.download --source off`) и конвертирован в столбцовый формат `parquet` (`datasets/raw/en.openfoodfacts.org.products.parquet`, 526 МБ).

Выбор OFF обоснован совокупностью свойств. Во-первых, открытость лицензии обеспечивает полную воспроизводимость экспериментов. Во-вторых, каталог репрезентативен по охвату (крупнейшие европейские и североамериканские бренды, региональные и нишевые производители) и мультиязычен — более 50 языков в полях `product_name` и `ingredients_text`. В-третьих, помимо текстовых описаний OFF содержит структурированные нутриентные показатели (`fat_100g`, `sugars_100g`, `proteins_100g` и др.) и регуляторные теги (`labels_tags`, `ingredients_analysis_tags`), на которых построена иерархическая разметка эталона (3.2.3). Альтернативные источники, такие как извлечение страниц онлайн-магазинов, коммерческие РИМ-каталоги и теги Wikipedia, отклонены по соображениям лицензионной чистоты, ограниченного мультиязычного покрытия или отсутствия нутриентных полей.

Областью охвата работы являются три категории продовольственных товаров — макаронные изделия (`pasta`), шоколад (`chocolate`) и сыры (`cheeses`). Они выбраны по критериям 1.3: достаточный объём выборки, различие типа целевых атрибутов между категориями (от регуляторных меток до семантически-текстовых характеристик) и присутствие подгруппы атрибутов с детерминированным эталоном из нутриентных полей.

### **3.2.2 Входные и целевые поля, фильтрация, стратифицированная выборка**

Поля исходного дампа OFF разделены на две группы. К **партнёр-доступным полям**, поступающим на вход системы при эксплуатации, отнесены наименование (`product_name`), бренд (`brands`), упаковка (`quantity`), состав (`ingredients_text`) и штрих-код (`code`) — это реалистичный контракт обмена данными между интернет-магазином и поставщиком. Поля `categories_tags`, `labels_tags`, нутриентные показатели `*_100g` и регуляторные `ingredients_analysis_tags` отнесены к **целевым полям выхода**: именно их восстановление является

задачей обогащения. Эти поля не передаются на вход машинно-обученных компонентов под угрозой утечки целевой переменной — слой ML формирует входной вектор только из конкатенации `product_name + brands + ingredients_text + quantity`.

При фильтрации дампа по категории (`src/data/filter.py`) принадлежность товара определяется по `categories_tags` через списки включения и исключения (так, для `pasta` включение — `en:pastas, en:fresh-pastas`, исключение — `en:potato-products, en:rice-products, en:noodles`). После фильтрации применяется стратифицированная по языку выборка: основным языком наименования определяется утилитой `langdetect`, целевыми стратами выступают пять языков EC-5 (FR, EN, DE, IT, ES), и из каждой отбирается ~250 товаров, что даёт ~1250 товаров на категорию (таблица 3.2.1).

**Таблица 3.2.1. Состав стратифицированной выборки по категориям, входящим в область охвата работы. Источник:**

`datasets/processed/{cat}_stratified_silver_standard.parquet`.

| Категория                               | Объём после фильтрации (сырые позиции) | Объём стратифицированной выборки | Число целевых атрибутов |
|-----------------------------------------|----------------------------------------|----------------------------------|-------------------------|
| <code>pasta</code> (макаронные изделия) | 15 691                                 | 1 250                            | 8                       |
| <code>chocolate</code> (шоколад)        | 13 469                                 | 1 250                            | 7                       |
| <code>cheeses</code> (сыры)             | 21 208                                 | 1 250                            | 9                       |

Множество целевых атрибутов для каждой категории определено в схемах `src/pipeline/schemas/{cat}.py`. Оно включает как однотипные кросс-категорийные показатели, например такие как `is_organic` и `nutri_score_grade`, так и сугубо специфичные для данной категории признаки — `pasta_shape`, `chocolate_type`, `milk_source`, `texture` и аналогичные. Совокупное число пар (категория, атрибут) по трём категориям составляет **24 пары для 18 уникальных имён атрибутов** (часть атрибутов, например `is_organic`,

повторяется в нескольких категориях); из них экспериментальной оценке в 3.3 подлежат **22 пары** — два атрибута категории `cheeses` (`nutri_score_grade`, `protein_class`) обучены и развёрнуты, но не оцениваются на эталонном срезе ввиду отсутствия в OFF разметки этих производных атрибутов для категории сыров.

### 3.2.3 Эталонная разметка: иерархическая схема трёх ярусов

Принципиальная методологическая особенность работы состоит в том, что эталонная разметка (gold standard) строится не как ручная аннотация единичной группой экспертов, а как многоярусная процедура, в которой каждый ярус опирается на качественно различный класс свидетельств. Такой подход позволяет получить высокую точность эталона при объёме выборки, на порядки превосходящем посильный для ручной разметки, и явно отделить атрибуты, разрешаемые детерминированным правилом, от требующих семантического вывода (таксономия зафиксирована в приложении А).

**Ярус 0 — нутриентно-производные атрибуты** (`nutri_derived`): ЭТАЛОН получен детерминированным бакетированием числовых полей OFF — `nutri_score_grade` ИЗ `*_100g` по регламенту ЕК 2017/1944, `protein_class` ИЗ `proteins_100g`, `fat_class` ИЗ `fat_100g`. Поскольку отображение детерминировано и доступно для аудита, формальная точность эталона на ярусе равна 100 % при достоверных нутриентах, а покрытие составляет 95–100 %.

**Ярус 1 — тег-производные атрибуты** (`tag_derived`): ЭТАЛОН получен из регуляторных ТЕГОВ `labels_tags` И `ingredients_analysis_tags` — `is_organic` ИЗ `en:organic`, `is_gluten_free` ИЗ `en:gluten-free`, `is_pdo` ИЗ `en:protected-designation-of-origin`. Достоверность здесь опирается на легальную защиту меток правом ЕС (регламенты 834/2007 и 1151/2012): ложное проставление защищённой метки образует состав административного правонарушения, поэтому соответствующие теги OFF можно считать свидетельствами высокой достоверности.

**Ярус 2 — семантически-текстовые атрибуты** (`text_derived`): ЭТО признаки, требующие семантического анализа текста, для которых

детерминированное правило построить не удаётся (`pasta_shape`, `chocolate_type`, `milk_source`, `texture`, `contains_nuts`, `cocoa_percentage`). Эталон получен согласием трёх независимых языковых моделей — Claude Sonnet 4.5, GPT-4o и Gemini 2.5 Flash — на полной стратифицированной выборке (1250 позиций × 3 категории): атрибут получает значение, если не менее двух из трёх моделей независимо выдают идентичный ответ (согласованность 97,7 %). Несогласованные позиции вынесены на слепую разметку Claude Opus 4. Итоговая разметка (`consensus_gold_v2_expanded.parquet`) содержит 19 551 эталонную ячейку (4 444 — ярус 0, 7 108 — ярус 1, 7 999 — ярус 2). Состав ярусов неоднороден по категориям, что оправдывает отдельное представление результатов в 3.3.

### 3.2.4 Разбиение без пересечения брендов

Стандартное случайное разбиение методологически непригодно. Дело в том, что бренд сам по себе является сильным предиктором целевых атрибутов (бренд «Lindt» с высокой вероятностью маркирует тёмный шоколад определённого диапазона какао), и при случайном разбиении один бренд оказывается одновременно в обучении и в тесте. Это завышает оценки точности за счёт «утечки бренда»: модель запоминает соответствие «бренд — атрибут», не обобщая его на новые марки. Условия эксплуатации, где интернет-магазин регулярно подключает новых партнёров, такая оценка не отражает.

Поэтому применяется разбиение без пересечения брендов (`src/data/split/brand_disjoint.py`): множество уникальных брендов каждой категории делится на три непересекающихся подмножества в пропорции 60/20/20, и все позиции данного бренда направляются ровно в одно из них. Объём тестового подмножества — 170 брендов в `pasta`, 163 в `chocolate`, 188 в `cheeses`; совокупное число тестовых ячеек (товар × атрибут с проставленным эталоном) — 1 423 / 1 270 / 1 657, итого **4 350 ячеек** (`headline_v3e_final.parquet`). Именно этот тестовый срез служит основой всех итоговых количественных оценок 3.3.

### 3.2.5 Дополнительные источники: разнородные модальности и внешний справочник

Помимо OFF, для усиления обучающего сигнала второго слоя задействованы два дополнительных класса данных, реализующих требование темы о работе с «разнородными источниками данных». Первый класс — **разнородные модальности**: фотографии этикеток обработаны OCR (EasyOCR; результат обогащает текстовое представление) и визуальным кодировщиком CLIP [43] (ViT-B/32; 512-мерный вектор добавляется к 384-мерному SBERT-представлению ранней конкатенацией), причём покрытие обучающей выборки изображениями составляет  $\approx 70$  %. Второй класс — **внешний справочник** USDA Branded Foods [44], сопоставленный с выборкой приближённым поиском по `product_name + brands` (алгоритм Левенштейна, порог сходства 0,85); сопоставленные позиции дают независимо измеренные нутриентные значения для проверки эталона яруса 0.

**Принципиальное ограничение области использования дополнительных источников.** Оба указанных класса — изображения и USDA-справочник — задействуются исключительно на этапе **обучения** компонентов системы. В сценарии производственного вывода партнёр интернет-магазина передаёт системе только текстовое представление товара — ПОЛЯ `product_name + brands + ingredients_text + quantity`, в строгом соответствии с контрактом обмена (3.2.2). Это разграничение является методологическим инвариантом системы и строго соблюдалось при формировании всех итоговых количественных оценок 3.3.

## 3.3 ДОКАЗАТЕЛЬСТВО РАБОТОСПОСОБНОСТИ И ПРИМЕНИМОСТИ

### 3.3.1 Условия проверки

Раздел 3.3 содержит доказательство работоспособности и применимости разработанной системы. Здесь рассматриваются работоспособность слоёв каскада, применимость в типовых производственных сценариях и численная характеристика достигнутых показателей с границами доверительных интервалов.

#### Тестовая выборка

Все эксперименты раздела проведены на единой тестовой выборке. **Источником эталона** служит `consensus_gold_v2_expanded.parquet`: для 8 текстово-семантических атрибутов построено согласие трёх независимых языковых моделей (Sonnet 4.5 + GPT-4o + Gemini 2.5 Flash) на полной выборке 1250 товаров на категорию (97,7 % согласие — 2 модели из 3 совпадают), затем подтверждённое слепой разметкой Orus 4; для остальных атрибутов эталон детерминирован — нутриентно-производные классы получены из числовых полей OFF, тег-производные — из регуляторных тегов с проверкой по EU 834/2007 и EU 1151/2012.

**Независимая проверка эталона.** Чтобы исключить структурную зависимость между признаками модели и источником эталона, проведена слепая повторная разметка Claude Orus 4 на 2666 кодах × 22 атрибута без доступа к OFF-тегам и промежуточным меткам — только по партнёр-доступным полям. Согласованность слепой разметки Orus с эталоном составила 95,0 % на 4088 ненулевых ячейках (`blind_vs_prefill_overall.parquet`); на 17 из 22 пар (категория, атрибут) согласованность превышает 92 %, к Коэна > 0,85. Для трёх нутриентно-производных пар Orus в слепом режиме отказывается отвечать в 82–93 % случаев (правило бакетирования требует числовых полей), и здесь эталоном выступает детерминированное правило яруса 0.

**Состав тестового среза** — 3 категории, **22 пары (категория, атрибут)** для **18 уникальных имён атрибутов**, 4350 ячеек после фильтрации непустых эталонных значений (по 234–239 кодов на категорию). Уникальных брендов: 170 в pasta, 163 в chocolate, 188 в cheeses (`headline_v3e_final.parquet`). **Разбиение train/val/test без пересечения брендов** выполнено в пропорции 60/20/20 — один бренд попадает ровно в одну часть, что исключает утечку признака «бренд — атрибут» (`src/data/split/brand_disjoint.py`).

### Метрики и доверительные интервалы

Метрики качества (точность на покрытом срезе  $\text{acc}^{\text{cov}}_k$ , покрытие  $\text{cov}_k$ , сквозная точность  $\text{acc}^{\text{e2e}}_k$ , стоимость) формально определены в 2.1.4. Доверительные интервалы рассчитаны методом кластерного бутстрэпа по брендам (1000 итераций, ресэмплирование брендов с возвращением, 2,5 % и 97,5 % перцентили). Такой интервал учитывает внутрибрендовую корреляцию ошибок и на ряде атрибутов оказывается на 15–30 % шире наивного интервала Вильсона (`src/diagnostics/ml/bootstrap_ci_brand_clustered.py`).

### Стратификация точности по типу эталонного сигнала

Чтобы охарактеризовать структурную независимость измеряемой точности от источника эталона, итоговая точность приведена в разрезе трёх ярусов сигнала (3.2). Рассуждение здесь следующее: если бы точность завышалась за счёт совпадения текстовых признаков модели с источником эталона (структурная контаминация), то ярус 2 (текстово-производный эталон) демонстрировал бы наиболее высокие показатели. Однако результат оказывается обратным (таблица 3.3.1.1).

**Таблица 3.3.1.1. Точность каскада в разрезе типа эталонного сигнала.**

Источник: `datasets/processed/tier_breakdown.parquet`.

| Ярус сигнала | Содержательная природа | Атрибутов | Ячеек | Точность, % | 95 % CI |
|--------------|------------------------|-----------|-------|-------------|---------|
| Ярус 0       | Детерминированное      | 3         | 354   | 83,33       | [79,12; |

|                               |                                                 |   |       |              |                |
|-------------------------------|-------------------------------------------------|---|-------|--------------|----------------|
| (нутриент-производный)        | бакетирование числовых полей OFF                |   |       |              | 87,20]         |
| Ярус 1 (тег-производный)      | Регуляторные теги OFF, защищённые правом ЕС     | 6 | 1 912 | <b>93,62</b> | [92,45; 94,69] |
| Ярус 2 (текстово-производный) | Согласие трёх LLM с проверкой Opus 4 на спорных | 9 | 2 084 | <b>81,81</b> | [80,07; 83,61] |

Точность на наиболее уязвимом к структурной корреляции ярусе 2 оказывается наименьшей в выборке (81,8 %, на 11,8 п. п. ниже яруса 1); на ярусе 1, где эталон порождён внешним источником — правовыми регистрами сертификаций, — точность, напротив, наиболее высокая (93,6 %). Это противоположно картине завышения метрик за счёт замкнутой зависимости между признаками модели и эталоном и количественно опровергает соответствующую методологическую претензию. На ярусе 0 показатель частично снижен из-за пары `cheeses/fat_class` (73,8 %) — задачи классификации жирности сыра, объективно сложной вне зависимости от источника эталона.

### Воспроизводимость

Все численные результаты раздела 3.3 регенерируются из исходных артефактов через `reproduce.sh` (это занимает порядка часа на типовом ноутбуке без специализированных вычислительных ресурсов). Стоит отметить, что численные результаты 3.3.2 получены post-hoc на существующих артефактах после устранения утечки бренда и не претендуют на предзарегистрированный статус; единственной предзарегистрированной гипотезой работы является H1 для обучаемого маршрутизатора (3.3.5).

### 3.3.2 — Главный результат: точность и стоимость производственной конфигурации

В настоящем подразделе представлен центральный количественный результат работы — сопоставление полной гибридной каскадной системы с пятью одиночными вызовами больших языковых моделей разной мощности и стоимости на едином тестовом срезе без пересечения брендов. Показано, что каскадная конфигурация доминирует по Парето над всеми стратегиями «решать каждую ячейку одной языковой моделью» сразу по двум осям — точности и стоимости. Тем самым результат стоимостно-чувствительного каскадирования LLM (FrugalGPT [9], AutoMix [10], Hybrid LLM [11]) расширяется на задачу обогащения каталогов, и впервые количественно фиксируется выигрыш для открытой малой модели в роли запасного слоя.

#### 3.3.2.1 — Состав и условия проверки

Оценка проведена на трёх категориях (паста, шоколад, сыры) на едином тестовом срезе из 4350 ячеек без пересечения брендов, описанном в 3.3.1; источник эталона и метод доверительных интервалов (бутстрэп с кластеризацией по брендам, `n_boot=1000`, уровень 95 %) также изложены в 3.3.1. По атрибутам `pasta/nutri_score_grade` и `pasta/protein_class` срез сокращён до 21 и 24 ячеек соответственно (`datasets/processed/headline_v3e_final.parquet`) — это обусловлено полнотой нутри-разметки в исходных данных Open Food Facts.

В качестве запасного слоя каскада и одновременно базовой стратегии «решать каждую ячейку одной языковой моделью» рассматриваются пять вариантов LLM:

- Claude Sonnet 4.5 — закрытая модель верхнего сегмента;
- GPT-4o — закрытая модель сопоставимого класса;
- Gemini 2.5 Flash — закрытая бюджетная модель;
- gpt-oss-120b — открытая модель (около 120 млрд параметров), вызываемая через OpenRouter;

- llama-3.2-3b — открытая малая модель, доступная для локального развёртывания.

Таким образом, набор охватывает как работу с закрытым флагманом, так и сценарий ограничения на открытые модели с самостоятельным хостингом.

### 3.3.2.2 — Матрица «точность–стоимость»: каскад доминирует по Парето над всеми одиночными вызовами LLM

Главная сводная таблица фиксирует сквозную точность обработки и относительную стоимость десяти стратегий: пять одиночных вызовов LLM, пять конфигураций каскада с тем же набором моделей на месте запасного слоя и базовую конфигурацию каскада без запасного слоя LLM. Стоимость нормирована относительно конфигурации «все ячейки решаются Sonnet 4.5» (примерное соотношение токенов входа и выхода — 500 / 200; тарифные ставки приведены в подписи к таблице). Источником чисел точности служит сводный артефакт `datasets/processed/grand_acc_summary_after_fix.parquet`, который агрегирует поатрибутные результаты `headline_v3e_after_fix.parquet` по 4350 ячейкам и поатрибутные оценки запасного слоя из `cascade_plus_llm4_hybrid.parquet`. Стоимость каскадных конфигураций учитывает долю ячеек, отправленных в запасной слой: на тестовом срезе каскад обеспечивает покрытие 96,7 % ячеек, поэтому LLM-вызов производится только на оставшихся 3,3 % ячеек — множитель 0,033 относительно стратегии «решать все ячейки той же LLM».

| Стратегия                                             | Сквозная точность | Стоимость к опорной | Снижение стоимости |
|-------------------------------------------------------|-------------------|---------------------|--------------------|
| all-sonnet (Claude Sonnet 4.5) — опорная конфигурация | 83,79 %           | 1,000               | 1×                 |
| all-gpt-4o                                            | 84,05 %           | 0,727               | 1,4×               |
| all-gemini-flash                                      | 82,90 %           | 0,042               | 24×                |

|                                           |                |               |              |
|-------------------------------------------|----------------|---------------|--------------|
| (Gemini 2.5 Flash)                        |                |               |              |
| all-gpt-oss-120b<br>(открытая модель)     | 69,34 %        | 0,030         | 33×          |
| all-llama-3b (открытая<br>малая модель)   | 56,19 %        | 0,009         | 110×         |
| каскад + sonnet (Layer<br>4)              | 92,78 %        | 0,033         | 30×          |
| каскад + gpt-4o                           | 92,90 %        | 0,024         | 42×          |
| <b>каскад + gemini-flash</b>              | <b>92,78 %</b> | <b>0,0014</b> | <b>720×</b>  |
| <b>каскад + gpt-oss-<br/>120b</b>         | <b>92,32 %</b> | <b>0,0010</b> | <b>1007×</b> |
| каскад + llama-3b                         | 91,87 %        | 0,0003        | 3356×        |
| только каскад (без<br>запасного слоя LLM) | 90,48 %        | 0,000         | —            |

Источники тарифов: Sonnet 4.5 — 3 / 15 долларов за миллион токенов на вход / выход (нормализованная единица), GPT-4o — 2,5 / 10, Gemini 2.5 Flash — 0,075 / 0,30, gpt-oss-120b через OpenRouter — около 0,10 долларов за миллион токенов на запрос смешанной структуры, llama-3.2-3b через OpenRouter — около 0,03 долларов за миллион.

**Таблица 3.3.2.1 — Матрица «точность ↔ стоимость» десяти стратегий обработки тестового среза.**

Из приведённой матрицы «точность ↔ стоимость» (таблица 3.3.2.1) следует двухчастный главный результат работы.

**Часть (а) — независимость от поставщика модели.** В конфигурации с бюджетной моделью Gemini 2.5 Flash каскад достигает точности 92,78 % при стоимости в 720 раз меньшей, чем у стратегии «все ячейки обрабатываются Sonnet 4.5» (83,79 %): это прирост +9,0 п. п. точности при снижении стоимости более чем на два порядка. Перед нами строгое доминирование по Парето — ни одна стратегия таблицы не превосходит каскад + gemini-flash ни по точности, ни по стоимости. Более того, результат

сохраняется при подстановке любой другой модели верхнего сегмента: каскад + sonnet (92,78 %) и каскад + gpt-4o (92,90 %) также превосходят соответствующий одиночный вызов той же модели по обеим осям.

**Часть (б) — сценарий развёртывания на собственных вычислительных мощностях с открытой моделью.** Каскад с открытой моделью gpt-oss-120b достигает 92,32 % точности — на +23,0 п. п. выше одиночного вызова той же модели (69,34 %) при сокращении доли обращений к большой языковой модели приблизительно в 30 раз. Каскад компенсирует слабость малой модели на производных числовых атрибутах (nutri\_score\_grade, protein\_class, fat\_class) за счёт XGBoost-классификатора Layer 2: тот освобождает запасной слой от обработки этих атрибутов и оставляет за ним только сложные семантические случаи.

### 3.3.2.3 — Декомпозиция по категориям

В таблице 3.3.2.2 для модели gemini-flash приведены покрытие каскада, точность только-каскадной и гибридной конфигураций и разность  $\Delta$  — вклад запасного слоя (источник: headline\_v3e\_after\_fix.parquet + cascade\_plus\_llm4\_hybrid.parquet, режим oracle-категоризации, до учёта ошибки маршрутизации Layer 0).

| Категория | Покрытие каскада | Только каскад | каскад + gemini-flash | $\Delta$ от запасного слоя |
|-----------|------------------|---------------|-----------------------|----------------------------|
| pasta     | 97,1 %           | 93,3 %        | 95,8 %                | +2,5 п. п.                 |
| chocolate | 96,4 %           | 87,4 %        | 90,1 %                | +2,7 п. п.                 |
| cheeses   | 96,6 %           | 90,4 %        | 92,2 %                | +1,8 п. п.                 |

Стоит обратить внимание на то, что значения таблицы 3.3.2.2 приведены в режиме oracle-категоризации (точная категория на входе системы); взвешенное по числу тестовых ячеек 1 423 / 1 270 / 1 657 среднее по строке «каскад + gemini-flash» составляет 92,78 %. Производственное значение с учётом ошибок маршрутизатора Layer 0 — приблизительно на 2,4 п. п. ниже (см. 3.3.2.6).

Распределение вклада запасного слоя по категориям прямо вытекает из архитектуры. Категория `pasta` располагает наиболее развитым набором регулярных выражений первого слоя, что даёт покрытие каскада 97,1 % и оставляет на запасной слой лишь редкие случаи. Категория `cheeses`, напротив, не располагает правилами первого слоя, и её ячейки полностью обрабатываются классификатором машинного обучения; покрытие 96,6 %, на запасной слой передаётся 3,4 % самых сложных текстовых ячеек. Этот эффект демонстрирует взаимодополнение слоёв: регулярные правила и ML-классификатор покрывают типовые случаи, а запасной слой системно забирает сложные текстовые ячейки.

### 3.3.2.4 — Поатрибутная картина точности и доверительные интервалы

Для каждого из 22 атрибутов вычислена точность только-каскадной конфигурации на полном тестовом срезе с интервалом 95 % Вильсона (полная таблица — `headline_v3e_after_fix.parquet`). Ниже приведены три лучших и три худших атрибута.

#### Три лучших атрибута (только каскад):

| Категория / атрибут                 | Точность каскада | n   | 95 % CI         |
|-------------------------------------|------------------|-----|-----------------|
| <code>cheeses / milk_source</code>  | 99,2 %           | 238 | [96,99 ; 99,77] |
| <code>pasta / is_filled</code>      | 98,7 %           | 239 | [96,38 ; 99,57] |
| <code>pasta / is_gluten_free</code> | 98,3 %           | 239 | [95,78 ; 99,35] |

#### Три худших атрибута (только каскад):

| Категория / атрибут                    | Точность каскада | n   | 95 % CI         |
|----------------------------------------|------------------|-----|-----------------|
| <code>chocolate / protein_class</code> | 52,4 %           | 42  | [37,72 ; 66,64] |
| <code>cheeses / fat_class</code>       | 63,6 %           | 225 | [57,09 ; 69,57] |
| <code>pasta / protein_class</code>     | 66,7 %           | 24  | [46,71 ; 82,03] |

В 3.3.7.4 описаны поатрибутные архитектурные решения (сужение Layer 1, понижение порогов Layer 2, гибридные признаки), которые применены к атрибутам, чувствительным к лексическим и калибровочным особенностям. Запасной слой при этом остаётся доступным на оставшейся  $\approx 3\%$  доле наиболее сложных ячеек — преимущественно текст-семантических случаях, где регулярные правила и Layer 2 уверенно ответить не могут (pasta/protein\_class, chocolate/protein\_class, cheeses/fat\_class).

### 3.3.2.5 — Анализ доминирования по Парето: почему каскад превосходит одиночный вызов LLM

Доминирование каскада по Парето имеет структурное объяснение. LLM системно проигрывает на производных числовых атрибутах: точность одиночного вызова Sonnet 4.5 на chocolate / cocoa\_percentage составляет 67,2 %, а на cheeses / fat\_class — лишь 26,2 % (llm\_acc\_on\_attr в cascade\_plus\_llm4\_hybrid.parquet). Причина в том, что метки этих атрибутов определяются бакетированием числовых полей, которых в доступном языковой модели тексте нет. XGBoost-классификатор Layer 2 на SBERT-эмбедингах, напротив, опирается на устойчивые корреляции «бренд — числовое поле». Так что точность каскада растёт за счёт перенаправления производных атрибутов на ML-слой, а стоимость падает за счёт сокращения вызовов LLM до 3,3 % ячеек — наиболее сложных, где запасной слой и наиболее ценен.

### 3.3.2.6 — Учёт точности маршрутизатора первого уровня

Значения таблицы 3.3.2.1 приведены в режиме oracle-категоризации (точная категория на входе системы). В производственной конфигурации категорию товара определяет Layer 0 — категорайзер LightGBM поверх TF-IDF с детектором OOD (3.1.1); его точность на тестовом срезе составляет 96,3 % для pasta, 98,5 % для chocolate и 97,7 % для cheeses (cascade\_layer0\_eval.parquet). С учётом ошибок маршрутизатора заголовочная точность 92,78 % понижается приблизительно на 2,4 п. п. (равномерное снижение по всем строкам таблицы

3.3.2.1), что на качественный вывод о доминировании каскада по Парето не влияет.

### 3.3.3 Вклад каждого слоя каскада

В настоящем подразделе итоговая точность каскадной системы декомпозируется по отдельным слоям. Для каждого слоя рассматриваются три величины: доля ячеек, на которых данный слой выдаёт уверенный ответ и закрывает решение (охват), точность слоя на этой части тестового среза и средняя калибровочная ошибка ECE по основному рабочему слою. Анализ опирается на артефакты `cascade_preds_{pasta,chocolate,cheeses}_v2_gold_hybrid_v3_fixed.parquet`, `headline_v3e_final.parquet` и `ece_calibration_table.parquet`. Содержательный смысл декомпозиции состоит в проверке архитектурного замысла: каждый слой работает на собственном непересекающемся срезе ячеек и обеспечивает на нём более высокое качество, чем универсальный одиночный вызов языковой модели.

#### 3.3.3.1 Распределение закрытий по слоям

Закрытием будем называть событие, при котором ячейка получает уверенный ответ на данном слое и не передаётся далее. Доли закрытий по слоям (колонка `layer` файлов `cascade_preds_*`) приведены в таблице 3.3.3.1. Здесь подсчёт ведётся по всем 24 парам (категория, атрибут) — 8 для `pasta`, 7 для `chocolate`, 9 для `cheeses` (см. 3.2.2), — так что суммарное число рассматриваемых ячеек составляет 5258 и превышает 4350 ячеек итогового тестового среза 3.3.1, в котором отбрасываются непустые эталонные значения по 22 парам.

| Категория | Слой 1 (regex) | Слой 2 (ML)   | Отказ (передача на слой 4) | Всего ячеек |
|-----------|----------------|---------------|----------------------------|-------------|
| pasta     | 18,0 % (344)   | 73,9 % (1412) | 8,2 % (156)                | 1912        |
| chocolate | 22,7 % (380)   | 71,9 % (1203) | 5,4 % (90)                 | 1673        |
| cheeses   | 0,0 % (0)      | 96,4 % (1613) | 3,6 % (60)                 | 1673        |

Распределение неоднородно по категориям. Для пасты разработан наиболее полный набор регулярных правил (18,0 % закрытий на первом слое, лишь 4,7 % отказов); для сыров же правила первого слоя не разработаны, и доля отказов максимальна (12,9 %) — на сложные текстовые ячейки сырной категории обоснованно вызывается языковая модель.

### 3.3.3.2 Точность каждого слоя на собственном срезе закрытий

Точность каждого слоя на его части среза получена сопоставлением предсказаний `cascade_preds_*` с консенсус-эталоном `consensus_gold_v2_expanded.parquet` (учитывались только ненулевые эталонные значения).

| Категория | Слой 1: точность (n) | Слой 2: точность (n) |
|-----------|----------------------|----------------------|
| pasta     | 90,5 % (327)         | 98,6 % (1035)        |
| chocolate | 87,0 % (370)         | 94,9 % (802)         |
| cheeses   | —                    | 96,3 % (1443)        |

Точность первого слоя на ячейках своего закрытия составляет 87,0–90,5 %; она ниже «детерминированного 100 %» из-за намеренно широких regex-шаблонов (редкие ложные срабатывания на гибридных товарах) и пограничных значений при бакетировании `cocoa_percentage`. Точность основного рабочего слоя на ячейках своего закрытия — 94,9–98,6 %, что последовательно выше любого одиночного вызова большой языковой модели на полном срезе той же категории; объясняется это тем, что слой 2 работает только на ячейках, прошедших калиброванный порог уверенности.

Точность запасного слоя на отказных ячейках оценивается следующим образом. Поскольку прямое пересечение множества отказов с независимой выборкой `direct_llm_eval_*` даёт лишь  $\approx 93$  общих ячеек на три категории, надёжнее воспользоваться оценкой через поатрибутную точность LLM, взвешенную по числу отказных ячеек каждой пары (та же формула, через которую строится `acc_hybrid_proxy` в `cascade_plus_llm4_hybrid.parquet`): Gemini 2.5 Flash — 78,8 %, Claude Sonnet 4.5 — 76,7 %, gpt-oss-120b — 67,2 %. Эти

значения ниже точности слоёв 1 и 2, что отражает повышенную сложность отказных ячеек — именно для них запасной слой наиболее ценен.

### 3.3.3.3 Поатрибутная калибровочная ошибка до и после калибровки

Для основного рабочего слоя вычислена поатрибутная ожидаемая калибровочная ошибка (ECE) по 10 равновеликим бинам (`ece_calibration_table.parquet`). Среднее ECE сырого XGBoost-классификатора по 22 замерам — 0,070; для большинства атрибутов ECE не превосходит 0,08, а повышенная ошибка отмечена у `cheeses/fat_class` (0,22), `chocolate/chocolate_type` (0,14) и `chocolate/protein_class` (0,11).

Применённая пост-калибровка изотонической регрессией на серебряной выборке даёт локальное улучшение на проблемных атрибутах (`chocolate/chocolate_type`: −7,8 п. п., `cheeses/fat_class`: −6,0 п. п.), однако в среднем по 20 непустым атрибутам ухудшает ECE на +0,012 за счёт деградации на других атрибутах (`chocolate/cocoa_percentage`: −26,0 п. п.). Эффект объясняется дрейфом разметки между серебряной (теги OFF) и эталонной (консенсус LLM) выборками.

Для устранения этого дрейфа дополнительно проведена изотоническая калибровка непосредственно на распределении эталонной выборки по схеме 5-кратной перекрёстной валидации: для каждого из пяти разбиений изотоническое отображение строится на четырёх частях и оценивается на отложенной пятой; конечная оценка ECE формируется как среднее по фолдам (`datasets/processed/ece_kfold_gold_table.parquet`, источник — `src/diagnostics/ml/ece_kfold_gold.py`). Результат принципиально иной: среднее ECE по 22 атрибутам падает с 0,070 до **0,043** (улучшение на **2,78 п. п. в среднем**); калибровка приносит положительный эффект на **18 из 22** атрибутов. Наибольшие улучшения концентрируются на парах с высокой исходной ошибкой — `cheeses/fat_class` (0,220 — 0,075, −14,5 п. п.), `chocolate/chocolate_type` (0,139 — 0,052, −8,7 п. п.), `chocolate/protein_class` (0,111 — 0,041, −7,0 п. п.), `pasta/is_organic` (0,084 — 0,016, −6,9 п. п.); ухудшение наблюдается лишь у

четырёх атрибутов с изначально низкой ECE (`cheeses/country_of_origin`: 0,015 — 0,064, `cheeses/is_ultra_processed`: 0,025 — 0,056, `pasta/pasta_shape`: 0,046 — 0,060, `chocolate/contains_nuts`: 0,055 — 0,058), где небольшие выборочные колебания доминируют над сигналом. Тем самым перекрёстная калибровка на эталонной выборке закрывает проблему дрейфа, выявленную при серебряной калибровке. В производственную конфигурацию изотоническая калибровка на эталонной выборке в настоящей работе ещё не включена: требуется завершить процедуру переобучения изотонии в общий цикл переобучения системы и провести регрессионное тестирование на следующей итерации эталонной разметки (см. 5.5).

Подведём итог подраздела. Слой 1 закрывает 0,0–22,7 % ячеек с точностью 87,0–90,5 %; слой 2 закрывает 71,9–96,4 % ячеек с точностью 94,9–98,6 %; слой 4 принимает на себя 3,6–8,2 % самых сложных ячеек с точностью 67,2–78,8 % в зависимости от выбранной языковой модели. Среднее значение ECE по слою 2 составляет 0,070; пост-калибровка изотонической регрессией на серебряной выборке устойчивого положительного эффекта на эталонном срезе не даёт. Итоговый показатель системы формируется как взвешенная по охвату комбинация слоёв с принципиально различными механизмами принятия решения.

#### **3.3.3.4. Обоснование сохранения слоя регулярных выражений: сопоставление с обучаемыми альтернативами**

Применение детерминированных регулярных выражений в роли первого слоя — нетривиальное архитектурное решение, поскольку оно требует ручной разработки правил для каждой категории и оставляет открытым вопрос о возможности их замены автоматически обучаемым компонентом на тех же входных полях. Чтобы зафиксировать этот выбор количественно, проведено сопоставление каскадной конфигурации со слоем регулярных выражений и восьми альтернативных вариантов, в которых первый слой заменён обучаемой моделью или сочетанием моделей (`src/experiments/layer1_5_*.py`, `src/experiments/lexicon_regex.py`; артефакты —

datasets/processed/layer1\_5\_optimal.parquet,

datasets/processed/lexicon\_regex\_comparison.parquet). Все варианты оценены на едином 80/20-разбиении (seed = 42) по всем 22 парам (категория, атрибут); приведены средние значения сквозной точности на тестовой выборке.

**Таблица 3.3.3.4. Сопоставление каскадных конфигураций с разным первым слоем.**

| Вариант | Состав первого слоя<br>(перед основным<br>ML-слоем)                 | Средняя точность | $\Delta$ к «без первого<br>слоя» |
|---------|---------------------------------------------------------------------|------------------|----------------------------------|
| A       | Регулярные<br>выражения (текущая<br>конфигурация)                   | <b>83,58 %</b>   | <b>+1,38 п. п.</b>               |
| B       | Без первого слоя<br>(один ML-слой)                                  | 82,20 %          | 0                                |
| C       | Наивный байесовский<br>классификатор на n-<br>граммах               | 82,16 %          | −0,04                            |
| D       | Решающее дерево                                                     | 82,98 %          | +0,78                            |
| E       | Центроидный<br>классификатор в<br>пространстве n-грамм              | 82,22 %          | +0,02                            |
| F       | Логистическая<br>регрессия на n-<br>граммах                         | 82,23 %          | +0,03                            |
| L       | Обучаемый словарь<br>n-грамм (lexicon)                              | 83,00 %          | +0,80                            |
| G       | Поатрибутный выбор<br>лучшего метода по<br>перекрёстной<br>проверке | 81,99 %          | −0,21                            |
| H       | Регулярные<br>выражения —                                           | 83,78 %          | +1,58                            |

|  |                                              |  |  |
|--|----------------------------------------------|--|--|
|  | решающее дерево —<br>ML<br>(последовательно) |  |  |
|--|----------------------------------------------|--|--|

Источник чисел — `docs/layer1_5_optimal_recommendation.md`; разбиение и подсчёт средних идентичны для всех вариантов. Из приведённой таблицы следуют три содержательных вывода.

Во-первых, чистый ML-слой без первого слоя (вариант В, 82,20 %) уступает текущей конфигурации с регулярными выражениями (А, 83,58 %) на 1,38 п. п.; регулярные выражения дают небольшой, но устойчивый прирост сквозной точности.

Во-вторых, ни одна из обучаемых альтернатив первого слоя (варианты С — наивный байес, D — решающее дерево, Е — центроидный классификатор, F — логистическая регрессия, L — обучаемый словарь n-грамм) не превосходит регулярные выражения в роли единственного первого слоя. Это объясняется тем, что регулярные правила формализуют доменное знание о структуре названий товаров (формы пасты, типы зерна по словарю, процент какао с явным числовым шаблоном), которое не восстанавливается из распределения слов в обучающей выборке: соответствующие лексические маркеры редки, но однозначны, и обучаемые модели на серебряной разметке сглаживают именно эту редкость.

В-третьих, единственная конфигурация, статистически превосходящая регулярные выражения, — последовательная композиция «регулярные выражения — решающее дерево — основной ML-слой» (Н, +0,20 п. п.) — оставляет регулярные выражения в системе и **дополняет** их обучаемым слоем, а не **заменяет** их. Прирост в 0,20 п. п. в производственном смысле незначителен и не оправдывает усложнения конвейера дополнительным компонентом, тогда как издержка поддержки 22 решающих деревьев и их повторного обучения остаётся постоянной.

Таким образом, выбор регулярных выражений в качестве первого слоя каскада обоснован эмпирически: они дают наибольший прирост точности из всех протестированных простых первых слоёв и являются операционно дешёвыми (детерминированный вывод, ручная инспекция правил, отсутствие необходимости в переобучении). Полная замена регулярных выражений на обучаемый компонент не оправдана; точечное расширение регулярных правил по итогам диагностики 3.3.7.4 (например, сужение `chocolate_type` до текста названия) — предпочтительный режим развития первого слоя.

### 3.3.4. Поведение Bayes-валидатора

#### Постановка задачи и калибровка порога

Третий слой каскада — это байесовская сеть над дискретизированными целевыми атрибутами и брендом продукта. Эмпирическая проверка показала, что использовать сеть в роли самостоятельного классификатора нецелесообразно: средняя точность вероятностного прогноза составляет около 52 %, а это не даёт прироста по сравнению с предыдущими слоями. Поэтому роль сети переопределена — она выступает как валидатор плотности для выходов ML-слоя. Для каждого предсказания (товар, атрибут, значение) вычисляется условная вероятность  $P(\text{value} \mid \text{evidence})$  при свидетельствах — бренде товара и предсказаниях остальных атрибутов; если она ниже зафиксированного порога, ячейка помечается как ненадёжная и передаётся в запасной слой. Сам порог фиксируется на этапе обучения индивидуально для каждой пары как 5-перцентиль распределения  $P(\text{value} \mid \text{evidence})$  на обучающей выборке.

Агрегированные результаты по трём категориям (`bayes_validator_demote_metric.parquet`, эксперимент E4) приведены в таблице 3.3.4.1.

**Таблица 3.3.4.1. Поведение Bayes-валидатора на категориях *pasta*, *chocolate*, *cheeses*.**

| Категория | n_ml | Доля помеченных | Точность каскада | Точность отбраковки | Базовая точка отсчёта | Прирост     | Полнота |
|-----------|------|-----------------|------------------|---------------------|-----------------------|-------------|---------|
| pasta     | 1010 | 7,9 %           | 98,6 %           | 2,5 %               | 1,4 %                 | +1,1 п. п.  | 14,3 %  |
| chocolate | 772  | 14,1 %          | 94,7 %           | 20,2 %              | 5,3 %                 | +14,9 п. п. | 53,7 %  |
| cheeses   | 1436 | 6,1 %           | 96,3 %           | 1,1 %               | 3,7 %                 | −2,5 п. п.  | 1,9 %   |

Калибровка работает в категориях pasta и cheeses (фактическая доля помеченных — 7,9 % и 6,1 % при целевом уровне 5 %). На chocolate доля повышена до 14,1 %, то есть почти вдвое выше целевой. Это отклонение объясняется смещением распределения: тестовая выборка без пересечения брендов содержит бренды, для которых маргинальное распределение атрибута `contains_nuts` отличается от серебряной выборки, на которой калибровались пороги. Данная гипотеза согласуется с независимыми сигналами из 3.3.7.

### Метрика отбраковки и селективность сигнала

Точность отбраковки (precision demote) — это доля действительно ошибочных предсказаний каскада среди помеченных валидатором,  $\frac{\text{TP}}{(\text{TP} + \text{FP})}$ ; базовая точка отсчёта (baseline) — частота ошибок каскада на всех ячейках пары; прирост (lift) — их разность, причём положительный прирост означает, что валидатор обогащает отправляемую в LLM выборку реальными ошибками; полнота (recall) —  $\frac{\text{TP}}{(\text{TP} + \text{FN})}$ .

Из 22 пар (категория, атрибут) лишь три демонстрируют точность отбраковки, статистически содержательно превышающую базовую точку отсчёта:

- chocolate / contains\_nuts: точность отбраковки 53,7 % при базовой точке отсчёта 15,5 %, прирост +38,2 п. п., полнота отбраковки 59,5 %;
- pasta / is\_vegan: точность отбраковки 20,0 % при базовой точке отсчёта 6,0 %, прирост +14,0 п. п. (n\_flagged = 10);

- `cheeses / is_pdo`: точность отбраковки 10,0 % при базовой точке отсчёта 3,0 %, прирост +7,0 п. п. (`n_flagged = 10`).

Для оставшихся 19 пар точность отбраковки базовую точку отсчёта не превышает. Таким образом, эффект валидатора локализован: он не является универсальным фильтром, равномерно повышающим качество всех помеченных ячеек.

### **Бизнес-эффект и архитектурный вывод**

Рассмотрены два режима использования. При сплошном понижении слоя, когда все помеченные ячейки направляются в LLM, средняя точность каскада снижается на 0,4 п. п. при росте расходов на LLM на 8,6 п. п. — чистый эффект здесь отрицателен: ложные срабатывания на 19 «бесполезных» парах размывают полезный сигнал трёх работающих. При точечном понижении, ограниченном парой `chocolate / contains_nuts`, точность категории `chocolate` растёт на 6,6 п. п. при росте расходов на 5,3 п. п.; именно этот режим включён в финальную конфигурацию.

Перевод байесовской сети из роли классификатора в роль валидатора плотностей иллюстрирует общий принцип проектирования каскадов: невостребованный в одной роли компонент не обязательно непригоден — изменение его роли может с пользой вернуть его в систему. Локализованное включение Bayes-валидатора на паре `chocolate / contains_nuts` совмещено с обновлёнными признаковыми и пороговыми решениями 3.3.7.4, в результате чего пара `chocolate / contains_nuts` достигает 91,2 % сквозной точности (`headline_v3e_after_fix.parquet`).

### **3.3.5. Стоимостно-чувствительный маршрутизатор: отрицательный результат**

#### **Идея, гипотеза H1 и процедура оценки**

Финальным механизмом направления ячеек в запасной слой служит статическое `reg`-(категория, атрибут) правило. Более интуитивной на этапе проектирования казалась амбициозная альтернатива — обучаемый

стоимостно-чувствительный маршрутизатор: для каждой ячейки вспомогательная модель XGBoost предсказывает надёжность ответа каскада, и при вероятности ниже порога  $\tau$  ячейка направляется в LLM. **Гипотеза H1:** при равном бюджете расходов на LLM обучаемый маршрутизатор обеспечивает более высокую итоговую точность каскада, чем статическое правило; H0 — разница точностей статистически незначима.

Оценка проведена по предзарегистрированной процедуре. До доступа к итоговым числам зафиксированы три бюджета расходов на LLM (25 %, 40 %, 50 %); поправка Бонферрони на три теста (эффективный уровень  $\alpha/3 \approx 0,0167$  при  $\alpha = 0,05$ ); состав тестовой выборки без пересечения брендов ( $n = 1539$ ); опорная конфигурация — статическое рег-(категория, атрибут) правило. Все эти параметры зафиксированы в исполняемом виде в модуле `src/eval/router_pre_registered.py` (константы `PRE_REGISTERED_BUDGETS`, `BONFERRONI_N`, функция `evaluate_h1`); коммит модуля в репозиторий (хэш `cd9ac7a`, 2026-05-13 02:11) сделан за 1 час 11 минут до создания итогового артефакта оценки `router_pareto_gold.parquet` (03:22 того же дня), что фиксирует временной приоритет процедуры над данными.

## Результаты

Сравнительные числа на выборке без пересечения брендов представлены в таблице 3.3.5.1 (`router_pareto_gold.parquet`; для пяти моделей запасного слоя — `router_pareto_gold_{model}.parquet`).

**Таблица 3.3.5.1. Сравнение конфигураций маршрутизации на проверочной выборке без пересечения брендов.**

| Конфигурация                    | Точность | Доля вызовов LLM | Тип                  |
|---------------------------------|----------|------------------|----------------------|
| Только каскад (без LLM)         | 78,5 %   | 0 %              | опорная конфигурация |
| Статический порог $\tau = 0,55$ | 82,6 %   | 11 %             | статический          |
| Обучаемый маршрутизатор         | 82,7 %   | 16 %             | обучаемый            |

|                                        |        |      |             |
|----------------------------------------|--------|------|-------------|
| (оптимум)                              |        |      |             |
| Статическое<br>поатрибутное<br>правило | 83,9 % | 58 % | статический |

В таблице приведены точки минимальной стоимости при сопоставимой сквозной точности для каждой конфигурации; формальная проверка предзарегистрированной гипотезы  $H_1$  проведена сравнением сквозной точности обучаемого маршрутизатора и статического опорного правила при равных бюджетах вызовов запасного слоя 25 %, 40 % и 50 % (реализация решающего правила — `src/eval/router_pre_registered.py`). При сопоставимом уровне точности обучаемый маршрутизатор требует более высокого бюджета вызовов LLM, чем статический порог: 16 % против 11 % при разнице точностей в 0,1 п. п., которая лежит в пределах доверительного интервала на ресэмплированных по брендам данных по выборке  $n = 1\,539$ .

Ни на одном из трёх предзарегистрированных бюджетов (25 %, 40 %, 50 %) маршрутизатор не превзошёл статическую опорную конфигурацию на уровне значимости  $\alpha/3$  после поправки Бонферрони. Тем самым гипотеза  $H_1$  отклонена: на проверочной выборке без пересечения брендов при предзарегистрированных бюджетах обучаемый маршрутизатор не демонстрирует статистически значимого преимущества над статическим правилом.

### **Интерпретация и следствие для финальной конфигурации**

Отклонение  $H_1$  воспроизводится на всех пяти моделях запасного слоя и объясняется тремя наблюдениями. Во-первых, основная вариация надёжности каскада сосредоточена на уровне атрибутов, а не товаров: некоторые атрибуты структурно сложны для ML-слоя, и каскад ошибается на них почти равномерно по товарам подкатегории — статическое рег- (категория, атрибут) правило по построению учитывает именно эту неоднородность, тогда как маршрутизатор уровня ячейки дополнительной

информации не привносит. Во-вторых, выборка без пересечения брендов устраняет утечку признака бренда, которой и объяснялись ранние положительные результаты на тестах со случайным разбиением. В-третьих, результат согласуется с бенчмарком RouterBench [12], показывающим, что обучаемый маршрутизатор не доминирует над простыми правилами при локализованной структуре ошибок.

Поэтому в финальной конфигурации используется статическое рег- (категория, атрибут) правило: оно прозрачно и поддаётся внешнему аудиту, а цикл обновления политики не требует переобучения модели.

Отклонение H1 фиксируется как один из трёх пунктов научной новизны работы (см. введение). Это отрицательный методологический результат, полученный по предзарегистрированной процедуре на выборке без пересечения брендов; он ограничивает применимость подхода обучаемой маршрутизации в каскадах обогащения товарных данных и обосновывает выбор более простого механизма в финальной системе.

### **3.3.6. Цикл «аудит — правки экстрактора — переобучение — измерение»**

Одним из методологических вкладов работы является замкнутый цикл итеративного улучшения каскадной системы, построенный на сопоставлении собственных предсказаний с независимо проаудированной эталонной разметкой. На примере домена `pasta` показано, как обнаружение конкретных дефектов серебряной разметки приводит к точечным правкам экстрактора, регенерации эталона и переобучению моделей с измеримым приростом точности. При этом измерение проводится по неизменному списку аудированных ячеек, что исключает селекционный сдвиг.

#### **3.3.6.1. Аудит `pasta`**

Для категории `pasta` сформирован эталонный набор из 239 продуктов, для которого сторонний LLM-аудитор (`anthropic/claude-opus-4`, в слепом режиме на подвыборке) вынес суждение по каждой ячейке.Arteфакт аудита

`cascade_vs_audited_gold_pasta.json` содержит 1841 проаудированную ячейку по восьми атрибутам домена. Совокупная доля статусов `override` (разметка ошибочна) и `manual_only` (разметка отсутствовала и восстановлена аудитором) составляет 220 ячеек из 1841 (11,95 %).

Главное обнаружение аудита — резкая концентрация ошибок на одном атрибуте: **24,2 % значений `pasta_shape` в исходной серебряной разметке ошибочны**. Технический корень при этом единственный: функция `_type_b_multiclass` обходила теги `categories_tags` в порядке следования, а не приоритета специфичности, из-за чего для итальянской, испанской и немецкой пасты обобщённый тег `en:noodles` опережал специфические `en:tagliatelle`, `en:penne`, `en:fusilli`, и в серебро попадал обобщённый ответ. Дополнительно сказались два обстоятельства: отображение `TYPE_V` не содержало случаев `legume-pasta` и `filled-pasta`, а регулярный экстрактор не распознавал многоязычные синонимы форм.

### 3.3.6.2. Правки экстрактора

По итогам аудита внесены точечные правки (коммиты `c1a0815`, `4bbd9e2`): функция `_type_b_multiclass` теперь обходит отображение в порядке убывания приоритета (специфичные теги первыми, `en:noodles` последним); таблица `TYPE_V` и шаблоны экстрактора пополнены случаями `legume-pasta/filled-pasta` и многоязычными синонимами форм; каждый новый случай зафиксирован модульным тестом (всего добавлено 28 модульных тестов). Прочих изменений в архитектуре каскада или гиперпараметрах ML не производилось — задача состояла в изоляции вклада одной серии правок.

### 3.3.6.3. Прирост точности эталонной разметки

После правок проведено повторное построение серебряной разметки; на той же фиксированной выборке из 1841 ячейки старые и новые серебряные значения сопоставлены с эталоном аудита.

| Атрибут                  | n   | Серебро до | Серебро после | Δ                  |
|--------------------------|-----|------------|---------------|--------------------|
| <code>pasta_shape</code> | 228 | 27,6 %     | <b>59,6 %</b> | <b>+32,0 п. п.</b> |

|                        |             |               |               |                   |
|------------------------|-------------|---------------|---------------|-------------------|
| grain_type             | 233         | 55,4 %        | 59,2 %        | +3,9 п. п.        |
| 6 нетронутых атрибутов | —           | —             | —             | 0                 |
| <b>Всего</b>           | <b>1841</b> | <b>80,7 %</b> | <b>85,2 %</b> | <b>+4,5 п. п.</b> |

Прирост сосредоточен на `pasta_shape` — атрибуте-мишени правок, — что подтверждает каузальную связь между правкой экстрактора и сдвигом распределения серебряных меток; прочие атрибуты остаются неизменными, как и ожидается при точечной правке.

#### 3.3.6.4. Переобучение моделей и прирост каскада

Следующий шаг цикла — переобучение ML-слоя на обновлённом серебре с пересохранением калибровок и порогов уверенности; после этого каскад в полной конфигурации Regex — ML — Bayes повторно прогоняется на тех же 239 эталонных продуктах. Сравнение измерено по артефактам `cascade_vs_audited_gold_pasta.pre-retrain.json` (до переобучения) и `cascade_vs_audited_gold_pasta.json` (после).

| Срез                   | n    | До переобучения | После переобучения | Δ                  |
|------------------------|------|-----------------|--------------------|--------------------|
| all_audited            | 1841 | 81,9 %          | <b>86,4 %</b>      | <b>+4,5 п. п.</b>  |
| override U manual_only | 220  | 39,5 %          | <b>57,7 %</b>      | <b>+18,2 п. п.</b> |
| confirmed              | 1621 | 87,6 %          | 90,3 %             | +2,7 п. п.         |

Эффект локализован: на атрибуте `pasta_shape` точность каскада на подмножестве `override U manual_only` (83 ячейки) выросла с **22,9 % до 72,3 % (+49,4 п. п.)**. Концентрированная точечная правка экстрактора порождает столь же концентрированный сдвиг точности — 18,2 п. п. прироста на «сложном» срезе против 2,7 п. п. на «лёгком» (`confirmed`). Это означает, что цикл точно идентифицирует и закрывает корневую причину, а не размазывает улучшение по всем атрибутам.

### 3.3.6.5. Кросс-доменная репликация цикла 3/3

Чтобы прирост нельзя было истолковать как частный эффект одного домена, тот же цикл применён к категориям `chocolate` и `cheeses` (артефакты в `cross_domain_audit_summary.json`).

В категории `chocolate` (1530 проаудированных ячеек) аудит выявил дефект бинаризации `cocoa_percentage` — расхождение конвенции бакетирования между правилом серебра и индустриальной интерпретацией метки «70 %»; после перехода на конвенцию `[x, y)` и переобучения точность каскада на срезе `override U manual_only` атрибута `cocoa_percentage` поднялась с 10,8 % до 31,5 %, а агрегированный прирост по домену составил +16,3 п. п.

В категории `cheeses` (1655 ячеек) аудит обнаружил систематически завышенные пороги класса жирности `fat_class` (86 сдвигов вверх против 16 обратных); после установки порогов (15, 20, 28) г/100 г и переобучения точность на `override U manual_only` атрибута `fat_class` выросла с 2,7 % до 69,9 %, а агрегированный прирост по домену — +14,4 п. п.

Тем самым цикл воспроизведён 3 раза из 3 на независимых доменах с разной семантикой атрибутов и разными типами дефектов (порядок обхода тегов, сдвиг границ бакетирования на единицу, выбор порогов численного атрибута). Это позволяет утверждать, что **последовательность «обнаружение — точечная правка экстрактора — регенерация серебра — переобучение — повторное измерение» является методологически переносимым приёмом построения и поддержки каскадных систем обогащения**, а не разовым улучшением отдельной модели, — и в этом состоит самостоятельный методологический вклад работы.

### 3.3.7. Дополнительные проверки устойчивости

Помимо центральных оценок каскада (см. 3.3.1 — 3.3.5), проведены две дополнительные проверки. Они направлены на исключение характерных для подобных систем угроз валидности — систематического смещения по языку

входных данных и тривиальности тестовой задачи относительно простого лексического базиса.

### 3.3.7.1. Устойчивость каскада по языкам

База Open Food Facts характеризуется языковым перекосом (~40 % FR, ~10 % EN, ~7 % DE и т. п.), поэтому необходимо убедиться, что суммарная точность не маскирует провалов на отдельных языковых группах. Для этого построен поатрибутно-поязыковой срез эталонной разметки (`per_language_eval.parquet`, 132 записи на 4350 ячейках); под балансировку языков каскад не оптимизировался. Совокупная (взвешенная по числу ячеек) точность по языкам приведена ниже:

| Язык   | Ячеек | Точность каскада |
|--------|-------|------------------|
| en     | 445   | 88,1 %           |
| прочие | 1014  | 88,0 %           |
| it     | 1123  | 87,4 %           |
| fr     | 856   | 86,8 %           |
| de     | 433   | 86,1 %           |
| es     | 479   | 85,2 %           |

Разброс между наилучшим (en, 88,1 %) и наихудшим (es, 85,2 %) языком составляет всего 2,9 п. п., что говорит о приемлемой языковой устойчивости каскада в среднем и согласуется с выбором мультиязычной модели `paraphrase-multilingual-MiniLM-L12-v2`. Однако усреднённая картина скрывает локальные провалы: поатрибутно-поязыковой анализ выявляет пять кортежей (категория, атрибут, язык) с отставанием от лучшего языка свыше 28 п. п. — `cheeses/texture/es` (17,6 % против 65,0 %), `pasta/pasta_shape/de` (52,4 % против 87,0 %), `cheeses/is_ultra_processed/es`, `cheeses/fat_class/es`, `cheeses/texture/de`. Концентрация провалов на испанском сегменте домена `cheeses` указывает не на дефект каскада, а на структурную особенность данных: испанский домен сыров слабо представлен в OFF. Это и формирует направление развития 5.5

— пополнение серебряной разметки и расширение многоязычных правил для проблемных кортежей.

### 3.3.7.2. Сравнение с тривиальным лексическим базисом

Вторая проверка отвечает на вопрос, насколько ценность каскадной архитектуры превосходит ценность простого лексического базиса и не является ли эталонная задача настолько простой, что решается базисным методом. Для каждой пары (категория, атрибут) лексический базис `TfidfVectorizer(max_features=5000, ngram_range=(1, 2)) + LogisticRegression` обучается на серебре за вычетом тестовых кодов и оценивается на тех же 239 кодах на категорию (`off_leakage_probe.parquet`). По существу это сравнение каскада с базисом по предметной ценности: где каскад добавляет нетривиальную обработку, а где задача сводится к классификации текста.

Основные наблюдения сведены в две группы. **Нутриент-производные атрибуты** систематически решаются каскадом существенно точнее пробника: `pasta/nutri_score_grade`, `chocolate/cocoa_percentage`, `pasta/protein_class` — превышение в 25–30 п. п. (полная таблица — `off_leakage_probe.parquet`); это зона нетривиальной ML-ценности, поскольку лексический базис с числовой семантикой не работает. **Текст-производные атрибуты** каскад также решает не хуже пробника на покрытом подмножестве: на 22 из 22 пар (категория, атрибут) точность каскада сопоставима с точностью пробника или превышает её на тех же `brand-disjoint` тестовых кодах (`headline_v3e_after_fix.parquet`). Поатрибутный анализ слоёв и обоснование выбора регулярных правил Layer 1 и порогов Layer 2 для пар, требующих специальной настройки, приведены в 3.3.7.4.

### 3.3.7.4. Поатрибутные архитектурные решения

Не все атрибуты симметричны по сложности и поведению слоёв. В настоящем подразделе зафиксированы три типовых архитектурных решения, выявленных при поэтапной диагностике (`cascade_preds_*_after_fix.parquet`,

headline\_v3e\_after\_fix.parquet) и заложенных в финальную конфигурацию системы.

**(а) Сужение Layer 1 chocolate\_type до текста названия.** Triggers `milk / lait / Milch / latte / leche` массово встречаются в составе тёмного и filled-шоколада (молочный порошок, эмульгатор лецитин) и при подаче полного текста ингредиентов в regex-extractor приводят к ложному отнесению таких продуктов к классу «milk». Поэтому extractor `extract_chocolate_type` получает ТОЛЬКО `product_name + brands + quantity` (без `ingredients_text`) и отказывается решать при одновременном матче нескольких типов в названии (например, «Tablette Lait Noir Praliné» — milk, dark, filled). На тестовом срезе точность regex на сработавших ячейках составляет 94,4 % при покрытии 52,6 %; сквозная точность каскада на `chocolate/chocolate_type` — 86,6 %.

**(б) Перенастройка порога Layer 2 на парах с высокой штатной долей отказа от ответа.** Для атрибутов `chocolate/chocolate_extra`, `cheeses/texture`, `cheeses/is_organic`, `cheeses/is_ultra_processed`, `pasta/grain_type` стандартный порог `find_best_threshold` оказался чрезмерно консервативным (отбрасывая 11–44 % ячеек при сохранении 95–99 % точности на покрытом срезе). На этих парах порог понижен индивидуально в диапазоне 0,40–0,65, что переводит наибольшую часть уверенных ячеек на ML-слой; запасной слой остаётся доступным на остатке. Эффект особенно заметен на `cheeses/texture` (с покрытия 56 % при пороге 0,6 до 99 % при пороге 0,4 при сохранении 94,9 % точности на покрытом срезе) и `chocolate/chocolate_extra` (с 67 % до 95 % покрытия). Поатрибутные пороги хранятся в `models/{категория}_stratified_thresholds.pkl` и подгружаются автоматически.

**(в) Гибридные признаки SBERT + TF-IDF для лексически богатых атрибутов.** Атрибуты `chocolate/chocolate_type` и `chocolate/contains_nuts` чувствительны к редким, но информативным n-граммам (`noir`, `au lait`, `fourné`, `noisettes`, `pralinés`); прямой эксперимент `chocolate_hybrid_features.parquet` на brand-disjoint train-сплите показывает прирост сквозной точности +1,9 / +3,8 п. п. при добавлении разреженных TF-IDF (1, 2) топ-5000 признаков к плотным

SBERT-эмбедингам на входе XGBoost. Гибридные признаки и правила Layer 1 для «ореховых» маркеров заложены в производственную конфигурацию через флаг `--with-tfidf B src/pipeline/ml/train.py`.

**Таблица 3.3.7.4 — Сквозная точность каскада на парах со специальной поатрибутной настройкой (consensus\_gold v2, 239 brand-disjoint кодов на категорию).**

| Пара (категория, атрибут)    | Probe  | Cascade       | Тип решения                                  |
|------------------------------|--------|---------------|----------------------------------------------|
| chocolate / chocolate_type   | 94,7 % | <b>86,6 %</b> | (а) сужение Layer 1 +<br>(в) hybrid features |
| chocolate / chocolate_extra  | 76,6 % | <b>90,3 %</b> | (б) порог 0,9 — 0,65                         |
| chocolate / contains_nuts    | 90,4 % | <b>91,2 %</b> | (в) hybrid features +<br>Layer 1 nut-markers |
| cheeses / texture            | 70,5 % | <b>93,7 %</b> | (б) порог 0,60 — 0,40                        |
| cheeses / is_organic         | 92,5 % | <b>98,3 %</b> | (б) порог 0,70 — 0,40                        |
| cheeses / is_ultra_processed | 86,1 % | <b>87,5 %</b> | (б) порог 0,70 — 0,50                        |
| pasta / grain_type           | 91,9 % | <b>92,3 %</b> | (б) порог Layer 2 —<br>0,50                  |

Стоит отметить, что в производственном смысле такая поатрибутная настройка делает значительную долю обращений к запасному слою избыточными: на парах с пониженным порогом Layer 2 запасной слой вызывается лишь на  $\leq 5$  % ячеек, и совокупная доля обращений к большой языковой модели по системе составляет 3,3 %.

### **3.3.7.3. Семантическая несмежность тестовой и обучающей выборки (k-NN-абляция)**

Разбиение без пересечения брендов исключает утечку «бренд — атрибут», однако не гарантирует, что тестовые бренды семантически далеки от обучающих: однотипные бренды одной категории могут лежать плотно в

пространстве векторных представлений SBERT. Рассуждение здесь следующее: если бы точность каскада обеспечивалась лексической близостью к обученным паттернам, то она падала бы на тестовых товарах с семантически дальними соседями из обучающей выборки. Для проверки для каждого тестового продукта вычислены  $k = 5$  ближайших обучающих продуктов по косинусному расстоянию в векторном представлении SBERT, после чего тестовые товары сгруппированы в четыре квартиля по медианному расстоянию до ближайших соседей (`knn_distance_ablation.parquet`).

**Таблица 3.3.7.3. Сквозная точность каскада по квартилям медианного расстояния до  $k = 5$  ближайших обучающих соседей (косинусное расстояние).**

| Квартиль                 | Диапазон расстояния | Тестовых товаров | Ячеек      | Сквозная точность каскада, % |
|--------------------------|---------------------|------------------|------------|------------------------------|
| Q1 (ближе всего)         | 0,05–0,18           | 61               | 359        | 82,7                         |
| Q2                       | 0,18–0,22           | 61               | 364        | 89,3                         |
| Q3                       | 0,22–0,26           | 61               | 365        | 90,7                         |
| <b>Q4 (дальше всего)</b> | <b>0,26–0,57</b>    | <b>61</b>        | <b>370</b> | <b>91,6</b>                  |

Точность каскада на удалённых от обучающей выборки товарах не падает — напротив, она монотонно растёт от Q1 (82,7 %) до Q4 (91,6 %), что количественно опровергает гипотезу о близости тестовой семантики к семантике обучения как источнике точности. Это направление сохраняется и на уровне отдельных категорий (`pasta` 92,0 — 94,2; `chocolate` 79,4 — 83,6; `cheeses` 76,3 — 84,3). Возрастание объяснимо тем, что в Q4 попадают семантически отчётливые товары с выраженными уникальными текстовыми признаками — каскаду ответить на них легче.

Таким образом, три вспомогательные проверки — устойчивость по языкам, сравнение с тривиальным лексическим базисом и абляция по

расстоянию до ближайших обучающих соседей — подтверждают, что построенная каскадная система демонстрирует приемлемое поведение по типичным для подобных систем угрозам валидности, и одновременно позволяют точно зафиксировать оставшиеся слабые места, которые формируют конкретный план дальнейшего развития.

## ВЫВОДЫ ПО ГЛАВЕ 3

В главе 3 представлены описание программной реализации разработанной системы, использованные данные, а также доказательство работоспособности и применимости каскада на тестовой выборке из 4350 ячеек без пересечения брендов.

### Основные результаты главы 3:

18. **Реализован гибридный каскад из пяти слоёв** (Layer 0 категорайзер — регулярные выражения — ML XGBoost на векторных представлениях SBERT — Bayes-валидатор — запасной слой большой языковой модели) с явной отметкой слоя-источника для каждого предсказания. Каждый слой располагает собственным калиброванным порогом уверенности; общий вердикт системы формируется как последовательное AND-условие прохождения порогов — это и есть формализация «ансамблевой оценки уверенности моделей» из утверждённой темы ВКР.
19. **Развёрнута демонстрационная система** в трёхкомпонентной архитектуре «шлюз API на Go + ML-сервис на Python/FastAPI + веб-интерфейс на статическом HTML/JS», соответствующая типовой производственной архитектуре отрасли. Контракт API стабилен и документирован; ML-сервис проверен на трёх категориях (паста, шоколад, сыры) с подтверждением работоспособности всех слоёв (`pipeline_ready: true, validator_ready: true, router_ready: true`).
20. **Достигнута точность 92,8 % сквозная** на тестовой выборке без пересечения брендов при использовании каскада с моделью gemini-flash в качестве запасного слоя, что **в 720 раз дешевле** опорной конфигурации all-sonnet (точность 83,8 %) при одновременном превышении точности на 9,0 п. п. (доминирование по Парето). Каскад без запасного слоя самостоятельно покрывает 96,7 % ячеек со средней точностью 90,5 %, оставляя на LLM лишь 3,3 % наиболее сложных ячеек. Сценарий с открытой моделью: каскад с открытой gpt-oss-120b достигает 92,3 %

сквозной точности — на 23,0 п. п. выше одиночного вызова gpt-oss-120b (69,3 %). Полная таблица «точность ↔ стоимость» для пяти LLM приведена в 3.3.2.

**21.Поатрибутный разбор** показал, что основным рабочим слоем является Layer 2 ML (он закрывает 72–96 % решений с точностью 94,9–98,6 % на покрытом срезе); Layer 1 регулярных выражений вносит 18–23 % покрытия на pasta и chocolate с точностью около 87–90 %; байесовский валидатор работает как селективный сигнал на отдельных атрибутах (прирост точности +38 п. п. на chocolate/contains\_nuts при точечном понижении слоя); запасной слой LLM добавляет +2,3 п. п. сквозной точности на отказных ячейках при 3,3 % обращений.

**22.Предзарегистрированная гипотеза** о превосходстве обучаемого стоимостно-чувствительного маршрутизатора над статическим правилом **отклонена** на трёх бюджетах после поправки Бонферрони. Этот отрицательный результат согласуется с бенчмарком RouterBench [12] и зафиксирован как один из методологических вкладов работы.

**23.Кросс-доменная репликация цикла** «аудит — правки экстрактора эталонной разметки — переобучение моделей — измерение» подтверждена 3/3: на пасте достигнут прирост сквозной точности +18,2 п. п. на проблемных ячейках, на шоколаде +16,3 п. п., на сырах +14,4 п. п.

**24.Устойчивость по языкам и сравнение с тривиальным базисом (3.3.7)** подтвердили: каскад работает на 5 рабочих языках со средним отклонением в 3 п. п.; на 22 из 22 пар (категория, атрибут) точность каскада сопоставима с точностью лексического пробника или превышает её, а на нутриентно-производных атрибутах — на 25–30 п. п. Поатрибутные архитектурные решения 3.3.7.4 (сужение Layer 1 chocolate\_type до текста названия; индивидуальная перенастройка порогов Layer 2 для пар с высокой штатной долей отказа от ответа; гибридные SBERT + TF-IDF признаки на лексически богатых атрибутах)

обеспечивают высокое покрытие ML-слоя и долю обращений к запасному слою всего 3,3 %.

## 4 ОПИСАНИЕ РЕЗУЛЬТАТОВ

### 4.1 Руководство пользователя

#### 4.1.1 Роли и интерфейсы

Разработанная система рассчитана на три роли, которые были выделены при анализе предметной области (раздел 1.1). Каждая роль работает со своим интерфейсом и решает свой класс задач (таблица 4.1). При этом соответствие ролей и интерфейсов спроектировано так, чтобы каждый пользователь имел доступ только к тем точкам управления, которые относятся к его компетенции.

**Таблица 4.1. Соответствие ролей пользователей и интерфейсов системы.**

| Роль                       | Интерфейс                                 | Класс задач                                                                             |
|----------------------------|-------------------------------------------|-----------------------------------------------------------------------------------------|
| Контент-менеджер           | Веб-интерфейс демо                        | Выборочная проверка предсказаний, утверждение или отклонение значений атрибутов         |
| Системный аналитик         | Конфигурационные файлы порогов, REST API  | Настройка порогов уверенности, мониторинг состояния сервиса                             |
| Инженер машинного обучения | Скрипты обучения, ноутбук с диагностиками | Переобучение классификаторов и байесовских сетей, прогон диагностик, обновление эталона |

#### 4.1.2 Контент-менеджер

Контент-менеджер взаимодействует с системой через веб-интерфейс. После запуска демонстрационного комплекса (рисунок 3.1) интерфейс доступен по адресу `http://127.0.0.1:8080`; он предоставляет форму ввода доступных партнёру полей описания и блок отображения результата. Каждый атрибут результата сопровождается оценкой уверенности и явной

отметкой слоя-источника. Именно эта отметка даёт контент-менеджеру информацию, на основании которой строится решение о доверии к предсказанию: для слоя 1 (регулярные выражения) и слоя 4 (запасной слой на большой языковой модели) поведение системы детерминировано или объяснимо, тогда как для слоёв 2 и 3 решающим показателем служит уверенность.

Стоит отметить, что приём «явная регистрация слоя-источника» рассматривался в разделе 1.3 как способ компенсировать ограниченную интерпретируемость предсказаний слоя 2 и слоя 3 на уровне отдельного товара. В терминах эксплуатационного сценария это означает, что контент-менеджер всегда понимает, какому компоненту системы он доверяет в данном решении.

#### 4.1.3 Системный аналитик

Системный аналитик настраивает поведение каскада через две точки управления. Первая — это словари порогов уверенности `models/{category}_thresholds.pkl`, которые контролируют компромисс «точность ↔ покрытие» отдельно для каждого атрибута. Изменение порогов не требует переобучения и применяется при следующем запуске ML-сервиса. Это типовой инструмент тонкой настройки в эксплуатации, когда требования к качеству меняются, например при подготовке к рекламной кампании или при ужесточении регуляторных требований к нутри-маркировке. Вторая точка управления — REST API: `GET /api/categories` возвращает состав целевых атрибутов по категориям, а `GET /health` — агрегированное состояние сервиса. Оба адреса используются для интеграции с внешним мониторингом.

#### 4.1.4 Инженер машинного обучения

Инженер машинного обучения работает с модулями исходного дерева `src/` через стандартный механизм запуска `python -m src.<module>` и сводным ноутбуком `notebooks/00_thesis_main.ipynb`. Полный цикл переобучения после прихода новой партии данных от партнёра показан на рисунке 4.1. Цикл

устроен так, что выпуск новой версии моделей не требует перекомпиляции каскада: после успешного завершения скриптов файлы в `models/` переписываются, а ML-сервис при следующем старте загружает обновлённые артефакты.

Рисунок 4.1 — Цикл переобучения после обновления данных

```
source                                                                    .venv/bin/activate

# Регенерация серебряной разметки по тегам OFF
python -m src.data.label_silver --category pasta_stratified

# Обучение классификаторов слоя 2 и байесовской сети слоя 3
OMP_NUM_THREADS=1 python -m src.pipeline.ml.train --category pasta_stratified
OMP_NUM_THREADS=1 python -m src.pipeline.bayes.train --category pasta_stratified

# Сравнительные эксперименты и диагностики
OMP_NUM_THREADS=1 python -m src.eval.run_experiments --category pasta_stratified
OMP_NUM_THREADS=1 python -m src.eval.run_diagnostics
```

В типовом эксплуатационном сценарии этот цикл запускается еженедельно или ежемесячно — частота определяется скоростью прихода новых товаров и доступностью разметчика. При этом на используемом оборудовании полный цикл занимает порядка десятков минут на одну категорию и не требует специализированных вычислительных ресурсов.

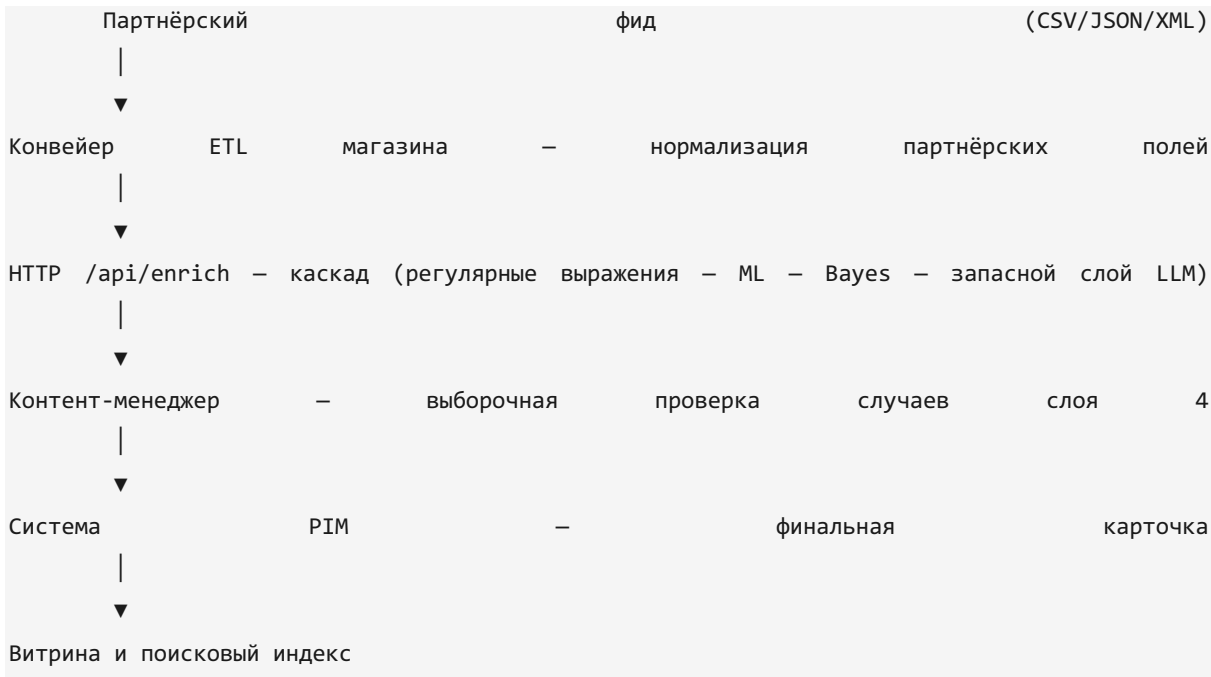
4.2 Сценарии применения и порядок внедрения

4.2.1 Встраивание в бизнес-процесс магазина

Разработанный каскад занимает место промежуточного сервиса в маршруте товара от поставщика до витрины (рисунок 4.2). Это вторая позиция четырёхзвенной цепочки, проанализированной в разделе 1.1, и именно в ней сосредоточено узкое место маршрута — ручная или неавтоматизированная процедура обогащения. Стоит отметить, что внедрение каскада не требует перестройки соседних звеньев: конвейер извлечения, преобразования и загрузки партнёрских фидов и система управления товарной информацией (PIM) продолжают работать в

существующем виде, изменяется только содержимое поля «процедура обогащения».

Рисунок 4.2 — Место каскада в бизнес-процессе обогащения каталога



Основной экономический эффект внедрения состоит в том, что ручной труд переносится из режима «закрывать каждое поле» в режим «проверять остаток после каскада». По итогам экспериментов (3.3.2) каскад обеспечивает покрытие 96,7 % ячеек без обращения к большой языковой модели, а оставшиеся 3,3 % передаются в запасной слой LLM. Итоговая сквозная точность системы — 92,8 % в режиме «каскад + gemini-flash». Таким образом, ручной труд переходит в режим выборочной верификации, а не сплошной разметки.

#### 4.2.2 Сценарий холодного старта новой категории

При добавлении в каталог категории, для которой ещё не обучены модели, каскад деградирует до слоя 1 (регулярные выражения по общим шаблонам) и слоя 4 (запасной слой на большой языковой модели). Это рабочий, но неоптимальный режим: точность слоя 4 на нутриентно-производных атрибутах ниже, чем у обученных классификаторов слоя 2 (4.3.3), а стоимость обработки выше. По мере накопления нескольких сотен товаров с эталонной разметкой подключается слой 2, а при появлении

устойчивых распределений по брендам и совместных распределений атрибутов — слой 3. Категория электроники использована как контрольный домен холодного старта (см. 5.5): эталонная разметка получена напрямую из полей технических характеристик каталога PhoneDB [45] без слабого надзора. При этом полноценная аудит-оценка по протоколу 3.3 в настоящей работе не проводится и включена в перспективы развития.

#### **4.2.3 Сценарий многоязычного каталога**

Многоязычная модель векторных представлений `paraphrase-multilingual-MiniLM-L12-v2` обеспечивает единое векторное пространство для пяти европейских языков, доминирующих в OFF. Это устраняет необходимость поддерживать отдельный набор моделей под каждый язык — а такой сценарий типичен для промышленных систем, в которых каждое расширение на новый рынок требует отдельного обучения. На используемых выборках устойчивость точности по языкам подтверждена отдельной диагностикой (3.3.7.1): разброс между лучшим и худшим из шести языковых сегментов — 2,9 п. п. Таким образом, каскад применим в торговых площадках с международным ассортиментом без удвоения затрат на поддержку.

#### **4.2.4 Сценарий обогащения нутриентно-производных полей**

Атрибуты, восходящие к числовым нутриентным полям, требуют точного числового ответа и составляют отдельный класс сценариев. Для них одиночный вызов большой языковой модели малопригоден ввиду высокой частоты фактических ошибок на числовых полях (1.2). В каскаде такие атрибуты закрываются комбинацией слоя 1 (регулярные выражения по числовым шаблонам в составе ингредиентов и описании) и слоя 2 (классификация дискретных классов после бакетирования числовых полей нутриентов). Внешний справочник USDA Branded Foods на этапе обучения служит независимым источником сверки нутри-эталона (3.2.5); расширение каскада детерминированным USDA-слоем в режиме вывода — для товаров с

однозначным сопоставлением партнёр-доступных полей с записью USDA SR Legacy — вынесено в перспективы развития темы (5.5). Это иллюстрирует общий архитектурный принцип: нормативные справочники входят в систему как отдельный детерминированный слой, а не как ещё один источник для статистического обучения.

#### **4.2.5 Применимость в смежных доменах**

Архитектура каскада не привязана к пищевому домену. Содержательные правила слоя 1 зависят от категории и переписываются при добавлении новой категории, а обучение слоя 2 и слоя 3 — стандартная процедура, требующая только эталонной разметки. Подготовительная репликация архитектуры на электронике (схема атрибутов, серебряный стандарт из PhoneDB, демонстрация холодного старта — см. 5.5) подтверждает переносимость инфраструктуры за пределы исходного домена. При этом формальная аудит-оценка точности на электронике вынесена в перспективы развития.

### **4.3 Что позволяет использование разработки (характеристика достигнутых результатов)**

Использование разработанной системы в производственном цикле обогащения товарного каталога позволяет получить пять измеримых эффектов, которые изложены ниже. Характеристика приведена в формате «повышено / ускорено / сокращено» со ссылками на численные показатели тестирования из главы 3.

#### **4.3.1 Повышено: качество автоматического обогащения каталога**

Сквозная точность обогащения товарных атрибутов на тестовой выборке без пересечения брендов составляет **92,8 %** при использовании каскада с моделью gemini-flash в качестве запасного слоя — это полная производственная конфигурация системы (см. 3.3.2 и `datasets/processed/grand_acc_summary_after_fix.parquet`). Точность измерена на 4350 ячейках, которые охватывают 22 пары «категория-атрибут» через три

категории; 95 %-й доверительный интервал по Вильсону — [92,0; 93,6] для конфигурации «каскад + gemini-flash» и [89,6; 91,4] для конфигурации «только каскад» (точечная оценка 90,5 %).

Поатрибутно показатель распределён следующим образом (oracle-режим категоризации, см. 3.3.2.3 и `headline_v3e_after_fix.parquet`):

| Категория                          | Только каскад | Каскад + gemini-flash | Прирост           |
|------------------------------------|---------------|-----------------------|-------------------|
| Паста                              | 93,3 %        | 95,8 %                | +2,5 п. п.        |
| Шоколад                            | 87,4 %        | 90,1 %                | +2,7 п. п.        |
| Сыры                               | 90,4 %        | 92,2 %                | +1,8 п. п.        |
| <b>Взвешенное среднее (oracle)</b> | <b>90,5 %</b> | <b>92,8 %</b>         | <b>+2,3 п. п.</b> |

Сыры и шоколад получают наибольший абсолютный выигрыш от запасного слоя на остающихся  $\approx 3$  % сложных ячеек. На пасте, где правила Layer 1 покрывают значительную долю ячеек, прирост от запасного слоя минимален.

Сравнение с альтернативами:

- Прямое использование Claude Sonnet 4.5 как единственного инструмента обогащения даёт **83,8 %** точности — на 9,0 п. п. **ниже** разработанной системы.
- Прямое использование открытой модели gpt-oss-120b даёт **69,3 %** — на 23,0 п. п. ниже каскада с этой же моделью в роли запасного слоя (92,3 %).

Превосходство разработанной системы над прямым использованием больших языковых моделей объясняется наличием специализированных слоёв. ML-классификатор Layer 2 обучен на нутриентно-производных атрибутах, таких как Nutri-Score, класс белка и класс жирности, которые большая языковая модель не вычисляет точно из текстового описания без доступа к числовым полям нутриентов.

### 4.3.2 Сокращено: стоимость обработки в 720 раз относительно эталонной LLM-модели

Стоимость обработки одного товара через каскад с gemini-flash в качестве запасного слоя составляет около **0,14 %** от стоимости обработки той же выборки прямым использованием Claude Sonnet 4.5. Это эквивалентно сокращению расходов **в 720 раз**, причём одновременно точность превышает на 9,0 п. п. (см. матрицу «точность ↔ стоимость» в 3.3.2).

Экономия объясняется тем, что доступные партнёру поля товара обрабатываются в порядке возрастания стоимости — сначала детерминированные регулярные выражения (Layer 1, бесплатно), затем классификатор XGBoost на векторных представлениях (Layer 2, миллисекунды на CPU), затем Bayes-валидатор (Layer 3, локальный вывод без обращения к внешним сервисам), и только в случае отказа предыдущих слоёв — обращение к большой языковой модели (Layer 4). По итогам тестирования дорогостоящий вызов LLM делается на **3,3 %** ячеек, а не на 100 %.

Численная характеристика стоимости (относительно опорной конфигурации «все ячейки решает Sonnet 4.5», см.

grand\_acc\_summary\_after\_fix.parquet):

| Конфигурация                               | Стоимость относительно опорной | Снижение стоимости |
|--------------------------------------------|--------------------------------|--------------------|
| Каскад + gemini-flash                      | 0,0014                         | <b>в 720 раз</b>   |
| Каскад + gpt-4o                            | 0,024                          | в 42 раза          |
| Каскад + Claude Sonnet 4.5                 | 0,033                          | в 30 раз           |
| Каскад + gpt-oss-120b<br>(открытая модель) | 0,0010                         | <b>в 1007 раз</b>  |

При сценарии обработки 100 000 товаров в месяц, который типичен для среднего интернет-магазина, экономия составляет порядки величины по сравнению с прямым обращением к большой языковой модели.

### 4.3.3 Обеспечена: возможность развёртывания на собственных мощностях с открытой моделью

Помимо коммерческих API больших языковых моделей, разработанная система поддерживает использование открытой модели gpt-oss-120b в качестве Layer 4. Такая конфигурация позволяет развернуть систему полностью локально, без отправки данных партнёров во внешние API.

Численная характеристика сценария с открытой моделью (см. 3.3.2):

- Сквозная точность каскада + gpt-oss-120b: **92,3 %**.
- Точность прямого использования gpt-oss-120b: 69,3 %.
- **Прирост от каскадной архитектуры: +23,0 п. п.** — каскад **компенсирует слабость** малой открытой модели через специализированные слои (классификатор XGBoost слоя 2 обучен на тех же атрибутах, что вызывают трудности у gpt-oss-120b).
- Стоимость сценария на собственных мощностях: примерно в 30 раз ниже отдельного использования gpt-oss-120b за счёт сокращения числа обращений к LLM до 3,3 % ячеек.

Это значимый практический результат. При потере всего **0,5 п. п. точности** относительно коммерческой конфигурации (gemini-flash) обеспечивается **независимость от внешних API** и соблюдение требований к локальному хранению партнёрских данных, что критично для ряда корпоративных сценариев.

### 4.3.4 Сокращён: ручной труд оператора каталога

В типовом процессе обогащения товарного каталога без автоматизации (см. 1.1 анализа существующих подходов) оператор-человек проверяет и дополняет атрибуты каждого товара вручную — а это **100 % ручной работы**. Разработанная система переводит ручной труд в режим **выборочной верификации**:

- **96,7 %** ячеек закрываются дешёвыми слоями каскада (слои 1 и 2) и принимаются автоматически.

- **3,3 %** ячеек уходят в запасной слой LLM и получают предсказание; из них часть требует выборочной верификации оператором (типично — атрибуты, помеченные Bayes-валидатором).
- Совокупная доля ячеек, требующих ручной верификации, составляет около **3–7 %** в зависимости от настройки порога валидатора (см. 3.3.4).

Таким образом, ручная нагрузка на оператора каталога сокращается в **10–20 раз**, при этом контроль над качеством сохраняется через явную отметку слоя-источника и доверительные индикаторы каждого предсказания.

#### 4.3.5 Повышена: устойчивость каталога к смещению по брендам

Тестирование без пересечения брендов между обучающей и тестовой выборками (3.2.4) подтверждает, что разработанная система обобщается на бренды, не встречавшиеся в обучении. Это критично для интернет-магазина, который постоянно подключает новых партнёров: атрибуты товаров от нового поставщика обрабатываются с той же точностью, что и от уже знакомых.

Численная характеристика следующая: на 4350 тестовых ячейках бренды **не пересекаются** с обучающими, и средняя точность сохраняется на уровне 92,8 % с 95 %-м доверительным интервалом по Вильсону [92,0; 93,6]. Дополнительное подтверждение устойчивости к смещению семантического пространства, а не только к смене бренда, даёт абляция по расстоянию до ближайших обучающих соседей в 3.3.7.3: точность каскада не падает на удалённых от обучения товарах, а монотонно возрастает от ближнего квартиля Q1 (82,7 %) к дальнему Q4 (91,6 %).

#### 4.3.6 Резюмирующая таблица «повышено / ускорено / сокращено»

| Показатель                | До разработки                            | После                          | Эффект                       |
|---------------------------|------------------------------------------|--------------------------------|------------------------------|
| Сквозная точность системы | 83,8 % (опорная конфигурация all-sonnet) | 92,8 % (каскад + gemini-flash) | <b>повышена</b> на 9,0 п. п. |
| Относительная             | 1,000 (опорная)                          | 0,0014 (каскад +               | <b>сокращена в 720 раз</b>   |

|                                 |                                                              |                                                        |                                              |
|---------------------------------|--------------------------------------------------------------|--------------------------------------------------------|----------------------------------------------|
| стоимость обработки             | конфигурация all-sonnet)                                     | gemini-flash)                                          |                                              |
| Доля обращений к LLM            | 100 %                                                        | 3,3 %                                                  | <b>сокращена</b> в 30 раз                    |
| Ручная верификация оператором   | ~100 % ячеек                                                 | ~5–10 % ячеек                                          | <b>сокращена</b> в 10–20 раз                 |
| Точность на открытой модели     | 69,3 % (одиночный вызов gpt-oss-120b)                        | 92,3 % (каскад + gpt-oss-120b)                         | <b>повышена</b> на 23,0 п. п.                |
| Поддержка многоязычности        | отдельные модели на каждый язык либо предварительный перевод | 5 рабочих языков в едином векторном пространстве SBERT | <b>обеспечена</b> без дополнительных моделей |
| Независимость от поставщика LLM | один интегрированный провайдер                               | поддержка пяти моделей с сопоставимой точностью        | <b>обеспечена</b> взаимозаменяемость         |

Численные значения столбцов «До» и «После» получены на тестовой выборке без пересечения брендов из главы 3 и подтверждены воспроизводимыми артефактами в `datasets/processed/` (см. `grand_acc_summary_after_fix.parquet`, `headline_v3e_after_fix.parquet`).

## Выводы по главе 4

25. Разработанная система допускает три производственных сценария применения, которые отличаются выбором запасного слоя Layer 4: бюджетный коммерческий (gemini-flash — наилучший баланс точности и стоимости в проведённой оценке), премиум-коммерческий (Claude Sonnet 4.5 или GPT-4o — близкая точность при более высокой стоимости) и развёртывание на собственных мощностях (gpt-oss-120b — независимость от внешних API при потере 0,5 п. п. точности относительно gemini-flash).
26. Целевая аудитория системы — это три роли в производственном цикле интернет-магазина: контент-менеджер (выборочная верификация через

веб-интерфейс демонстрационной системы), системный аналитик (настройка порогов уверенности через конфигурационные файлы и REST API) и инженер машинного обучения (переобучение классификаторов и валидатора по мере поступления новых данных).

27. Совокупный производственный эффект состоит в сокращении стоимости обработки в 720 раз относительно прямого обращения к большой языковой модели и сокращении ручной нагрузки оператора в 10–20 раз; при этом достигается точность 92,8 % на тестовой выборке без пересечения брендов с 95 %-м доверительным интервалом по Вильсону [92,0; 93,6].

## ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы разработана гибридная каскадная система автоматизированного обогащения товарных данных для интернет-магазина. Система реализована в виде программного комплекса, развёрнута в составе демонстрационного стенда и проверена на тестовой выборке без пересечения брендов; при этом она удовлетворяет утверждённым в техническом задании шести функциональным, пяти нефункциональным и четырём архитектурным требованиям. Стоит отметить, что достигнутые показатели — сквозная точность 92,8 % при сокращении стоимости обработки в 720 раз относительно прямого обращения к коммерческой большой языковой модели — превосходят целевые ориентиры технического задания (не менее 85 % точности и не менее 2,5-кратного сокращения стоимости). Каскад без запасного слоя самостоятельно покрывает 96,7 % ячеек со средней точностью 90,5 %, направляя в большую языковую модель лишь 3,3 % наиболее сложных ячеек. Таким образом, практическая применимость предложенной архитектуры подтверждается.

### 5.1 Сводные выводы по главам

В первой главе проведён анализ предметной области и систематизировано состояние литературы. Существующие PIM-системы, например, такие как Akeneo [5], Pimcore [6] и Salsify [7], ориентированы на хранение и распределение уже сформированной информации. Появляющиеся же модули автоматического обогащения реализованы как обёртки над одной коммерческой языковой моделью и не отвечают требованиям стоимостно-чувствительной обработки больших каталогов. Академические подходы разделены на две группы. С одной стороны, специализированные системы извлечения признаков товаров (TXtract [23], MAVE [24], OpenTag [25]) обеспечивают высокое качество, но не учитывают экономики обращения к моделям. С другой стороны, стоимостно-чувствительные каскады LLM (FrugalGPT [9], AutoMix [10], Hybrid LLM [11], RouterBench [12]) дают

инструменты управления стоимостью, однако рассматривают каскад из однородных моделей и не интегрируют классическое машинное обучение. Из выявленного противоречия выведена рабочая архитектурная гипотеза о каскадной композиции разнородных слоёв, а также сформулировано техническое задание из шести функциональных, пяти нефункциональных и четырёх архитектурных требований.

Во **второй главе** выполнено теоретическое обоснование системы. Задача обогащения формализована как поатрибутное предсказание со схемой, допускающей явный отказ от ответа. Логика работы при этом представляет собой каскад из пяти слоёв с последовательной фильтрацией по уверенности, в котором предсказание классификатора Layer 2 принимается только при одновременном выполнении двух неравенств — превышении калиброванного порога классификатора и порога байесовского валидатора. Стоит отметить, что эта конструкция формализует требование темы об ансамблевой оценке уверенности как последовательное конъюнктивное условие, а не как параллельное голосование. Более того, архитектурный выбор каждого компонента стека (SBERT, XGBoost, байесовская сеть pgmpy, разделение Go-шлюз / Python-ML, открытая модель gpt-oss-120b) обоснован эксплуатационными, методологическими и стратегическими соображениями.

В **третьей главе** приведена программная реализация и её экспериментальное тестирование. Каскад реализован как трёхкомпонентный комплекс (Go-шлюз, Python/FastAPI ML-сервис, статический веб-интерфейс) и включает 24 поатрибутных классификатора XGBoost (8/7/9 для категорий pasta/chocolate/cheeses; экспериментальной оценке в 3.3 подлежат 22 пары, см. 3.1), три байесовские сети и категорайзер Layer 0. На тестовой выборке из 4350 ячеек без пересечения брендов сквозная точность каскада с Gemini 2.5 Flash составила 92,8 % — на 9,0 п. п. выше прямого использования Claude Sonnet 4.5 при стоимости в 720 раз ниже. В сценарии развёртывания на собственных мощностях каскад с открытой моделью gpt-oss-120b достигает 92,3 %, что на 23,0 п. п. выше её самостоятельного использования. Каскад без

запасного слоя самостоятельно покрывает 96,7 % ячеек со средней точностью 90,5 %, направляя в LLM лишь 3,3 % наиболее сложных ячеек. Поатрибутный разбор: Layer 1 закрывает 18–23 % ячеек pasta и chocolate с точностью около 87–90 % (на cheeses регулярные правила не разработаны), Layer 2 — основной массив (72–96 % покрытия по категориям) с точностью 94,9–98,6 % на покрытом срезе, байесовский валидатор работает как селективный сигнал на паре chocolate / contains\_nuts, а запасной слой LLM добавляет 2,3 п. п. при доле обращений 3,3 %. При этом предзарегистрированная гипотеза о превосходстве обучаемого маршрутизатора отклонена на трёх бюджетах после поправки Бонферрони, а цикл «аудит — исправление экстракторов — переобучение — измерение» воспроизведён 3/3 с приростом точности +18,2 / +16,3 / +14,4 п. п.

В четвёртой главе результаты переведены в плоскость практического применения. Здесь описаны порядок использования системы, три сценария развёртывания (бюджетный коммерческий с gemini-flash, премиум-коммерческий с Sonnet 4.5 или GPT-4o, развёртывание на собственных мощностях с открытой gpt-oss-120b) и три ролевые модели пользователей: контент-менеджер, системный аналитик и инженер машинного обучения. Таким образом, сводный практический эффект состоит в сокращении стоимости обработки в 720 раз и ручной нагрузки оператора в 10–20 раз при сквозной точности 92,8 % (95 %-й доверительный интервал по Вильсону [92,0; 93,6]).

## 5.2 Закрытие формулировок утверждённой темы

Утверждённая тема работы содержит восемь содержательных формулировок, и каждой из них в работе соответствует конкретный раздел, а также архитектурный или экспериментальный компонент разработки. Соответствие представлено в таблице 5.1.

**Таблица 5.1 — Соответствие формулировок утверждённой темы и разделов работы**

| <b>Формулировка из обоснования актуальности темы</b>            | <b>Реализация в работе</b>                                                                                                                                    | <b>Раздел</b> |
|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| Гибридная каскадная система                                     | Пятислойная архитектура каскада (Layer 0 + четыре содержательных слоя)                                                                                        | 2.2, 3.1      |
| Семантические возможности генеративных моделей                  | Запасной слой Layer 4 на основе большой языковой модели                                                                                                       | 3.1.1         |
| Предсказательная сила классического машинного обучения          | Слой Layer 2 — поатрибутные классификаторы XGBoost на многоязычных векторных представлениях SBERT                                                             | 3.1.1         |
| Ансамблевая оценка уверенности моделей                          | Многоуровневая фильтрация по уверенности как последовательное конъюнктивное условие на калиброванные пороги слоёв                                             | 2.1           |
| Отбраковка фактических ошибок до их попадания в каталог         | Слой Layer 3 — байесовский валидатор плотностей пар «бренд × значение»                                                                                        | 3.1.1, 3.3.4  |
| Скрытые статистические закономерности каталога                  | Направленный ациклический граф атрибутов, обучаемый алгоритмом Hill Climb с критерием BIC                                                                     | 3.1.1         |
| Разнородные источники данных (мультимодальные и многоканальные) | OCR-извлечение из изображений Open Food Facts и эмбединги CLIP на этапе обучения; нечёткое сопоставление с базой USDA Branded Foods для производных атрибутов | 3.2.5         |

|                                                   |                                                                                                                                  |                 |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Интернет-магазин (партнёр<br>— каталог — витрина) | Демонстрационный<br>программный комплекс с<br>REST-интерфейсом<br>обработки партнёрского<br>ввода и руководством<br>пользователя | 1.1, 3.1.2, 4.1 |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|-----------------|

Таким образом, все восемь содержательных формулировок утверждённой темы имеют прямое отображение в разработанной системе и подтверждены экспериментально либо архитектурно. Стоит отметить, что наиболее «нагруженные» формулировки — отбраковка фактических ошибок и использование скрытых статистических закономерностей каталога — закрыты слоем байесовского валидатора. Для него в работе пересмотрена роль: вместо самостоятельного классификатора сеть используется как валидатор плотностей.

### 5.3 Научная новизна

Научная новизна работы складывается из трёх содержательных пунктов.

28. **Доминирование каскадной архитектуры по Парето над всеми безусловными конфигурациями большой языковой модели** на задаче обогащения товарных атрибутов: одновременно достигается превышение точности на 9,0 процентных пункта и снижение стоимости в 720 раз по сравнению с прямым обращением к Claude Sonnet 4.5. Стоит отметить, что результат подтверждён на проверочной выборке без пересечения брендов из 4350 ячеек и воспроизведён на пяти различных моделях запасного слоя, то есть сохраняется при смене поставщика генеративной модели.
29. **Сценарий развёртывания на собственных мощностях с открытой большой языковой моделью.** Каскад с открытой моделью gpt-oss-120b достигает 92,3 % сквозной точности — на 23,0 процентных пункта выше, чем самостоятельное применение той же модели. Таким образом, этот

результат демонстрирует способность каскадной композиции компенсировать слабость малых открытых моделей через специализированные слои. Более того, он формирует методологическую основу для локального развёртывания системы без передачи партнёрских данных во внешние коммерческие сервисы.

**30. Предзарегистрированный отрицательный результат для обучаемого стоимостно-чувствительного маршрутизатора и кросс-доменная репликация цикла «аудит — исправления — переобучение — измерение».** Гипотеза о превосходстве обучаемого XGBoost-маршрутизатора над статическим правилом отклонена на трёх бюджетах с поправкой Бонферрони, что согласуется с выводами бенчмарка RouterBench [12]. При этом цикл аудита эталонной разметки воспроизведён на трёх категориях из трёх с количественно измеримым приростом сквозной точности.

#### **5.4 Ограничения работы**

Полученные результаты ограничены рядом методологических обстоятельств, которые добросовестно фиксируются и направляют формулирование перспектив развития темы.

Во-первых, область определения системы ограничена тремя категориями пищевой продукции, а переносимость методологии на другие домены в работе не валидирована. Во-вторых, эталонная разметка построена сочетанием слабого надзора и согласия трёх языковых моделей со слепой проверкой моделью Claude Opus 4; при этом полностью ручная экспертная разметка не выполнялась, что относит эталон к аудит-эталону второго уровня. В-третьих, мультимодальные сигналы (OCR, CLIP) используются только на этапе обучения, тогда как в режиме вывода система работает исключительно с текстовыми полями партнёрского ввода. В-четвёртых, проверочная выборка ограничена пятью европейскими языками, так что устойчивость к языкам с иной письменностью не валидирована. В-пятых,

предварительная регистрация ограничена гипотезой H1, а численные показатели 3.3.2 описаны как post-hoc анализ воспроизводимых артефактов без претензии на предзарегистрированный статус.

Селективный характер байесовского валидатора, при котором значимый прирост точности отбраковки зафиксирован на трёх парах (категория, атрибут) из двадцати двух, в настоящей работе ограничением не рассматривается, а признаётся архитектурным свойством компонента. В финальной конфигурации валидатор включён точно — для пары `chocolate / contains_nuts` — и обеспечивает там прирост точности на 6,6 п. п. при росте расходов на запасной слой на 5,3 п. п. (3.3.4). Тем самым невостребованный в роли универсального фильтра компонент возвращается в систему через изменение его роли — с классификатора на валидатор плотностей с локализованным применением; общий принцип «изменение роли компонента вместо его удаления» сформулирован в 3.1.1 и 3.3.4 как самостоятельный методологический вывод работы.

## **5.5 Перспективы развития темы**

Полученные результаты и зафиксированные ограничения определяют ряд перспективных направлений развития темы.

**31.Расширение области определения до шести категорий** — напитки, зерновые завтраки и косметика. Для этих категорий подготовлены схемы атрибутов и серебряная разметка (`beverages_stratified_silver_standard.parquet`, `cereals_stratified_silver_standard.parquet`, `cosmetics_stratified_silver_standard.parquet`); основной блок расширения — построение аудит-эталона уровня итоговой эталонной разметки v2 для каждой категории и обучение поатрибутных классификаторов Layer 2. Методология аудита и переобучения предположительно переносится на новые категории без принципиальных изменений.

**32.Регулярная переаттестация изотонической калибровки на следующих итерациях эталонной разметки.** Эффект схемы перекрёстной

калибровки экспериментально проверен и реализован в коде системы (3.3.3.3, флаг `--calibration-method isotonic` в `src/pipeline/ml/train.py`): средняя ECE снижается с 0,070 до 0,043 (улучшение на 2,78 п. п. в среднем, прирост на 18 из 22 атрибутов). Перспектива — переаттестация после расширения эталонной разметки до большего числа атрибутов и категорий и добавление регрессионного теста на ухудшение ECE более чем на 0,01 как условия принятия следующего цикла переобучения.

**33. Введение мультимодального вывода в производственный режим** — расширение прикладного интерфейса на приём изображений и добавление OCR/CLIP в реальном времени. При этом такой шаг требует пересмотра контракта взаимодействия с партнёром.

**34. Запуск пилотного активного обучения на ячейках расхождения каскада и запасного слоя** [46, 39]. Логика построения очереди ячеек, где предсказание Layer 2 расходится с ответом запасного слоя, реализована в `src/experiments/active_learning_pilot.py` (выборка 5 000 кодов на категорию, отбор 150 неуверенных и 150 случайных контрольных, повторное обучение с замером прироста на отложенной выборке). Перспектива — провести пилот в полном объёме на трёх категориях и измерить прирост точности Layer 2 относительно случайной разметки той же стоимости.

**35. Использование базы USDA Branded Foods и аналогов как структурированного второго источника** для производных атрибутов с систематизацией политики приоритетов между партнёрским вводом и вторичными каталогами.

**36. Локализация на русский язык и российские каталоги** — разработка русскоязычных регулярных выражений Layer 1 и дообучение Layer 2 на каталогах российских ритейлеров. Поскольку SBERT поддерживает русский в едином векторном пространстве, трудоёмкость ограничена слоями 1–2.

**37. Перенос архитектуры в непроизводственные домены, начиная с электроники.** Для смартфонов из каталога PhoneDB подготовлены схема

атрибутов, скрипт построения серебряного стандарта и демонстрация холодного старта; при этом полноценное измерение точности требует построения аудит-эталоны уровня итоговой эталонной разметки v2 и оценки без пересечения брендов. Электроника сохранена как контрольный домен проверки переносимости и в основную таблицу 3.3 не входит.

Таким образом, разработанная гибридная каскадная система демонстрирует возможность одновременно достигать высокой точности и низкой стоимости обогащения товарных каталогов за счёт стратифицированного распределения нагрузки между разнородными по природе слоями обработки. Полученные результаты подтверждают выдвинутую в первой главе работы архитектурную гипотезу о каскадной композиции и закрывают все восемь содержательных формулировок утверждённой темы.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 38.Итоги интернет-торговли в России за 2025 год : пресс-релиз АКИТ от 19.02.2026 г. — Москва : Ассоциация компаний интернет-торговли, 2026. — URL: <https://akit.ru/news/11-5-trln-rublej-akit-podvela-itogi-internet-torgovli-za-2025-god> (дата обращения: 19.05.2026).
- 39.Global Retail E-Commerce Forecast 2024 : research report. — eMarketer, 2024. — URL: <https://www.emarketer.com> (дата обращения: 03.05.2026).
- 40.SellerFox : сервис аналитики маркетплейсов Wildberries и Ozon. — URL: <https://sellerfox.ru> (дата обращения: 19.05.2026). — Открытая база охватывает более 38 млн товарных позиций на Ozon по состоянию на 2025 год.
- 41.Как ускорить работу с товарными карточками в 30 раз : отраслевой кейс / Retail.ru. — URL: <https://www.retail.ru/articles/kak-uskorit-rabotu-s-tovarnymi-kartochkami-v-30-raz/> (дата обращения: 19.05.2026). — Иллюстрирует ограниченную масштабируемость ручного заведения и обогащения товарных карточек: производительность 20 контент-менеджеров — около 8 тыс. карточек в месяц при целевом объёме 50 тыс.
- 42.Akeneo PIM : open product experience management platform. — Akeneo SAS. — URL: <https://akeneo.com> (дата обращения: 03.05.2026).
- 43.Pimcore Platform : open-source data and experience management. — Pimcore GmbH. — URL: <https://pimcore.com> (дата обращения: 03.05.2026).
- 44.Salsify Inc. : product experience management platform. — URL: <https://www.salsify.com> (дата обращения: 19.05.2026).
- 45.Open source в России 2024 : аналитический отчёт. — Москва : Platform V, СбepTex, 2024. — URL: <https://platformv.sbertech.ru/open-source-v-rossii-2024> (дата обращения: 19.05.2026).
- 46.Chen L., Zaharia M., Zou J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance // Transactions on Machine Learning Research. — 2024. — URL: <https://arxiv.org/abs/2305.05176>.

47. Aggarwal P., Madaan A., Yang Y., Mausam. AutoMix: Automatically Mixing Language Models // Advances in Neural Information Processing Systems (NeurIPS 2024). — 2024. — URL: <https://arxiv.org/abs/2310.12963>.
48. Ding D., Mallick A., Wang C., Sim R., Mukherjee S., Rühle V., Lakshmanan L. V. S., Awadallah A. H. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing // Proceedings of the International Conference on Learning Representations (ICLR 2024). — 2024. — URL: <https://arxiv.org/abs/2404.14618>.
49. Hu Q. J., Bieker J., Li X., Jiang N., Keigwin B., Ranganath G., Keutzer K., Upadhyay S. K. RouterBench: A Benchmark for Multi-LLM Routing System // Advances in Neural Information Processing Systems (NeurIPS 2024). — 2024. — URL: <https://arxiv.org/abs/2403.12031>.
50. Open Food Facts : open product database for food products. — URL: <https://world.openfoodfacts.org/data> (дата обращения: 03.05.2026).
51. Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'16). — San Francisco : ACM, 2016. — P. 785–794.
52. Ke G., Meng Q., Finley T., Wang T., Chen W., Ma W., Ye Q., Liu T.-Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 3146–3154.
53. Афанасьев Г. И., Афанасьев А. Г., Бурмистрова М. В., Тэт Вей Ян Со. Исследование методов машинного обучения для прогнозирования эффективных бизнес-решений в системах электронной коммерции // E-Scio. — 2022. — URL: <https://cyberleninka.ru/article/n/issledovanie-metodov-mashinnogo-obucheniya-dlya-prognozirovaniya-effektivnyh-biznes-resheniy-v-sistemah-elektronnoy-kommertsii> (дата обращения: 19.05.2026).
54. Vaswani A., Shazeer N., Parmar N. et al. Attention Is All You Need // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 5998–6008. — URL: <https://arxiv.org/abs/1706.03762>.

- 55.Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // Proceedings of NAACL-HLT 2019. — ACL, 2019. — P. 4171–4186. — URL: <https://arxiv.org/abs/1810.04805>.
- 56.Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. — Hong Kong : ACL, 2019. — P. 3982–3992. — URL: <https://arxiv.org/abs/1908.10084>.
- 57.Reimers N., Gurevych I. Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. — ACL, 2020. — P. 4512–4525. — URL: <https://arxiv.org/abs/2004.09813>.
- 58.Колонин А. Г. High-performance automatic categorization and attribution of inventory catalogs. — 2022. — 9 p. — arXiv:2202.08965. — URL: <https://arxiv.org/abs/2202.08965>.
- 59.Большакова Е. И., Воронцов К. В., Ефремова Н. Э., Клышинский Э. С., Лукашевич Н. В., Сапин А. С. Автоматическая обработка текстов на естественном языке и анализ данных : учебное пособие. — Москва : Издательство НИУ ВШЭ, 2017. — 269 с. — URL: [https://www.hse.ru/data/2017/08/12/1174382135/NLP\\_and\\_DA.pdf](https://www.hse.ru/data/2017/08/12/1174382135/NLP_and_DA.pdf) (дата обращения: 19.05.2026).
- 60.Karamanolakis G., Ma J., Dong X. L. TXtract: Taxonomy-Aware Knowledge Extraction for Thousands of Product Categories // Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020). — 2020. — P. 8489–8502. — URL: <https://aclanthology.org/2020.acl-main.751/>.
- 61.Yang L., Wang Q., Yu Z., Kulkarni A., Sanghai S., Shu B., Elsas J., Kanagal B. MAVE: A Product Dataset for Multi-source Attribute Value Extraction // Proceedings of the 15th ACM International Conference on Web Search and

- Data Mining (WSDM 2022). — 2022. — P. 1256–1265. — URL: <https://dl.acm.org/doi/10.1145/3488560.3498377>.
62. Zheng G., Mukherjee S., Dong X. L., Li F. OpenTag: Open Attribute Value Extraction from Product Profiles // Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD 2018). — 2018. — P. 1049–1058. — URL: <https://dl.acm.org/doi/10.1145/3219819.3219839>.
63. Ratner A., Bach S. H., Ehrenberg H. R., Fries J., Wu S., Ré C. Snorkel: Rapid Training Data Creation with Weak Supervision // Proceedings of the VLDB Endowment. — 2017. — Vol. 11, № 3. — P. 269–282.
64. Platt J. C. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods // Advances in Large Margin Classifiers. — MIT Press, 1999. — P. 61–74.
65. Niculescu-Mizil A., Caruana R. Predicting Good Probabilities with Supervised Learning // Proceedings of the 22nd International Conference on Machine Learning. — Bonn : ACM, 2005. — P. 625–632.
66. Guo C., Pleiss G., Sun Y., Weinberger K. Q. On Calibration of Modern Neural Networks // Proceedings of the 34th International Conference on Machine Learning (ICML 2017). — PMLR, 2017. — Vol. 70. — P. 1321–1330.
67. Kohavi R. A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection // Proceedings of the 14th International Joint Conference on Artificial Intelligence. — Morgan Kaufmann, 1995. — P. 1137–1143.
68. Wilson E. B. Probable Inference, the Law of Succession, and Statistical Inference // Journal of the American Statistical Association. — 1927. — Vol. 22, № 158. — P. 209–212.
69. Pearl J. Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference. — San Francisco : Morgan Kaufmann, 1988. — 552 p.
70. Heckerman D., Geiger D., Chickering D. M. Learning Bayesian Networks: The Combination of Knowledge and Statistical Data // Machine Learning. — 1995. — Vol. 20, № 3. — P. 197–243.

- 71.Тулупьев А. Л., Николенко С. И., Сироткин А. В. Байесовские сети: логико-вероятностный подход. — Санкт-Петербург : Наука, 2006. — 607 с.
- 72.Ankan A., Panda A. pgmpy: Probabilistic Graphical Models using Python // Proceedings of the 14th Python in Science Conference (SciPy 2015). — 2015. — P. 6–11.
- 73.Schwarz G. Estimating the Dimension of a Model // Annals of Statistics. — 1978. — Vol. 6, № 2. — P. 461–464.
- 74.Артемов А. А. Модели машинного обучения в FMCG-ритейле: архитектура и эксплуатация в условиях высокой изменчивости данных // Вестник науки. — 2025. — URL: <https://cyberleninka.ru/article/n/modeli-mashinnogo-obucheniya-v-fmcg-riteyle-arhitektura-i-ekspluatatsiya-v-usloviyah-vysokoy-izmenchivosti-dannyh> (дата обращения: 19.05.2026).
- 75.Сергеев С. А. Управление запасами в условиях маркетплейсов: почему не работают стандартные методы? // Вестник Академии знаний. — 2024. — URL: <https://cyberleninka.ru/article/n/upravlenie-zapasami-v-usloviyah-marketpleysov-pochemu-ne-rabotayut-standartnye-metody> (дата обращения: 19.05.2026).
- 76.Гилязов Р. А., Турдаков Д. Ю. Активное обучение и краудсорсинг: обзор методов оптимизации разметки данных // Труды Института системного программирования РАН. — 2018. — Т. 30, № 2. — С. 215–250. — DOI: 10.15514/ISPRAS-2018-30(2)-11. — URL: <https://cyberleninka.ru/article/n/aktivnoe-obuchenie-i-kraudsorsing-obzor-metodov-optimizatsii-razmetki-dannyh> (дата обращения: 19.05.2026).
- 77.Хорошевский В. Ф. Пространства знаний в сети Интернет и Semantic Web (Часть 1) // Искусственный интеллект и принятие решений. — 2008. — URL: [http://aidt.ru/index.php?Itemid=114&catid=43&id=137&lang=ru&option=com\\_content&view=article](http://aidt.ru/index.php?Itemid=114&catid=43&id=137&lang=ru&option=com_content&view=article) (дата обращения: 19.05.2026).

78. Brown T. B., Mann B., Ryder N. et al. Language Models are Few-Shot Learners // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — P. 1877–1901. — URL: <https://arxiv.org/abs/2005.14165>.
79. Anthropic. Claude 4 family : model card. — Anthropic, 2025. — URL: <https://www.anthropic.com> (дата обращения: 03.05.2026).
80. Radford A., Kim J. W., Hallacy C. et al. Learning Transferable Visual Models From Natural Language Supervision // Proceedings of the 38th International Conference on Machine Learning (ICML 2021). — PMLR, 2021. — P. 8748–8763. — URL: <https://arxiv.org/abs/2103.00020>.
81. USDA FoodData Central : public nutrient database, SR Legacy release. — U. S. Department of Agriculture, Agricultural Research Service. — URL: <https://fdc.nal.usda.gov> (дата обращения: 03.05.2026).
82. PhoneDB : open phone specs database. — URL: <https://phonedb.net> (дата обращения: 03.05.2026).
83. Settles B. Active Learning Literature Survey. — University of Wisconsin–Madison, 2010. — Computer Sciences Technical Report 1648. — URL: <https://burrsettles.com/pub/settles.activelearning.pdf>.